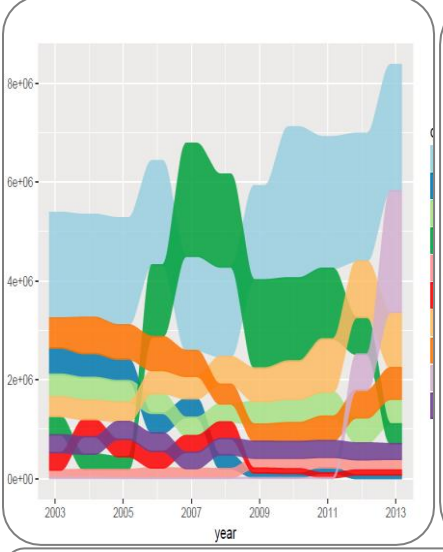
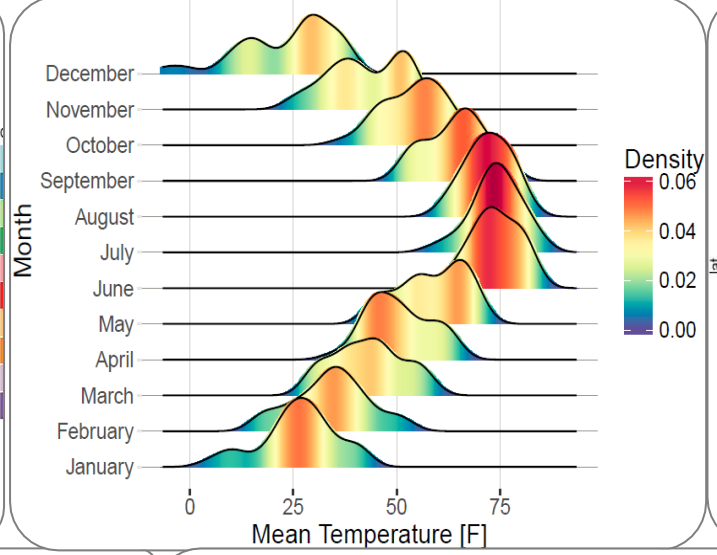
澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

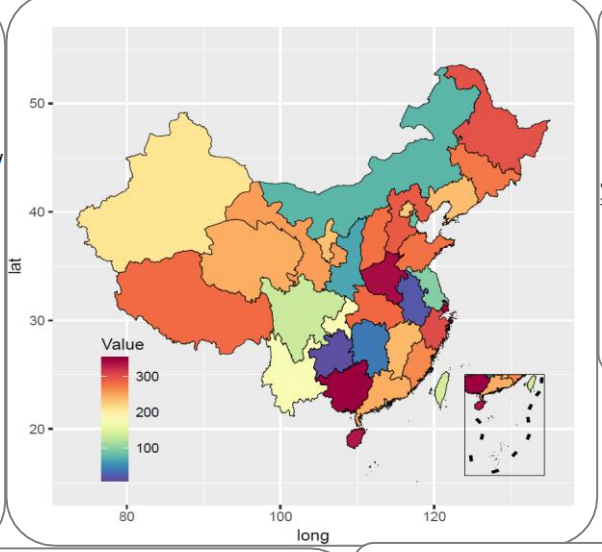
Foundation of Data Visualization

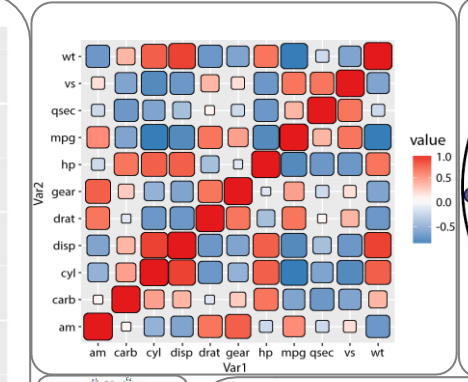
Jie Zhang
Faculty of Applied Sciences, Macao Polytechnic University
Email: jpeter.zhang@mpu.edu.mo
Personal Profile: easy-shu.github.io

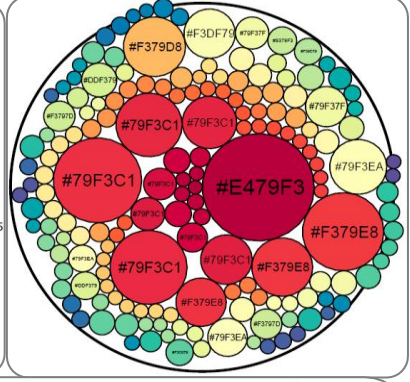


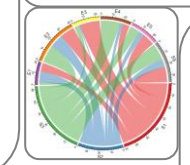

year

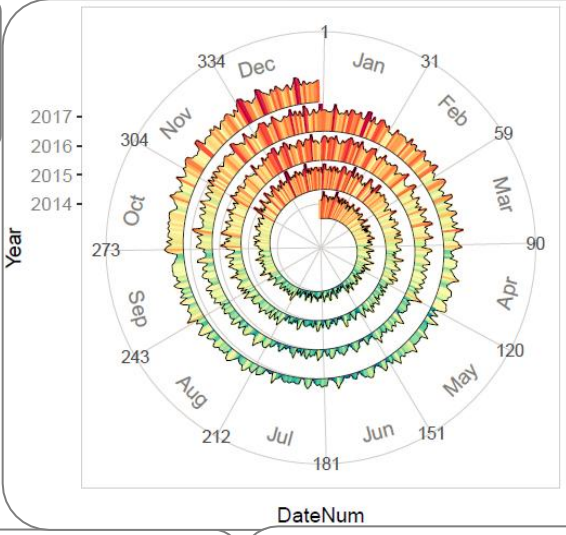

December
November
October
September
August
July
June
May
April
March
February
January
Month
Density
0.06
0.04
0.02
0.00
Mean Temperature [F]

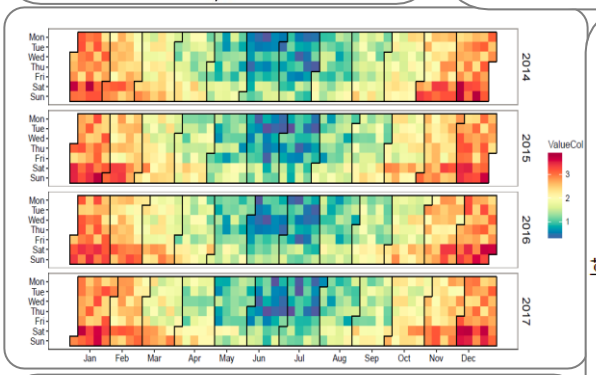

lat
long
Value

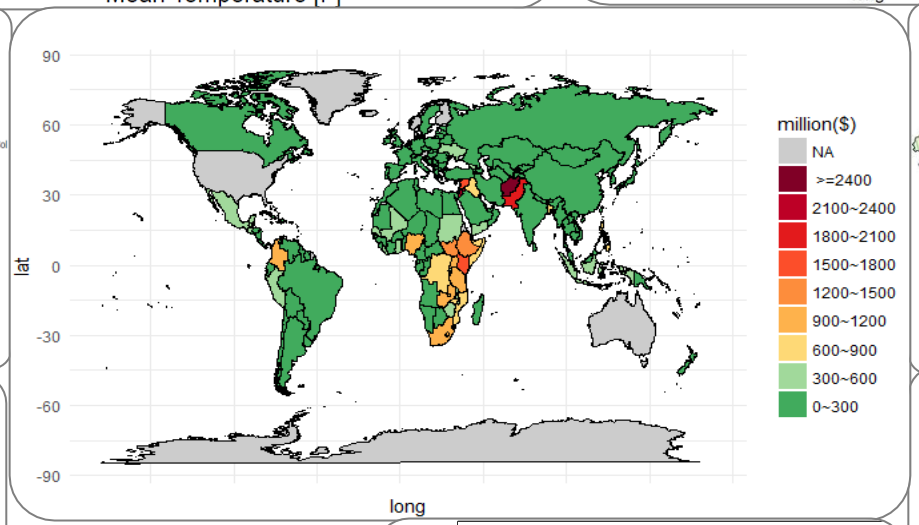

wt
vs
qsec
mpg
hp
gear
drat
disp
cyl
carb
am
Var2
Var1
value

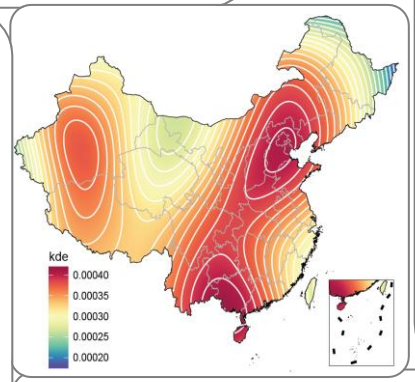

#F379D8
#F3DF79
#79F37F
#79F3C1
#E479F3
#79F3EA
#F379E8

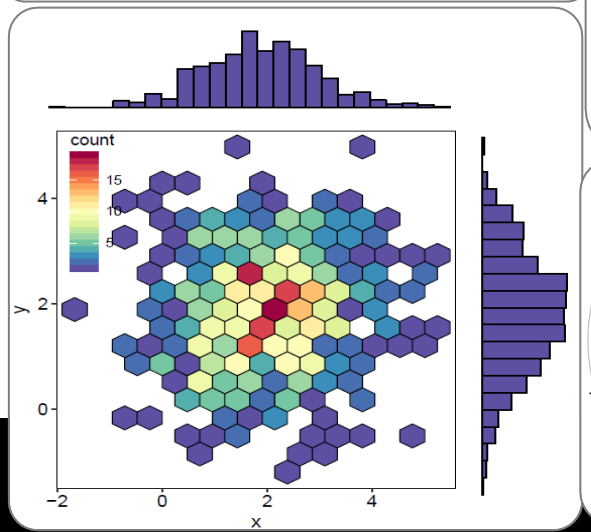


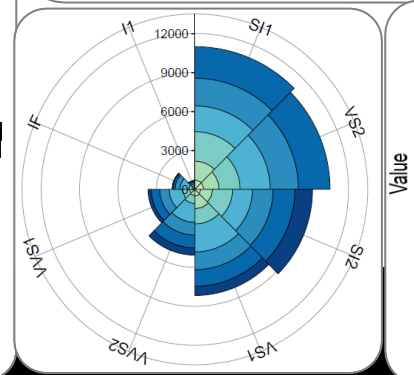

Year
DateNum
ValueCol
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec

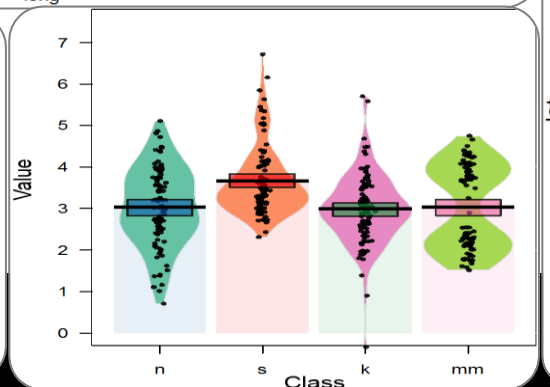

2014
2015
2016
2017
ValueCol

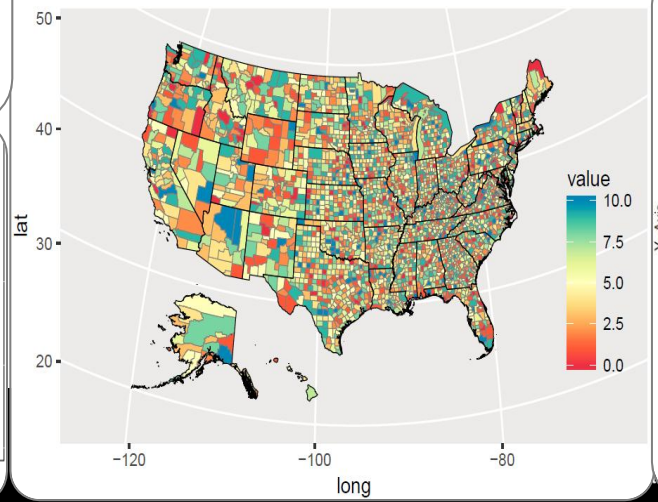

lat
long
million($)
NA
>=2400
2100~2400
1800~2100
1500~1800
1200~1500
900~1200
600~900
300~600
0~300

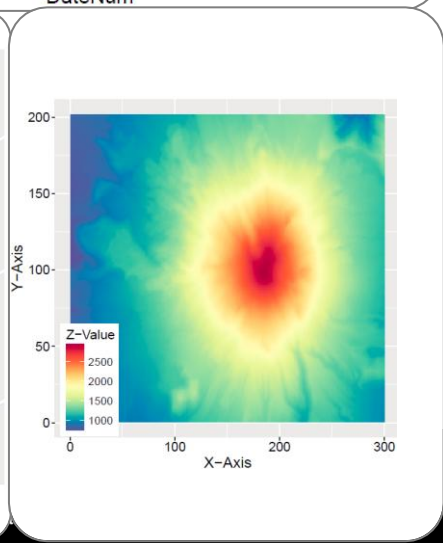

kde


count
x
y


I1
SI1
VS2
SI2
VSI
VVS2
VVS1
IF


Value
Class
n
s
k
mm


lat
long
value


Y-Axis
X-Axis
Z-Value

Python

Python's flexibility, ease of use, extensive library support, and interactivity make it a powerful choice for data visualization tasks, whether for exploratory data analysis, dashboards, reports, or presentations.

matplotlib Version 3.1.1
A Python 2D plotting library

Seaborn
Statistical data visualization

plotnine
A Grammar of Graphics for Python

Cartopy
Geographic information visualization library

Bokeh
An interactive visualization library

folium
Geographic information visualization library-leaflet.js

Pandas' builtin-plotting
Powerful Python data analysis toolkit

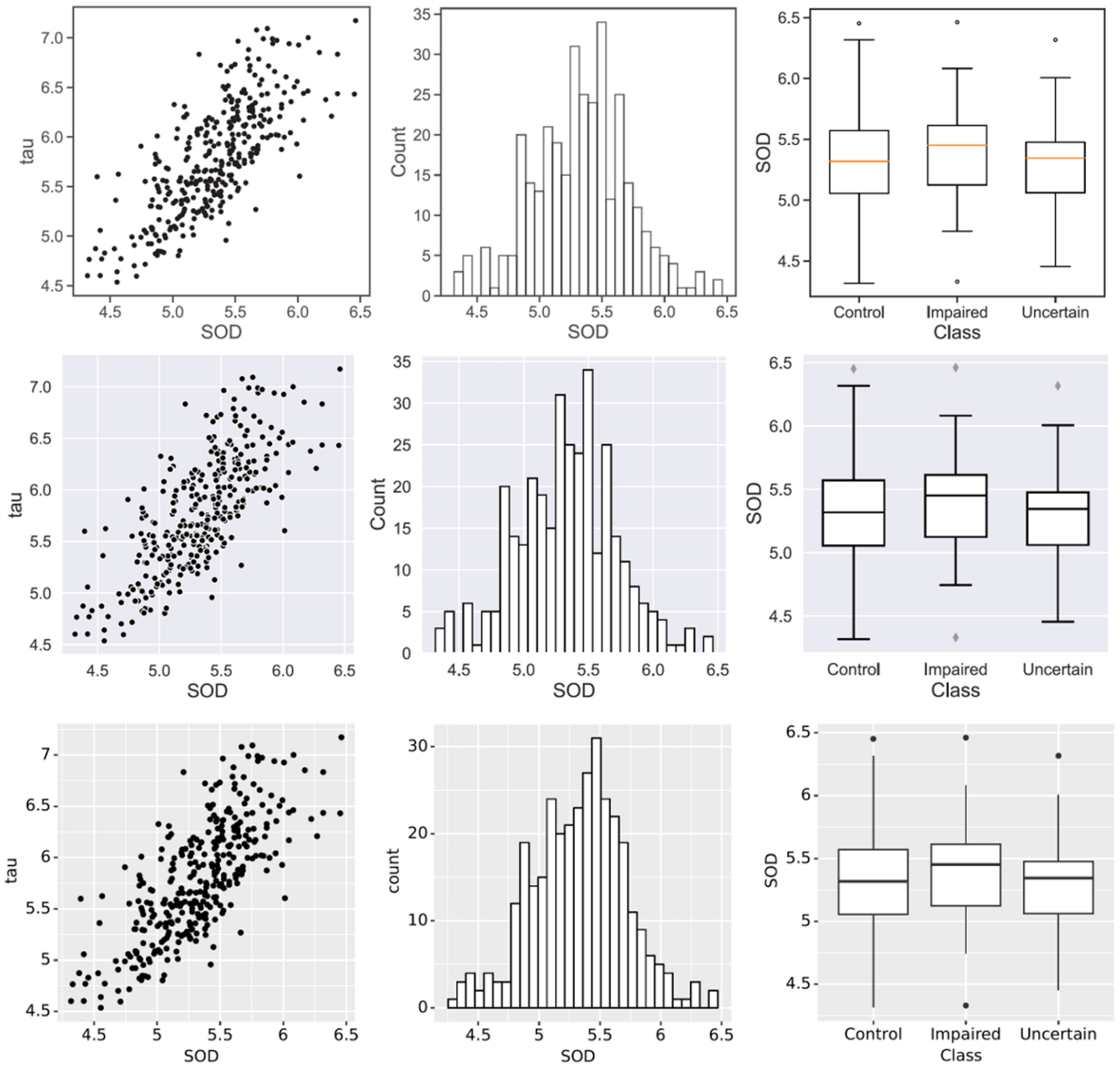
plotly
Open Source Graphing Libraries

pyecharts
A Python Echarts Plotting Library

https://www.youtube.com/watch?v=4O_o53ag3ag



Matplotlib
Seaborn
Plotnine



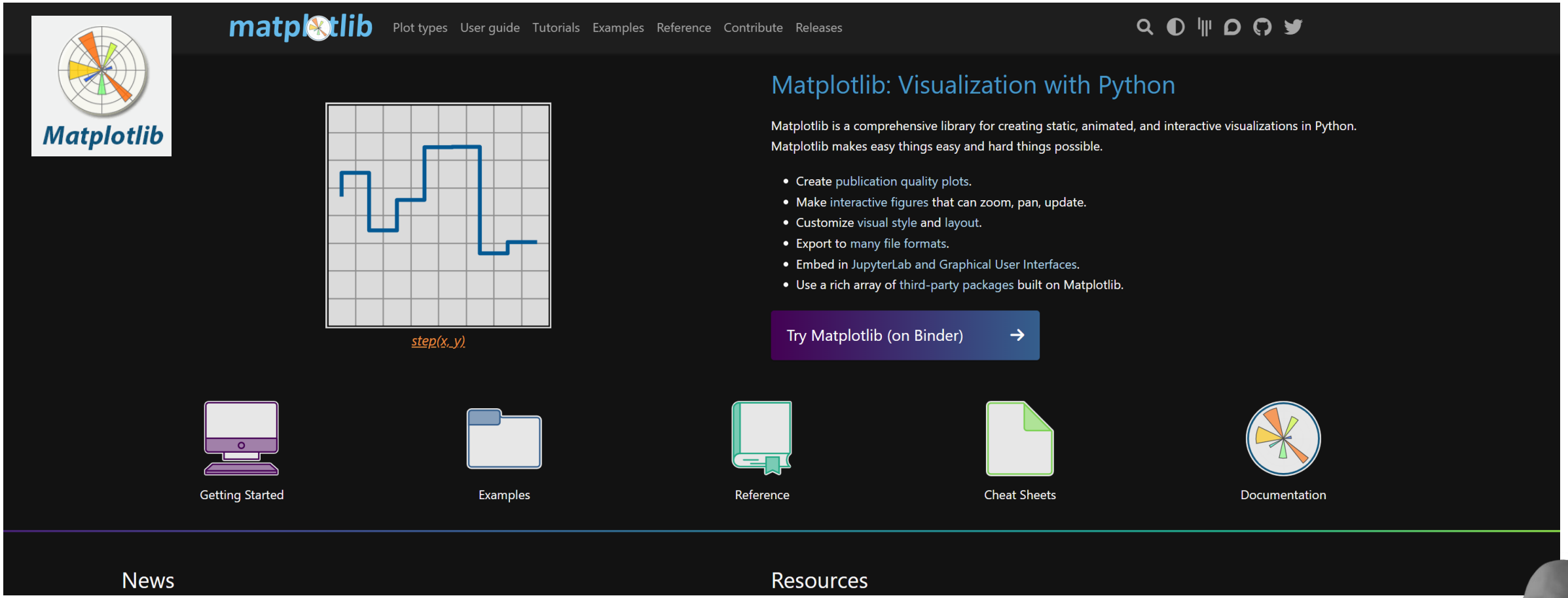
Package	Scatter plot	histgram plot	box plot
matplotlib	plt.scatter(df['SOD'], df['tau'], c='black', s=15, marker='o')	plt.hist(df['SOD'], 30, density= False, facecolor='w',edgecolor="k")	labels=np.unique(df['Class']) all_data = [df[df['Class']==label]['SOD'] for label in labels] plt.boxplot(all_data, widths=0.6, notch= False,labels=labels)
Seaborn	sns.relplot(x="SOD", y="tau", data=df,color='k')	sns.distplot(df['SOD'], kde=False, bins=30, hist_kws=dict(edgecolor="k", facecolor="w",linewidth=1,alpha=1))	sns.boxplot(x="Class", y="SOD", data= df, width =0.6,palette=['w'])
plotnine	(ggplot(df, aes(x='SOD',y='tau')) + geom_point())	(ggplot(df, aes(x='SOD')) + geom_ histogram(bins=30,colour="black",fil l="white"))	(ggplot(df, aes(x='Class',y='SOD'))+ geom_boxplot(show_legend=False))

Matplotlib

Chinese Website: https://www.matplotlib.org.cn/
Chinese Website: https://www.osgeo.cn/matplotlib/
Official website: https://matplotlib.org/stable/index.html

https://matplotlib.org/

Matplotlib is a popular Python library used for creating static, interactive, and animated plots and visualizations. It is widely used for data visualization due to its flexibility and extensive capabilities.



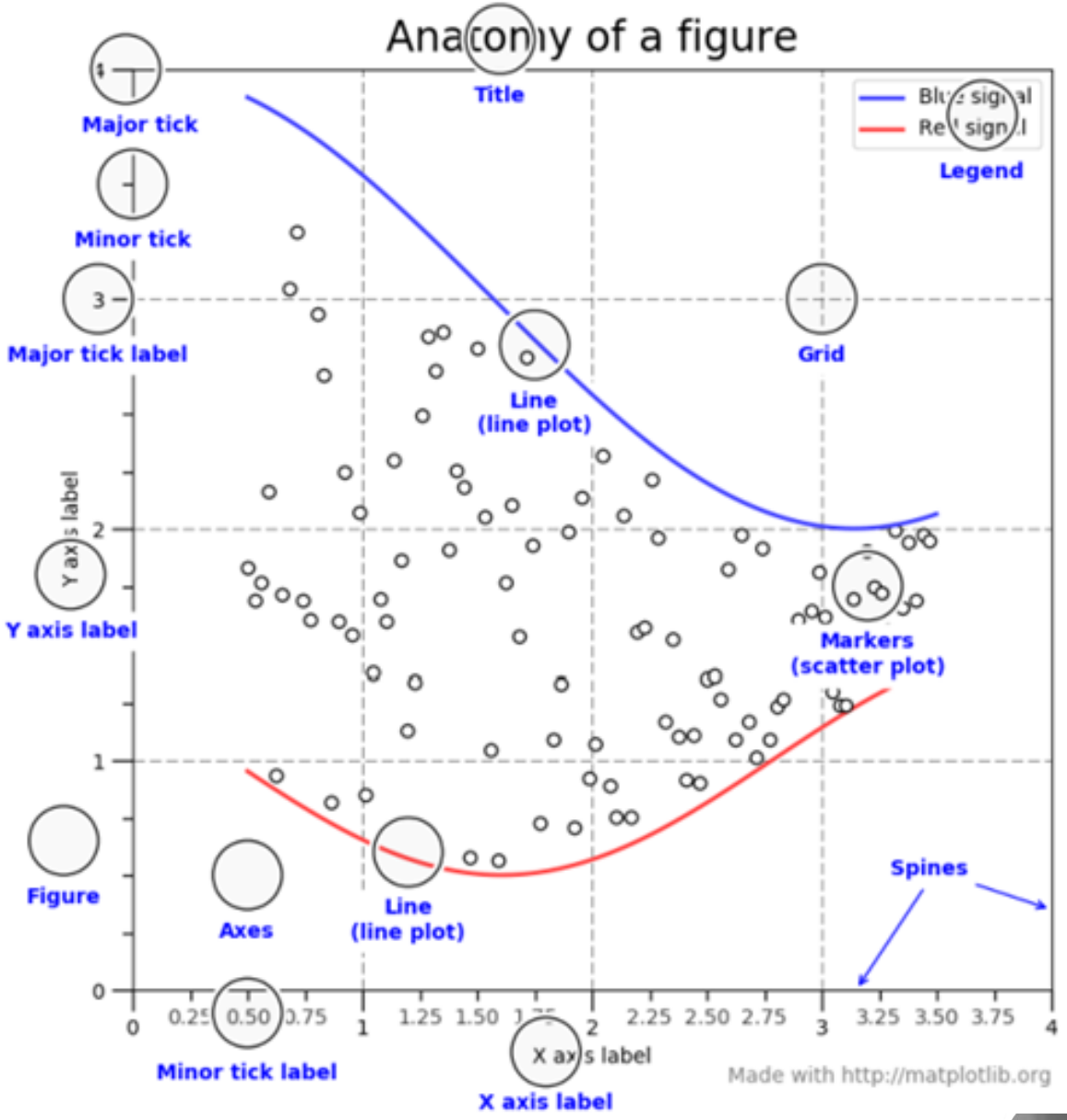
Matplotlib

In the object-oriented plotting library of Matplotlib, pyplot serves as a convenient interface with a commonly used calling convention:

import matplotlib.pyplot as plt

Graphic objects and elements

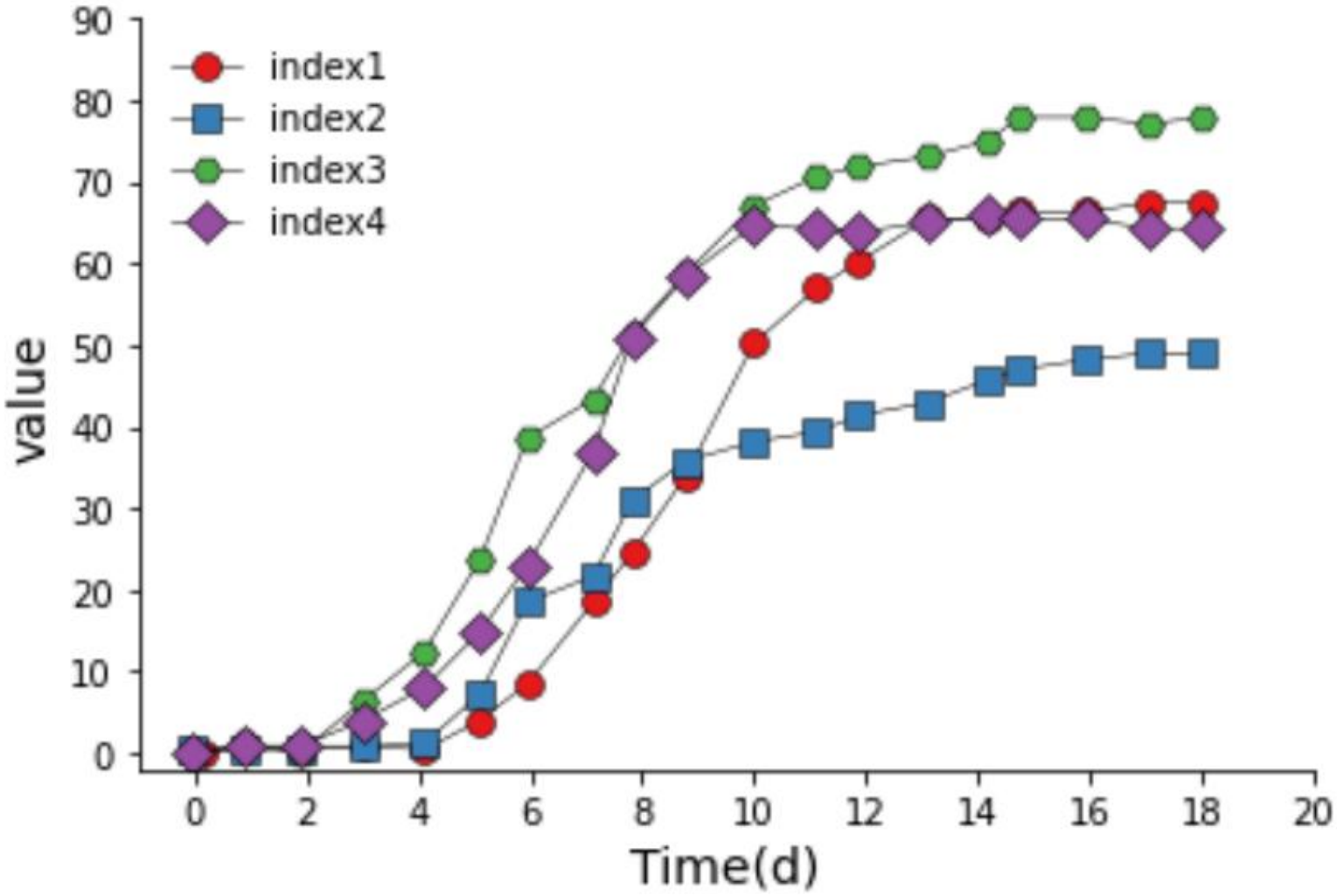
- ☐ Figure
- ☐ Axes
- ☐ Title
- ☐ Legend
- ☐ Major Tick, Minor Tick
- ☐ Major Tick Label, Minor Tick Label,
- ☐ Y-Axis Label, X-Axis Label,
- ☐ Line, Markers,
- ☐ Grid.



Matplotlib: Scatter plot with a line connecting markers.

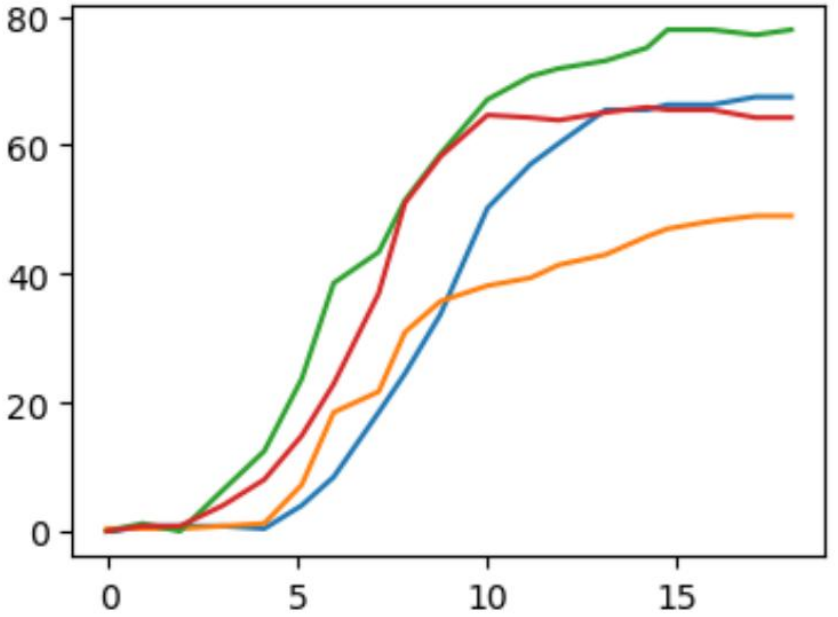
MappingAnalysis_Data.csv

	A	B	C	D
1		Time	variable	value
2	1	-0.0495	0%(Control)	0
3	2	0.891089	0%(Control)	0.803213
4	3	1.881188	0%(Control)	0.803213
5	4	3.019802	0%(Control)	0.803213
6	5	4.108911	0%(Control)	0.401606
7	6	5.09901	0%(Control)	4.016064
8	7	5.940594	0%(Control)	8.433735
9	8	7.128713	0%(Control)	18.4739
10	9	7.821782	0%(Control)	24.49799
11	10	8.762376	0%(Control)	33.73494
12	11	10	0%(Control)	50.2008
13	12	11.13861	0%(Control)	57.02811
14	13	11.88119	0%(Control)	60.24096
15	14	13.11881	0%(Control)	65.46185
16	15	14.20792	0%(Control)	65.46185
17	16	14.75248	0%(Control)	66.26506
18	17	15.94059	0%(Control)	66.26506
19	18	17.07921	0%(Control)	67.46988
20	19	18.0198	0%(Control)	67.46988
21	20	-0.0495	1%	0.401606
22	21	0.891089	1%	0.401606
23	22	1.881188	1%	0.401606
24	23	3.019802	1%	0.803213
25	24	4.108911	1%	1.204819
26	25	5.09901	1%	7.228916
27	26	5.940594	1%	18.4739
28	27	7.128713	1%	21.68675
29	28	7.821782	1%	30.92369
30	29	8.762376	1%	35.74297
31	30	10	1%	38.15261
32	31	11.13861	1%	39.35743
33	32	11.88119	1%	41.36546
34	33	13.11881	1%	42.97189



Matplotlib: Scatter plot with a line connecting markers.

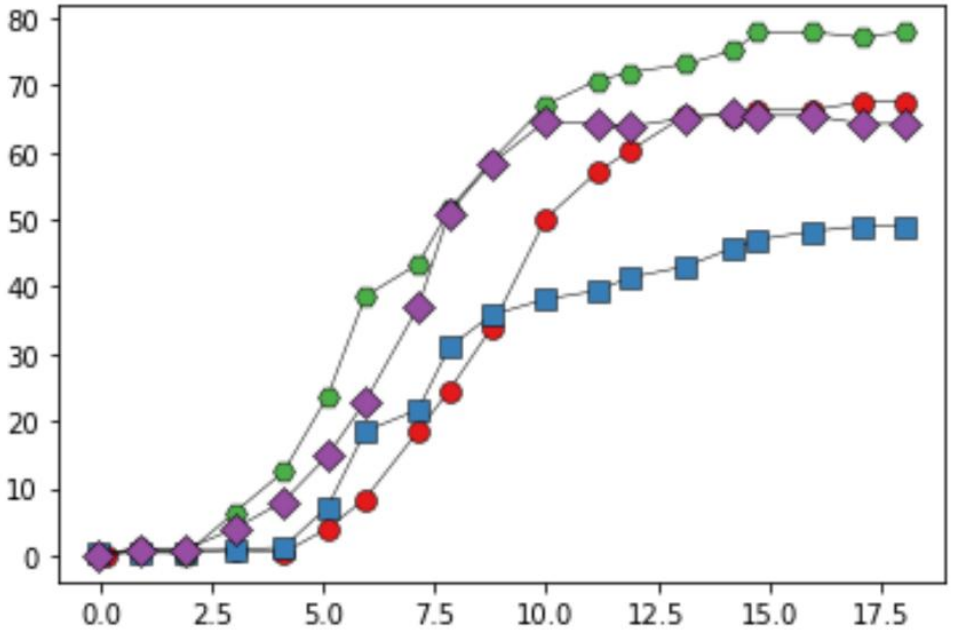
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
df=pd.read_csv("MappingAnalysis_Data.csv")
print(df)
group=np.unique(df.variable)
# Setting the plotting order of chart data series.
fig=plt.figure(figsize=(4,3),dpi=100)
# Set the image size using figsize and the image resolution using dpi
for i in range(0,4):
    temp_df=df[df.variable==group[i]]
    plt.plot(temp_df.Time,temp_df.value)
plt.show()    # Draw a line chart
```



MappingAnalysis_Data.csv

Matplotlib: Scatter plot with a line connecting markers.

```
colors=['#e41a1c','#377eb8','#4daf4a','#984ea3']
markers=['o','s','H','D']
for i in range(0,4):
      plt.plot(df[df.variable==group[i]].Time,
               df[df.variable==group[i]].value,
               marker=markers[i],
               markerfacecolor=colors[i],markersize=8,
               markeredgewidth=0.5,color='k',
               linewidth=0.5,linestyle='-',
               label=group[i])
plt.show()
```



MappingAnalysis_Data.csv

Matplotlib: Scatter plot with a line connecting markers.

```
colors=['#e41a1c','#377eb8','#4daf4a','#984ea3']
markers=['o','s','H','D']
for i in range(0,4):
      plt.plot(df[df.variable==group[i]].Time,
               df[df.variable==group[i]].value,
               marker=markers[i],
               markerfacecolor=colors[i],markersize=8,
               markeredgewidth=0.5,color='k',
               linewidth=0.5,linestyle='-',
               label=group[i])
plt.show()
```

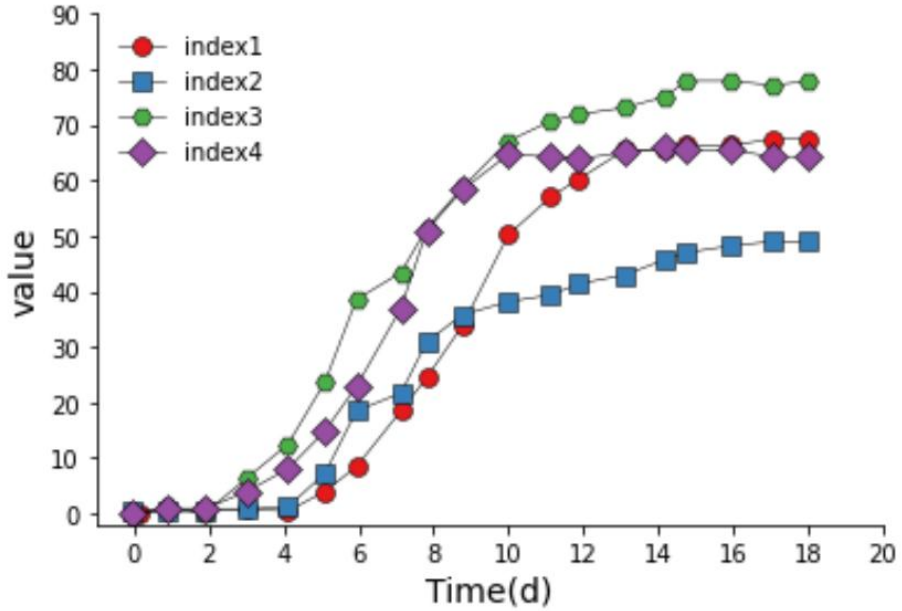
Line Plot with marks: plt.plot()

Parameter	Meaning	Common Values
x, y	X-axis, Y-axis data	Sequence data
marker	Data point shape	'o', 's', '^', 'D'(diamond)
markerfacecolor	Marker fill color	Color string
markeredgewidth	Marker edge width	0.5, 1.0
markersize	Marker size	4, 8, 10
color / c	Line color	'k'(black), 'r'(red), 'b'(blue)
linewidth	Line width	0.5, 1.0, 2.0
linestyle	Line style	'-'(solid), '--'(dashed), ':'(dotted), '-.'(dash-dot)
label	Legend name	String, used with plt.legend()



Matplotlib: Scatter plot with a line connecting markers.

```
plt.xlabel('Time(d)',fontsize=14)
plt.ylabel('value',fontsize=14)
plt.xlim(-1,20)
plt.ylim(-2,90)
plt.xticks(np.linspace(0,20,11,endpoint=True),fontsize=10)
plt.yticks(np.linspace(0,90,10,endpoint=True),fontsize=10)
ax=plt.gca()
ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
plt.legend()
plt.show()
```



MappingAnalysis_Data.csv

Matplotlib: Scatter plot with a line connecting markers.

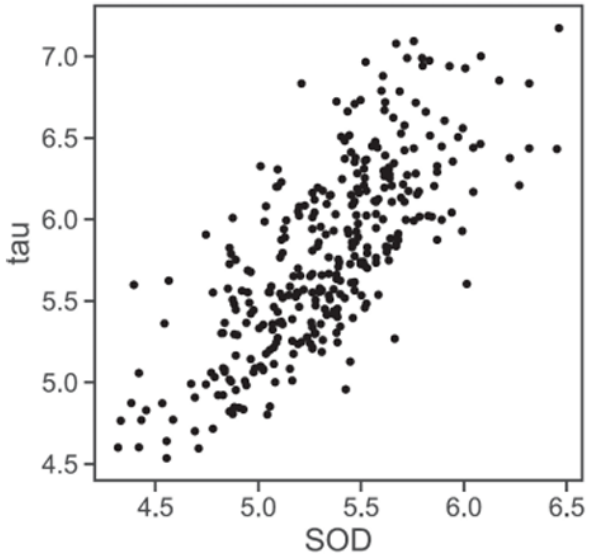
Common Helper Functions

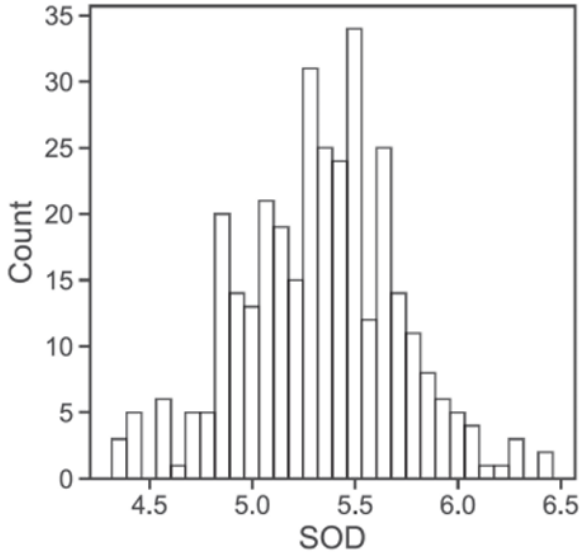
Function	Purpose	Example
plt.figure(figsize=(w,h), dpi=300)	**Create figure, set size & resolution**	**plt.figure(figsize=(4,3), dpi=300)**
plt.gca()	Get current axes object	ax = plt.gca()
plt.xlabel('text', fontsize=14)	**Set X-axis label**	**plt.xlabel('Time(d)', fontsize=14)**
plt.ylabel('text', fontsize=14)	**Set Y-axis label**	**plt.ylabel('value', fontsize=14)**
plt.title('text')	Set chart title	plt.title('Mapping Analysis')
plt.xlim(min, max)	**Set X-axis range**	**plt.xlim(-1, 20)**
plt.ylim(min, max)	**Set Y-axis range**	**plt.ylim(-2, 90)**
plt.xticks(ticks, fontsize=10)	Set X-axis tick positions	plt.xticks(np.linspace(0,20,11), fontsize=10)
plt.yticks(ticks, fontsize=10)	Set Y-axis tick positions	plt.yticks(np.linspace(0,90,10), fontsize=10)
plt.legend(loc='best')	**Auto-place legend**	**plt.legend(loc='best', fontsize=10)**
ax.spines['right'].set_color('none')	Hide right spine	ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')	Hide top spine	ax.spines['top'].set_color('none')
plt.tight_layout()	Auto-adjust spacing	plt.tight_layout()
plt.show()	**Render/display the plot**	**plt.show()**
plt.savefig('file.png')	**Save figure to file**	**plt.savefig('file.pdf', bbox_inches='tight')**

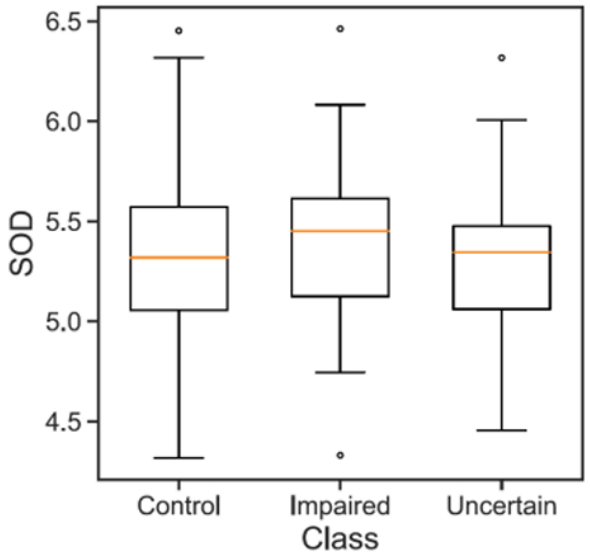


Extended Applications **Facet_Data.csv**

Package	Scatter plot	histgram plot	box plot
matplotlib	plt.scatter(df['SOD'], df['tau'], c='black', s=15, marker='o')	plt.hist(df['SOD'], 30, density= False, facecolor='w',edgecolor="k")	labels=np.unique(df['Class']) all_data = [df[df['Class']==label]['SOD'] for label in labels] plt.boxplot(all_data, widths=0.6, notch= False,labels=labels)







Matplotlibe: figure parameters

Scatter Plot plt.scatter()

Parameter	Meaning	Common Values
x, y	X-axis and Y-axis data	df['col1'], df['col2']
c	Point color	'r', 'g', 'b', 'k'(black), 'w'(white)
s	Point size	Integer, e.g. 15, 50
marker	Point shape	'o'(circle), 's'(square), '^'(triangle), '*'(star)
alpha	Transparency	0.0–1.0, useful for overlapping points

Histogram plt.hist()

Parameter	Meaning	Common Values
x	Input data	df['column']
bins	Number of bars	Integer, e.g. 10, 30
density	Normalize or not	False(count/frequency), True(probability density)
facecolor	Bar fill color	'w'(white), 'b'(blue)
edgecolor	Bar edge color	'k'(black), 'r'(red)

Package	Scatter plot	histgram plot	box plot
matplotlib	plt.scatter(df['SOD'], df['tau'], c='black', s=15, marker='o')	plt.hist(df['SOD'], 30, density= False, facecolor='w',edgecolor="k")	labels=np.unique(df['Class']) all_data = [df[df['Class']==label]['SOD'] for label in labels] plt.boxplot(all_data, widths=0.6, notch= False,labels=labels)



Matplotlibe: figure parameters

Box Plot plt.boxplot()

Parameter	Meaning	Common Values
x	**Input data**	**List/array, e.g. [group1, group2]**
widths	Box width	0.5, 0.6
notch	Notched or not	False(plain box), True(notched CI)
labels	X-axis labels	List, e.g. ['A', 'B', 'C']
showfliers	Show outliers	True(default), False(hide)
vert	Orientation	True(vertical), False(horizontal)

In plt.boxplot(), why do we pass a **list of arrays** like [group1, group2] instead of a single array?

Because plt.boxplot() is designed to draw **multiple boxes side by side in one call**. Each element in the list is **one dataset = one box**.

plt.boxplot([group1, group2, group3])

→ draws **3 boxes** on the same axes, positioned at x = 1, 2, 3.

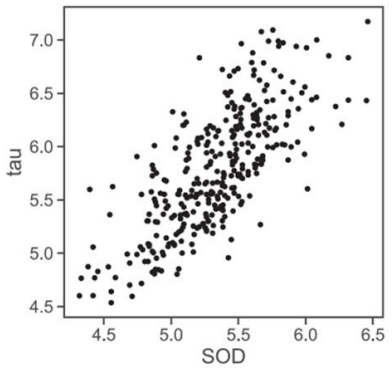
Package	Scatter plot	histgram plot	box plot
matplotlib	plt.scatter(df['SOD'], df['tau'], c='black', s=15, marker='o')	plt.hist(df['SOD'], 30, density= False, facecolor='w',edgecolor="k")	labels=np.unique(df['Class']) all_data = [df[df['Class']==label]['SOD'] for label in labels] plt.boxplot(all_data, widths=0.6, notch= False,labels=labels)



Matplotlibe: figure parameters

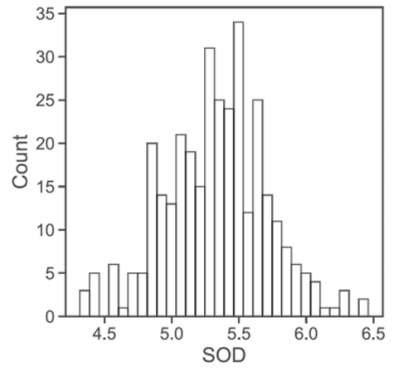
Parameter	Meaning	Common Values
x, y	X-axis and Y-axis data	df['col1'], df['col2']
c	Point color	'r', 'g', 'b', 'k'(black), 'w'(white)
s	Point size	Integer, e.g. 15, 50
marker	Point shape	'o'(circle), 's'(square), '^'(triangle), '*'(star)
alpha	Transparency	0.0–1.0, useful for overlapping points

```
df=pd.read_csv("Facet_Data.csv")
#=========================================
fig = plt.figure(figsize=(5,5),dpi =300)
plt.scatter(df['SOD'], df['tau'], c='black', s=15,  marker='o')
plt.xlabel('SOD')
plt.ylabel('tau')
plt.show()
```



Parameter	Meaning	Common Values
x	Input data	df['column']
bins	Number of bars	Integer, e.g. 10, 30
density	Normalize or not	False(count/frequency), True(probability density)
facecolor	Bar fill color	'w'(white), 'b'(blue)
edgecolor	Bar edge color	'k'(black), 'r'(red)

```
df=pd.read_csv("Facet_Data.csv")
#=========================================
fig = plt.figure(figsize=(5,5),dpi =300)
plt.hist(df['SOD'], 30, density=False, facecolor='w',edgecolor="k")
plt.xlabel('SOD')
plt.ylabel('Count')
plt.show()
```

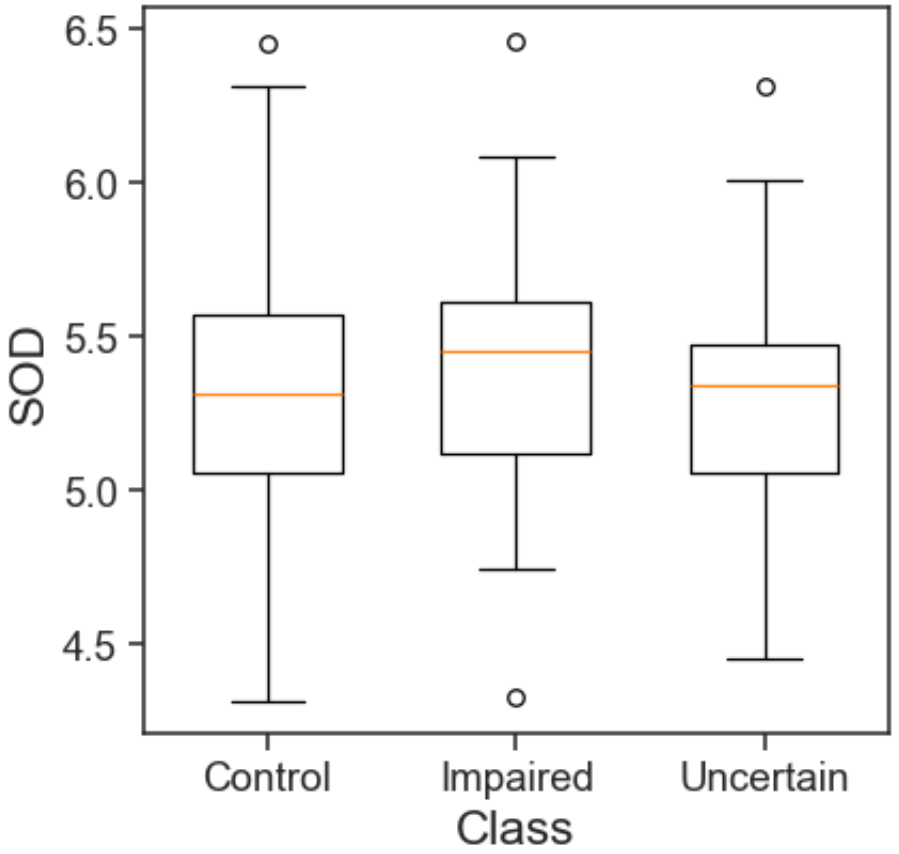




Matplotlibe: figure parameters

Parameter	Meaning	Common Values
x	**Input data**	**List/array, e.g. [group1, group2]**
widths	Box width	0.5, 0.6
notch	Notched or not	False(plain box), True(notched CI)
labels	X-axis labels	List, e.g. ['A', 'B', 'C']
showfliers	Show outliers	True(default), False(hide)
vert	Orientation	True(vertical), False(horizontal)

```
df=pd.read_csv("Facet_Data.csv")
labels=np.unique( df['Class'])
all_data = [df[df['Class']==label]['SOD'] for label in labels]
#=============================================
fig = plt.figure(figsize=(5,5),dpi =300)
plt.boxplot(all_data, widths =0.6, notch=False, labels=labels)
plt.xlabel('Class')
plt.ylabel('SOD')
plt.show()
```



Matplotlib: subplot example

Faceted Chart

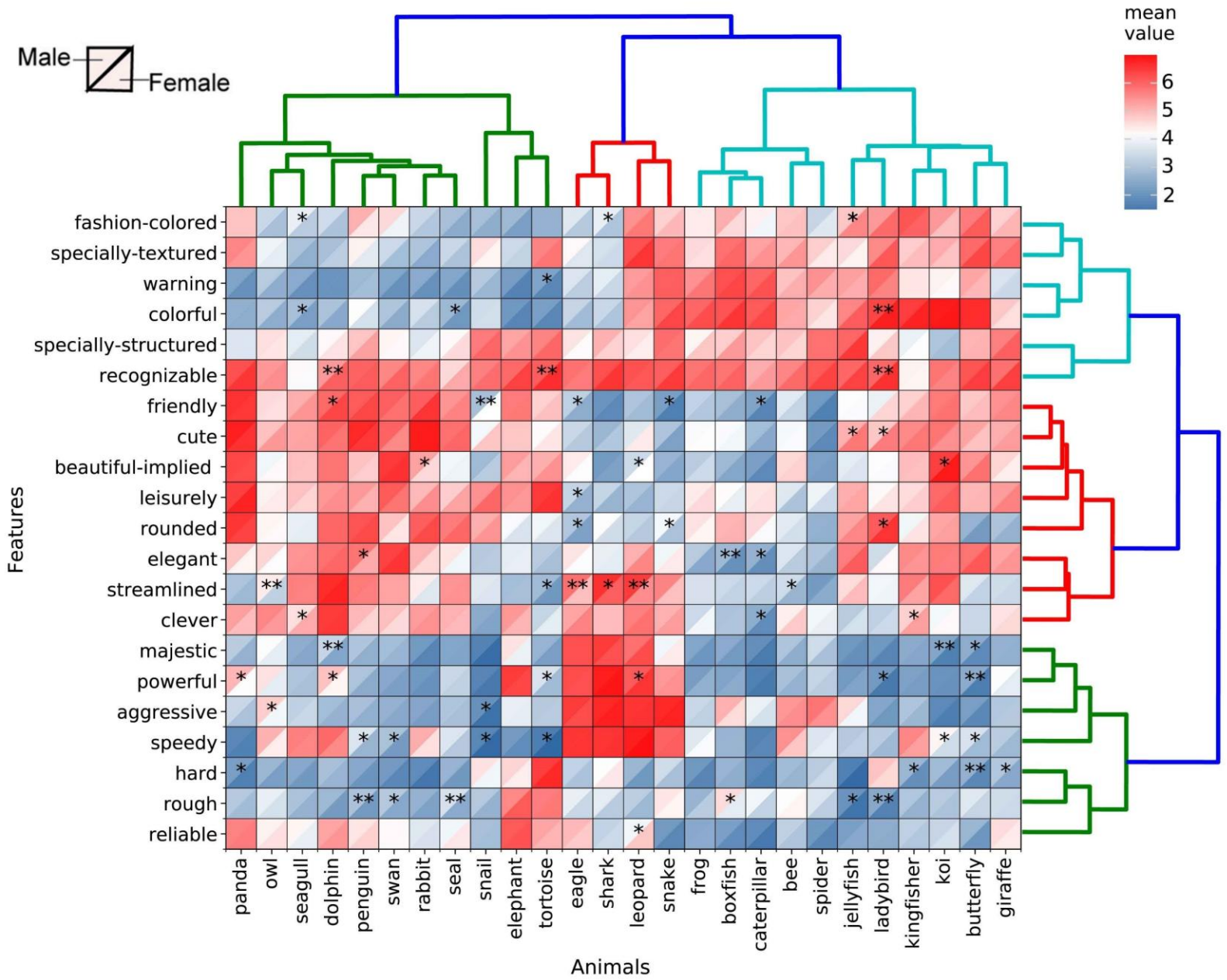


Matplotlib: subplot example

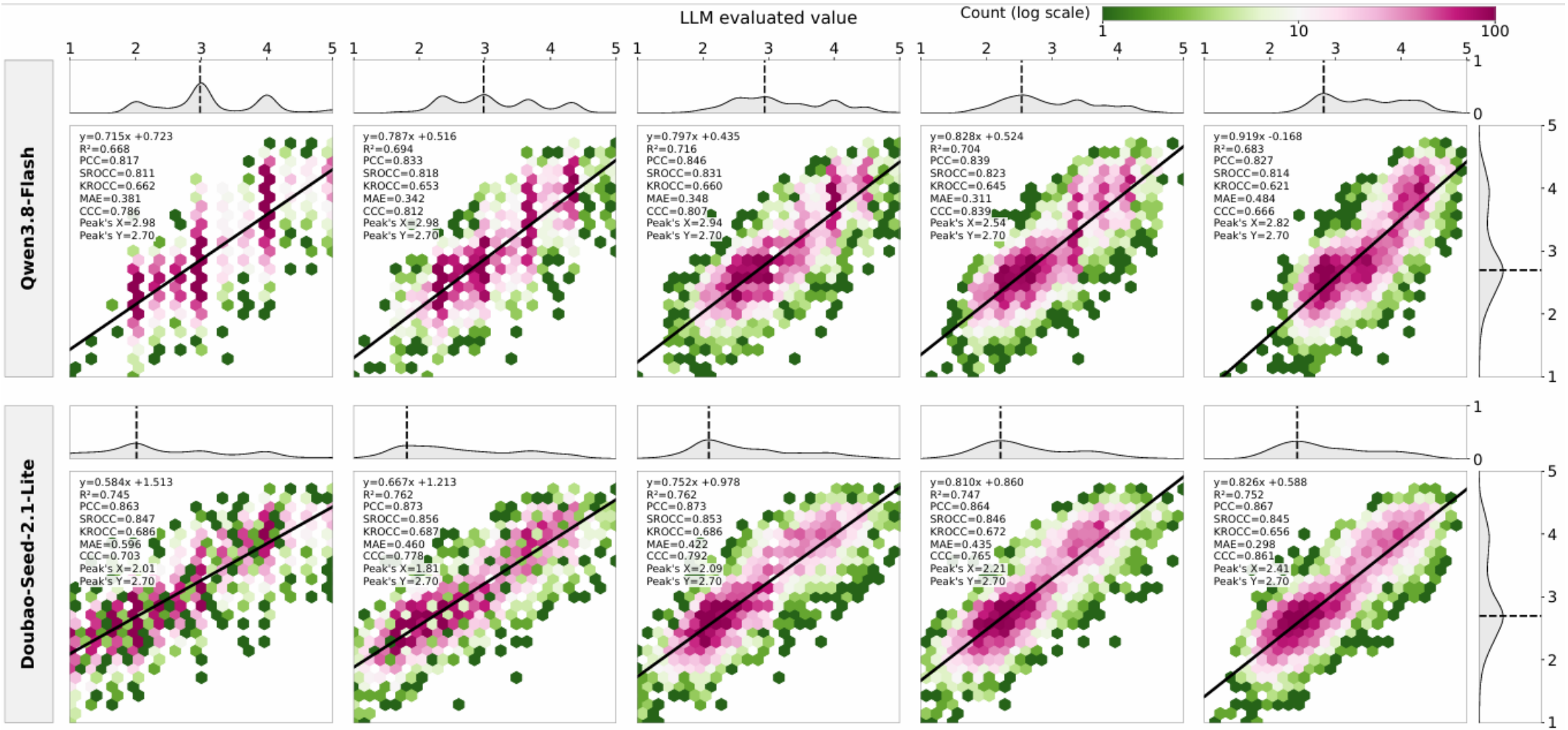
Faceted Chart



Matplotlib: subplot example



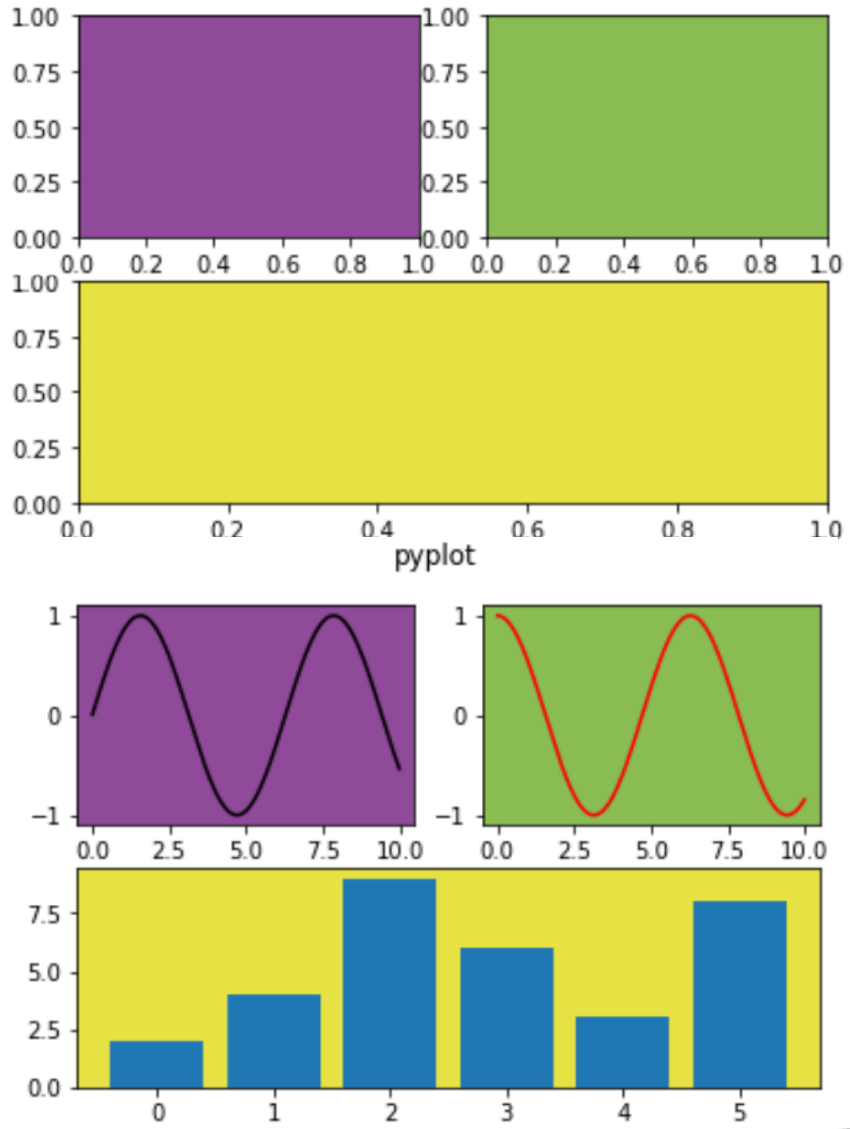
Example


LLM evaluated value
Count (log scale)
1
10
100
Qwen3.8-Flash
y=0.715x +0.723
R²=0.668
PCC=0.817
SROCC=0.811
KROCC=0.662
MAE=0.381
CCC=0.786
Peak's X=2.98
Peak's Y=2.70
y=0.787x +0.516
R²=0.694
PCC=0.833
SROCC=0.818
KROCC=0.653
MAE=0.342
CCC=0.812
Peak's X=2.98
Peak's Y=2.70
y=0.797x +0.435
R²=0.716
PCC=0.846
SROCC=0.831
KROCC=0.660
MAE=0.348
CCC=0.807
Peak's X=2.94
Peak's Y=2.70
y=0.828x +0.524
R²=0.704
PCC=0.839
SROCC=0.823
KROCC=0.645
MAE=0.311
CCC=0.839
Peak's X=2.54
Peak's Y=2.70
y=0.919x -0.168
R²=0.683
PCC=0.827
SROCC=0.814
KROCC=0.621
MAE=0.484
CCC=0.666
Peak's X=2.82
Peak's Y=2.70
Doubao-Seed-2.1-Lite
y=0.584x +1.513
R²=0.745
PCC=0.863
SROCC=0.847
KROCC=0.686
MAE=0.596
CCC=0.703
Peak's X=2.01
Peak's Y=2.70
y=0.667x +1.213
R²=0.762
PCC=0.873
SROCC=0.856
KROCC=0.687
MAE=0.460
CCC=0.778
Peak's X=1.81
Peak's Y=2.70
y=0.752x +0.978
R²=0.762
PCC=0.873
SROCC=0.853
KROCC=0.686
MAE=0.422
CCC=0.792
Peak's X=2.09
Peak's Y=2.70
y=0.810x +0.860
R²=0.747
PCC=0.864
SROCC=0.846
KROCC=0.672
MAE=0.435
CCC=0.765
Peak's X=2.21
Peak's Y=2.70
y=0.826x +0.588
R²=0.752
PCC=0.867
SROCC=0.845
KROCC=0.656
MAE=0.298
CCC=0.861
Peak's X=2.41
Peak's Y=2.70

Matplotlib: subplot

In a single figure, it is possible to have multiple axes, meaning that there can be multiple subplots within one figure. There are two main ways to create multiple subplots and axes in matplotlib: using the pyplot interface and the object-oriented approach with axes.

```
import matplotlib.pyplot as plt
import numpy as np
plt.figure()
X = np.arange(0.01, 10, 0.01)
plt.subplot(221,facecolor="#8F4B99")
#plt.plot(X,np.sin(X),'k-')
plt.subplot(222,facecolor="#89BD54")
#plt.plot(X,np.cos(X),'r-')
plt.subplot(212,facecolor="#E6E243")
#plt.bar(np.arange(6), np.array([2, 4, 9, 6, 3, 8]))
#plt.savefig('add_subplot1.pdf',format='pdf')
#plt.suptitle('pyplot')
plt.show()
```



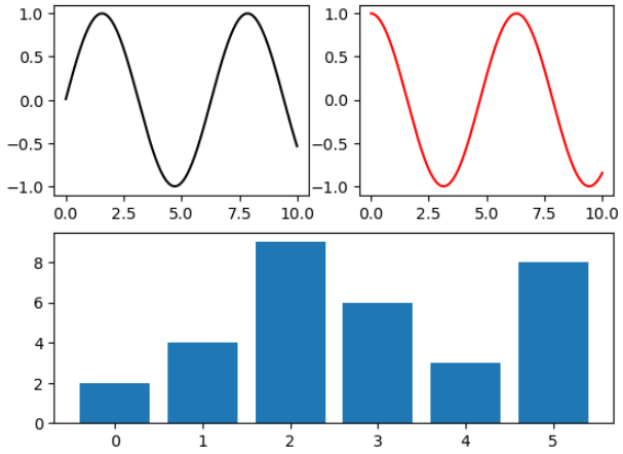
Matplotlib: subplot

Aspect	Syntax / Rule	Example	Key Notes
3-Digit Syntax	plt.subplot(nrows, ncols, index) (combined into one integer)	**plt.subplot(221)**	2 rows, 2 cols, 1st position (top-left).
Comma Syntax (Recommended)	plt.subplot(nrows, ncols, index) (separated by commas)	**plt.subplot(2, 2, 1)**	Use this when rows or cols > 9 (e.g., subplot(10, 2, 1)).
Position Indexing	Counts left-to-right, top-to-bottom.	In a 2x2 grid: 1 = Top-Left 2 = Top-Right 3 = Bottom-Left 4 = Bottom-Right	Index must not exceed nrows * ncols

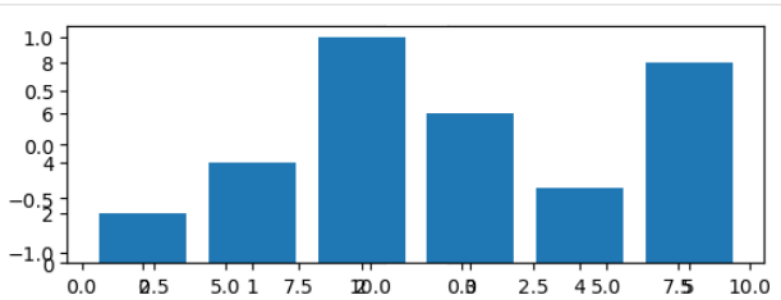


plt.subplot()

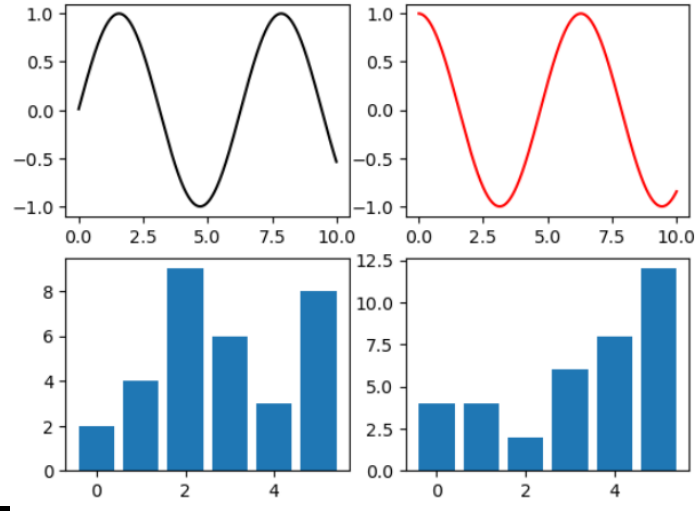
```
plt.figure()
X = np.arange(0.01, 10, 0.01)
plt.subplot(221)
plt.plot(X,np.sin(X),'k-’)
plt.subplot(222)
plt.plot(X,np.cos(X),'r-’)
plt.subplot(212)
plt.bar(np.arange(6), np.array([2, 4, 9, 6, 3, 8]))
plt.show()
```



```
plt.figure()
X = np.arange(0.01, 10, 0.01)
plt.subplot(221)
plt.plot(X,np.sin(X),'k-’)
plt.subplot(222)
plt.plot(X,np.cos(X),'r-’)
plt.subplot(211)
plt.bar(np.arange(6), np.array([2, 4, 9, 6, 3, 8]))
plt.show()
```



```
plt.figure()
X = np.arange(0.01, 10, 0.01)
plt.subplot(221)
plt.plot(X,np.sin(X),'k-’)
plt.subplot(222)
plt.plot(X,np.cos(X),'r-’)
plt.subplot(223)
plt.bar(np.arange(6), np.array([2, 4, 9, 6, 3, 8]))
plt.subplot(224)
plt.bar(np.arange(6), np.array([4, 4, 2, 6, 8, 12]))
plt.show()
```

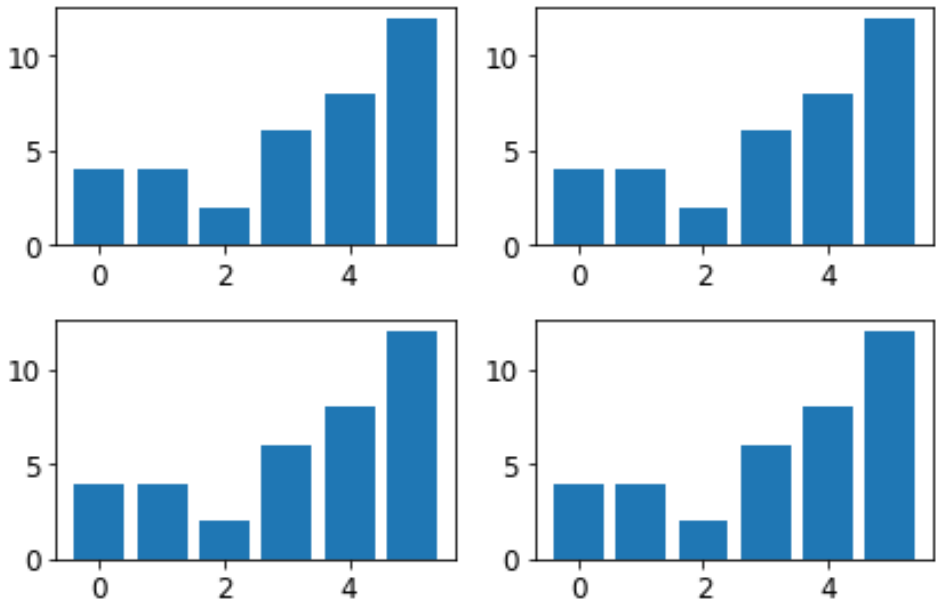




plt.subplot()

```
nrow, ncol = 2, 2
# 生成 [221, 222, 223, 224]
idx_list = [nrow * 100 + ncol * 10 + i for i in range(1, nrow * ncol + 1)]
plt.figure()
for idx in idx_list:
    plt.subplot(idx)
    plt.bar(np.arange(6), np.array([4, 4, 2, 6, 8, 12]))
plt.tight_layout()
plt.show()
```

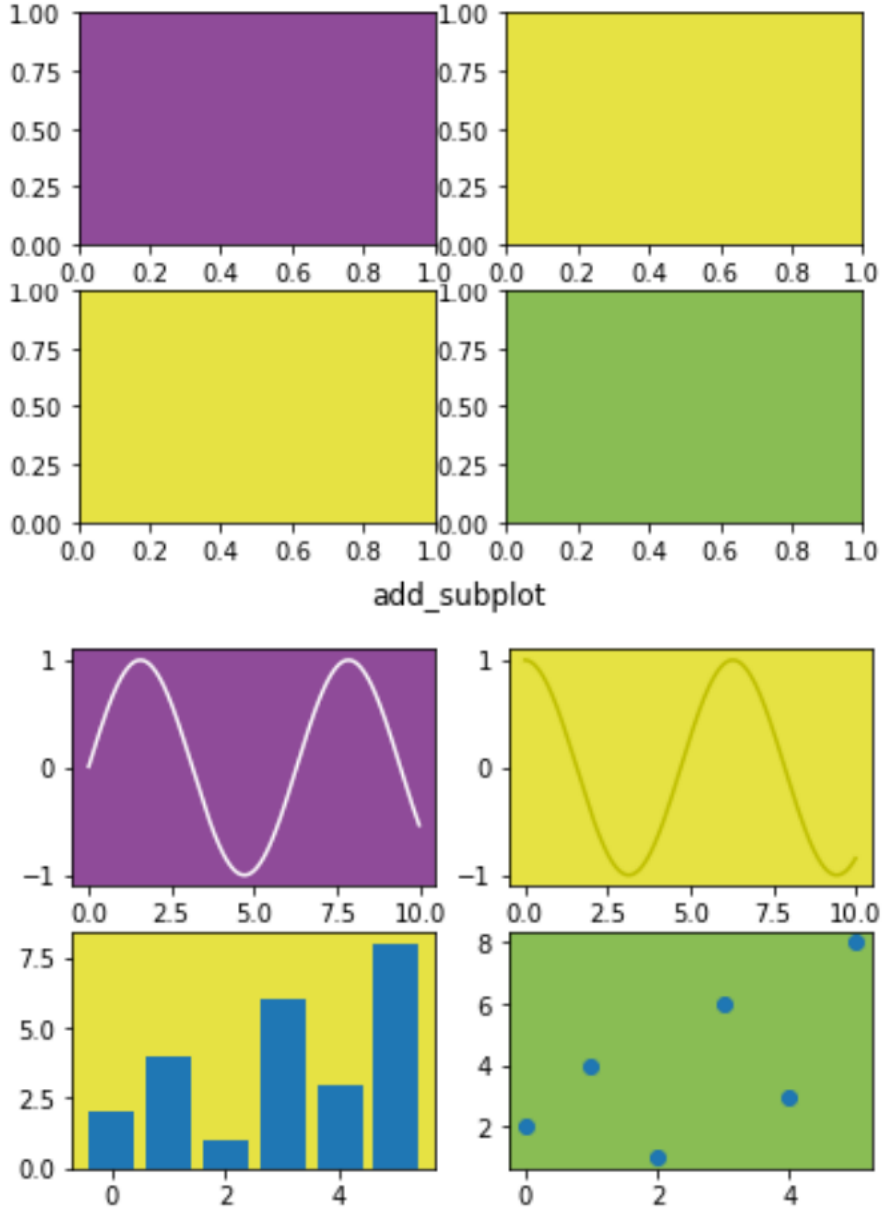
```
nrow, ncol = 2, 2
plt.figure()
for idx in range(4):
    plt.subplot(nrow, ncol ,idx+1)
    plt.bar(np.arange(6), np.array([4, 4, 2, 6, 8, 12]))
plt.tight_layout()
plt.show()
```



Matplotlib: subplot

In a single figure, it is possible to have multiple axes, meaning that there can be multiple subplots within one figure. There are two main ways to create multiple subplots and axes in matplotlib: using the pyplot interface and the object-oriented approach with axes.

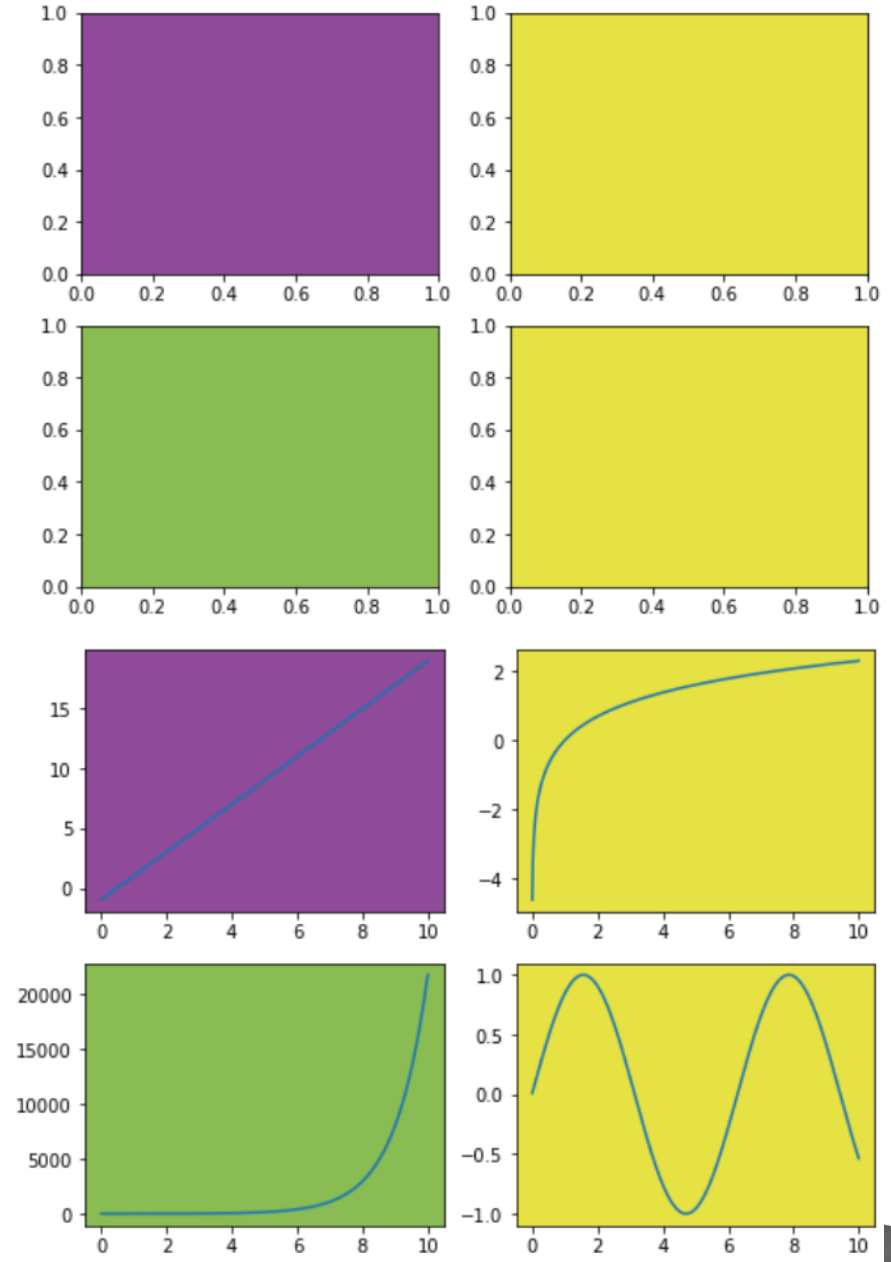
```
import matplotlib.pyplot as plt
import numpy as np
fig = plt.figure()
X = np.arange(0.01, 10, 0.01)
ax1 = fig.add_subplot(221,facecolor="#8F4B99")
#ax1.plot(X, np.sin(X), 'w-')
ax2 = fig.add_subplot(222,facecolor="#E6E243")
#ax2.plot(X,np.cos(X),'y-')
ax3 = fig.add_subplot(223,facecolor='#E6E243')
#ax3.bar(np.arange(6), np.array([2, 4, 1, 6, 3, 8]))
ax4 = fig.add_subplot(224,facecolor='#89BD54')
#ax4.scatter(np.arange(6), np.array([2, 4, 1, 6, 3, 8]))
#fig.suptitle('add_subplot')
#plt.savefig('add_subplot2.pdf',format='pdf')
plt.show()
```



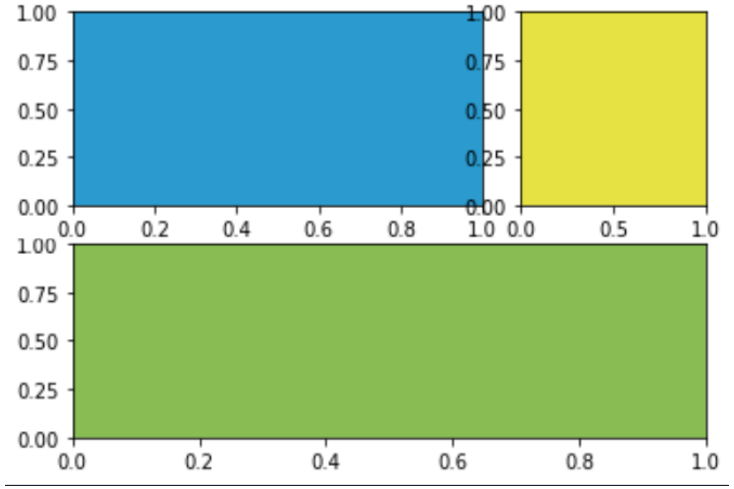
Matplotlib: subplot

In a single figure, it is possible to have multiple axes, meaning that there can be multiple subplots within one figure. There are two main ways to create multiple subplots and axes in matplotlib: using the pyplot interface and the object-oriented approach with axes.

```
import matplotlib.pyplot as plt
import numpy as np
fig,ax=plt.subplots(ncols=2,nrows=2,figsize=(8,6))
X = np.arange(0.01,10,0.01)
#ax[0,0].plot(X,2*X-1)
ax[0,0].set_facecolor("#8F4B99")
#ax[0,1].plot(X,np.log(X))
ax[0,1].set_facecolor("#E6E243")
#ax[1,0].plot(X,np.exp(X))
ax[1,0].set_facecolor("#89BD54")
#ax[1,1].plot(X,np.sin(X))
ax[1,1].set_facecolor("#E6E243")
plt.show()
```



Matplotlib: subplot



In a single figure, it is possible to have multiple axes, meaning that there can be multiple subplots within one figure. There are two main ways to create multiple subplots and axes in matplotlib: using the pyplot interface and the object-oriented approach with axes.

```
import matplotlib.pyplot as plt

import numpy as np

plt.subplot2grid((2,3),(0,0),colspan=2,facecolor='#2B9ACE')

#(2,3): Split into 2 rows and 3 columns; (0,0): The position of the first subplot is in the 0th row and 0th column; colspan=2:

#The 2 columns are merged into a single column；

plt.subplot2grid((2,3),(0,2),facecolor='#E6E243')

plt.subplot2grid((2,3),(1,0),colspan=3,facecolor='#89BD54')

#(1,0): The position of the third subplot is in the 1st row and 0th column; colspan=3: The 3 columns are

#merged into a single column.

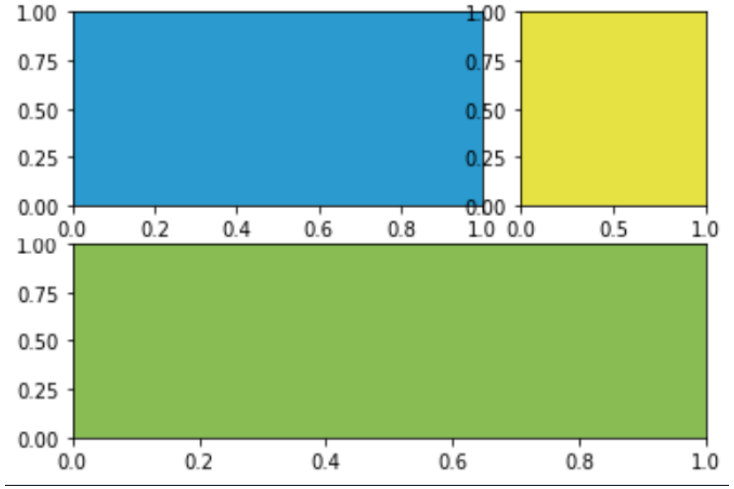
#plt.savefig('add_subplot4.pdf',format='pdf')

plt.show()
```




Matplotlib: subplot

In a single figure, it is possible to have multiple axes, meaning that there can be multiple subplots within one figure. There are two main ways to create multiple subplots and axes in matplotlib: using the pyplot interface and the object-oriented approach with axes.

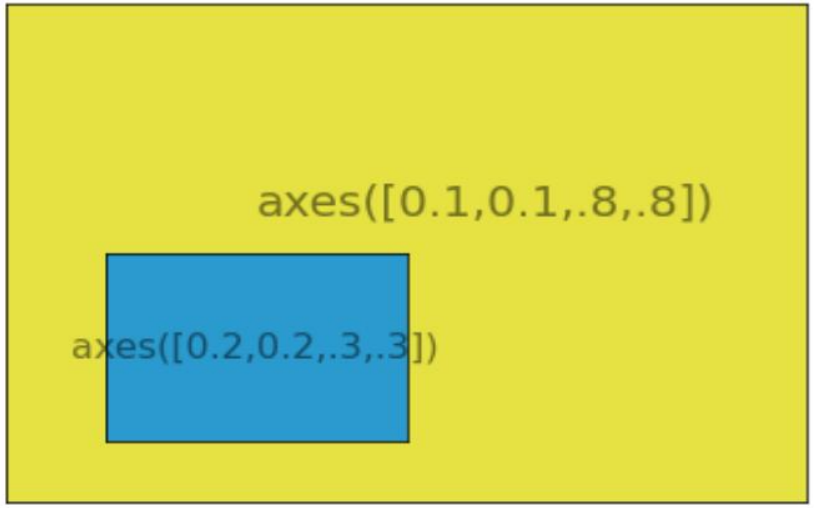



```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
G = gridspec.GridSpec(2,3)
axes1 = plt.subplot(G[0,0:2],facecolor='#2B9ACE')
axes2 = plt.subplot(G[0,2],facecolor='#E6E243')
axes3 = plt.subplot(G[1,:],facecolor='#89BD54')
plt.show()
```




Matplotlib: subplot

In a single figure, it is possible to have multiple axes, meaning that there can be multiple subplots within one figure. There are two main ways to create multiple subplots and axes in matplotlib: using the pyplot interface and the object-oriented approach with axes.

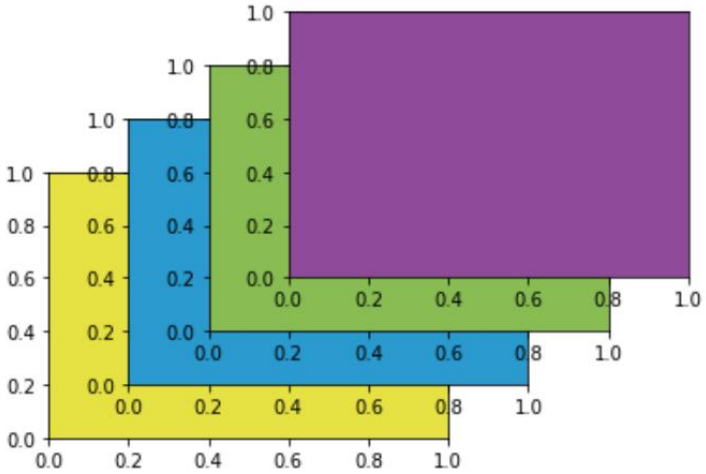


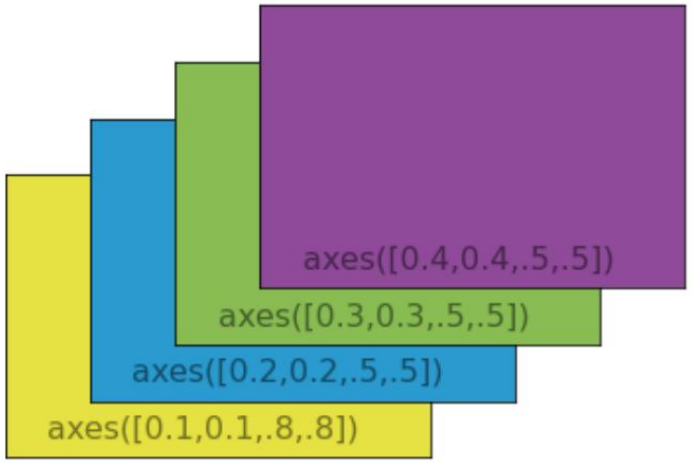

```
import matplotlib.pyplot as plt
plt.axes([0.1,0.1,.8,.8],facecolor='#E6E243')
#plt.xticks([]), plt.yticks([])
#plt.text(0.6,0.6, 'axes([0.1,0.1,.8,.8])',ha='center',va='center',size=20,alpha=.5)
plt.axes([0.2,0.2,.3,.3],facecolor='#2B9ACE')
#plt.xticks([]), plt.yticks([])
#plt.text(0.5,0.5, 'axes([0.2,0.2,.3,.3])',ha='center',va='center',size=16,alpha=.5)
#plt.savefig("axes1.pdf")
#plt.show()
```

Matplotlib: subplot

In a single figure, it is possible to have multiple axes, meaning that there can be multiple subplots within one figure. There are two main ways to create multiple subplots and axes in matplotlib: using the pyplot interface and the object-oriented approach with axes.

```
import matplotlib.pyplot as plt
plt.axes([0.1,0.1,.5,.5],facecolor='#E6E243')
#plt.xticks([]), plt.yticks([])
#plt.text(0.1,0.1, 'axes([0.1,0.1,.5,.5])',ha='left',va='center',size=16,alpha=.5)
plt.axes([0.2,0.2,.5,.5],facecolor='#2B9ACE')
#plt.xticks([]), plt.yticks([])
#plt.text(0.1,0.1, 'axes([0.2,0.2,.5,.5])',ha='left',va='center',size=16,alpha=.5)
plt.axes([0.3,0.3,.5,.5],facecolor='#89BD54')
#plt.xticks([]), plt.yticks([])
#plt.text(0.1,0.1, 'axes([0.3,0.3,.5,.5])',ha='left',va='center',size=16,alpha=.5)
plt.axes([0.4,0.4,.5,.5],facecolor='#8F4B99')
#plt.xticks([]), plt.yticks([])
#plt.text(0.1,0.1, 'axes([0.4,0.4,.5,.5])',ha='left',va='center',size=16,alpha=.5)
#plt.savefig("add_subplot7.pdf")
plt.show()
```





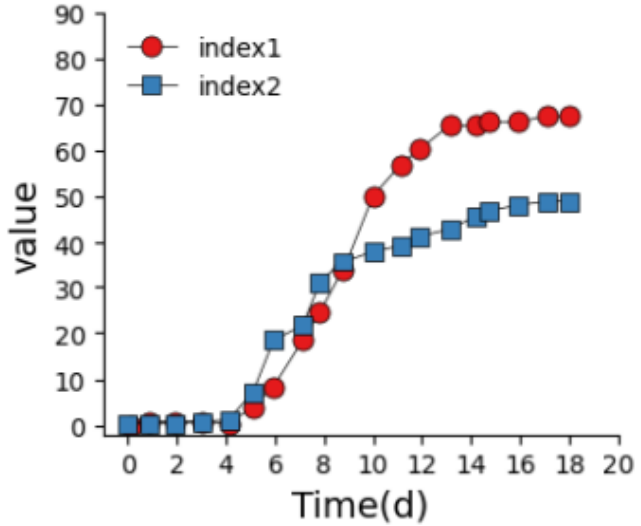
Matplotlib: subplot

```
import matplotlib.pyplot as plt
#import numpy as np
import pandas as pd
import matplotlib as mpl
mpl.rcParams['font.sans-serif']=['SimHei']     #To display Chinese labels properly
mpl.rcParams['axes.unicode_minus']=False     # To display the minus sign normally
plt.style.use('default') #print(plt.style.available)
df=pd.read_csv("MappingAnalysis_Data.csv")
group=np.unique(df.variable)
df1=df[df.variable== group[0]]
df2=df[df.variable==group[1]]
df3=df[df.variable==group[2]]
df4=df[df.variable==group[3]]
# ==========================================================================================
colors=['#e41a1c','#377eb8','#4daf4a','#984ea3']
markers=['o','s','H','D']
labels= group.copy()
#==========================================================================================
```



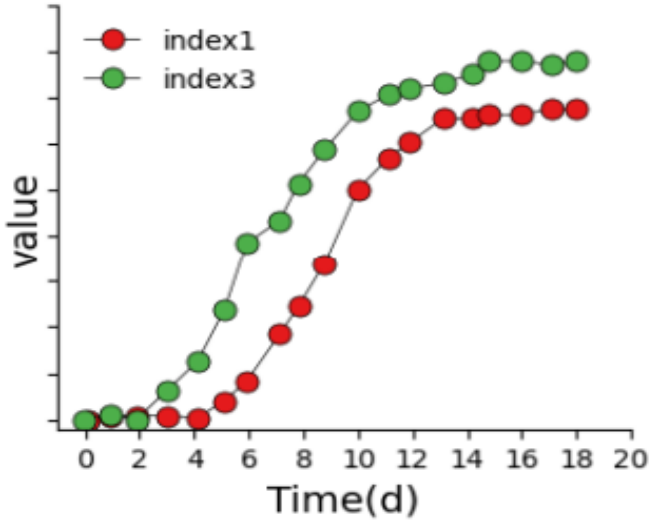
Matplotlib: subplot

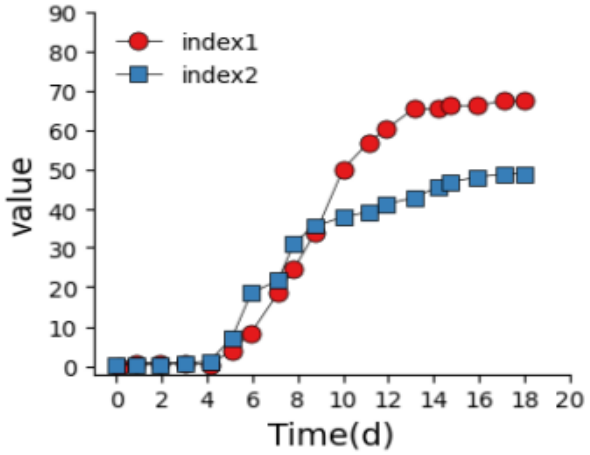
```
fig,(ax0,ax1)= plt.subplots(nrows=1,ncols=2,sharey=True,figsize=(8,3))
#"sharey" indicates sharing the y-axis, and "figsize" sets the size of the image or plot.
ax0.plot(df1.Time, df1.value,
        marker=markers[0], markerfacecolor=colors[0], markersize=8,markeredgewidth=0.5,
        color="k", linewidth=0.5, linestyle="-",label=labels[0])
ax0.plot(df2.Time, df2.value,
        marker=markers[1], markerfacecolor=colors[1], markersize=7,markeredgewidth=0.5,
        color="k", linewidth=0.5, linestyle="-",label=labels[1])
ax0.spines['right'].set_color('none')
ax0.spines['top'].set_color('none')
ax0.set_xlabel('Time(d)',fontsize=14)
ax0.set_ylabel('value',fontsize=14)
ax0.set_xlim(-1,20)
ax0.set_ylim(-2,90)
ax0.set_xticks(np.linspace(0,20,11,endpoint=True))
ax0.set_yticks(np.linspace(0,90,10,endpoint=True))
ax0.legend(loc='upper left',edgecolor='none',facecolor='none')
```

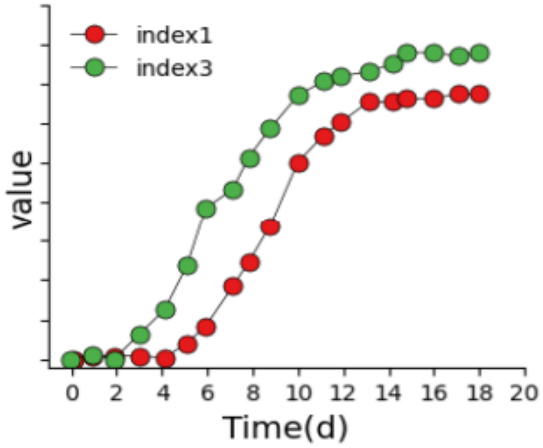


Matplotlib: subplot

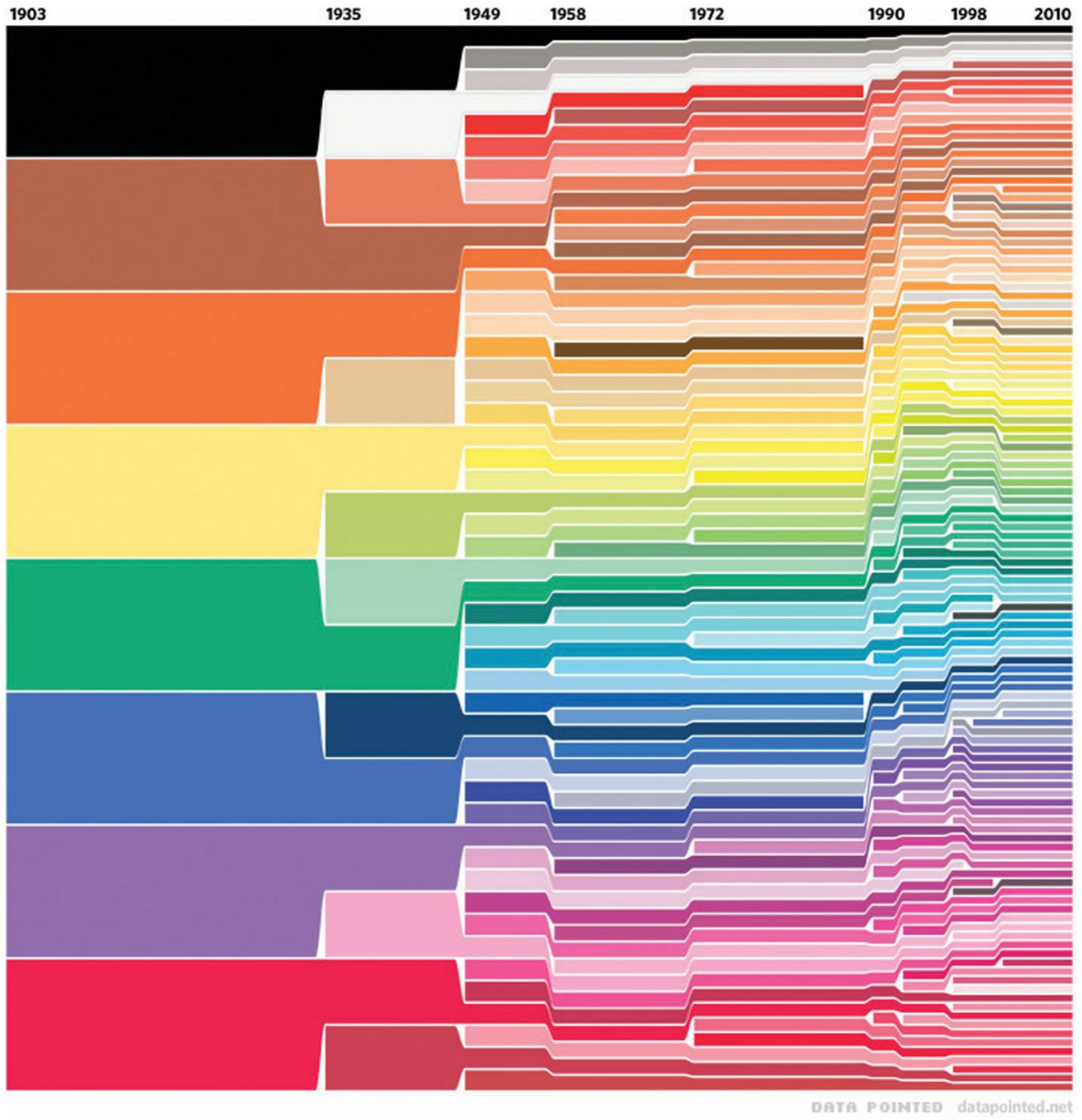
```
ax1.plot(df1.Time,df1.value,marker=markers[0],markerfacecolor=colors[0],
        markersize=8,markeredgewidth=0.5,color='k',linewidth=0.5,
        linestyle='-',label=labels[0])
ax1.plot(df3.Time,df3.value,marker=markers[0],markerfacecolor=colors[2],
        markersize=8,markeredgewidth=0.5,color='k',linewidth=0.5,
        linestyle='-',label=labels[2])
ax1.spines['right'].set_color('none')
ax1.spines['top'].set_color('none')
ax1.set_xlabel('Time(d)',fontsize=14)
ax1.set_ylabel('value',fontsize=14)
ax1.set_xlim(-1,20)
ax1.set_ylim(-2,90)
ax1.set_xticks(np.linspace(0,20,11,endpoint=True))
ax1.set_yticks(np.linspace(0,90,10,endpoint=True))
ax1.legend(loc='upper left',edgecolor='none',facecolor='none')
fig.savefig("axes3.pdf")
```







Color


1903
1935
1949
1958
1972
1990
1998
2010
DATA POINTED datapointed.net

Color Scale Options

Sequential

The same or similar hues are used, and saturation varies for a single metric.



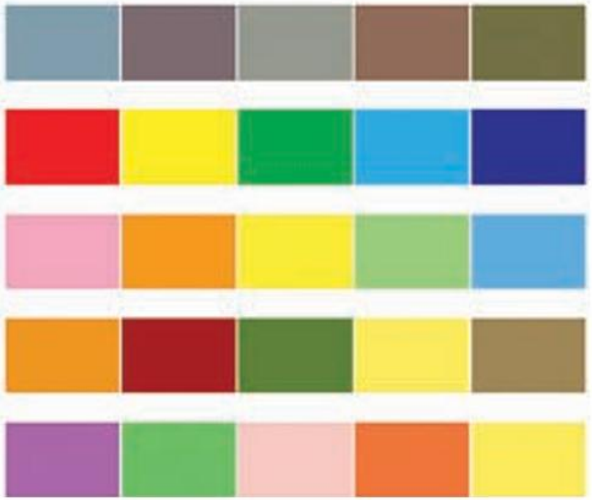
Diverging

Two hues are used to indicate a division, such as positive and negative values.



Qualitative

When data is non-numeric, contrasting colors are used for each category.



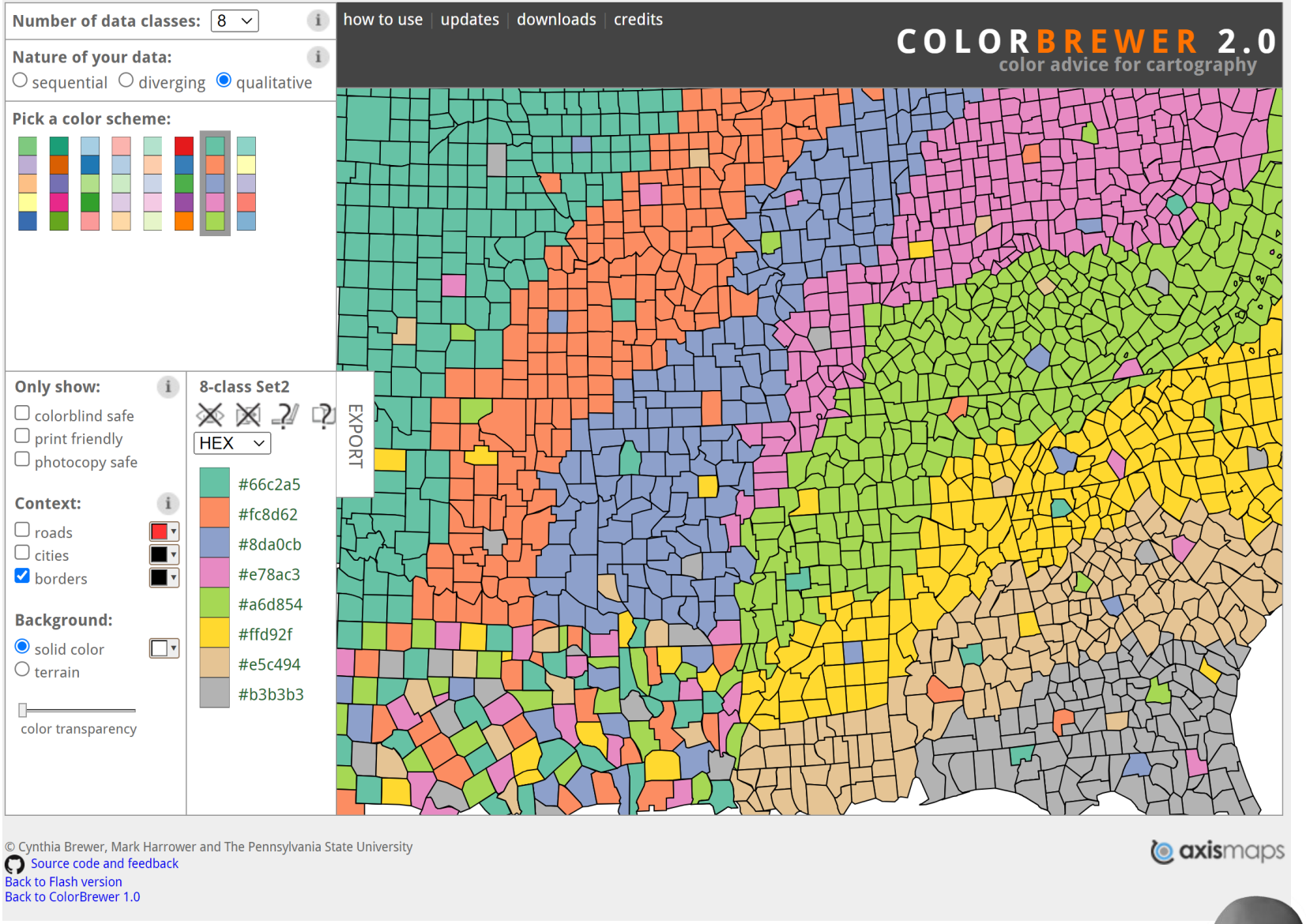


Colorbrewer

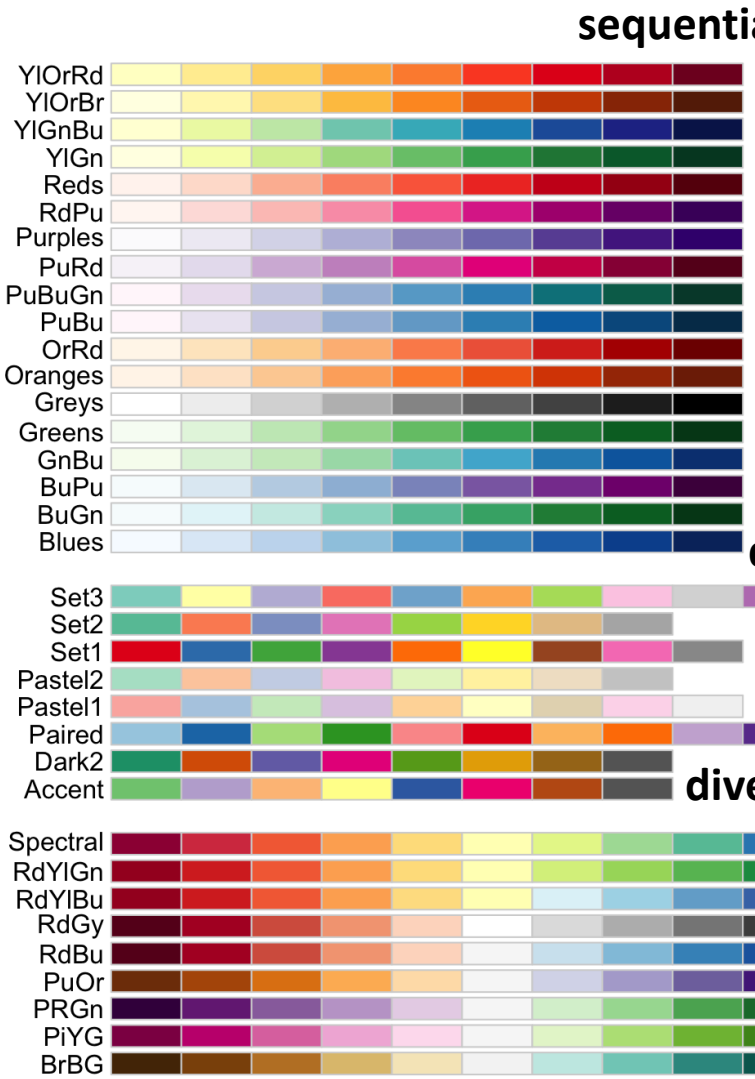
ColorBrewer is an online tool for selecting map color schemes based on palettes created by Cynthia Brewer It was launched in 2002 by Brewer, Mark Harrower, and The Pennsylvania State University.

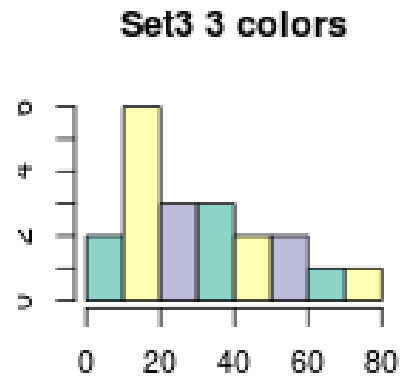
Suggested color schemes are based on data type (sequential, diverging, or qualitative). It also provides options for varied display environments, such as laptop, photocopy, and LCD projector, and colorblind safe options.

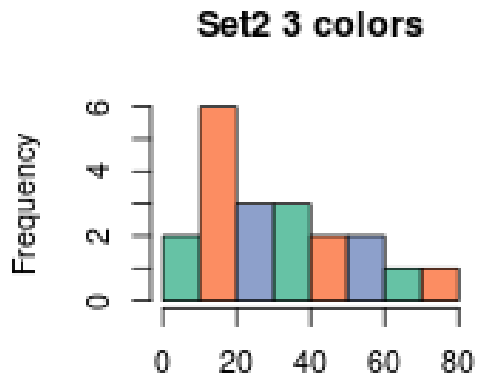
https://colorbrewer2.org

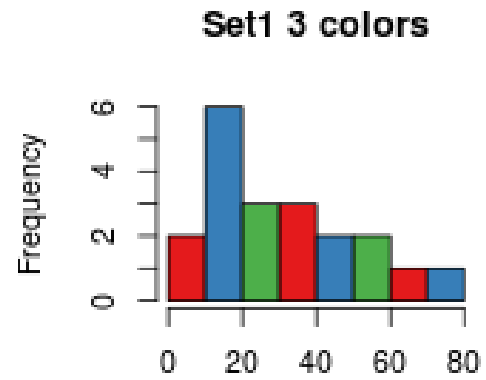


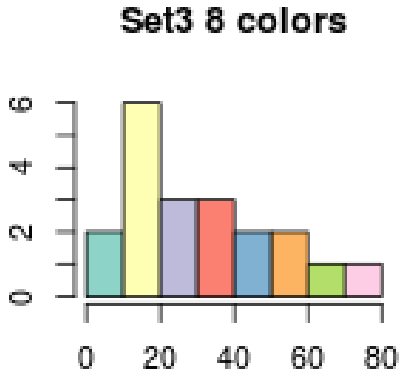
Color Palettes

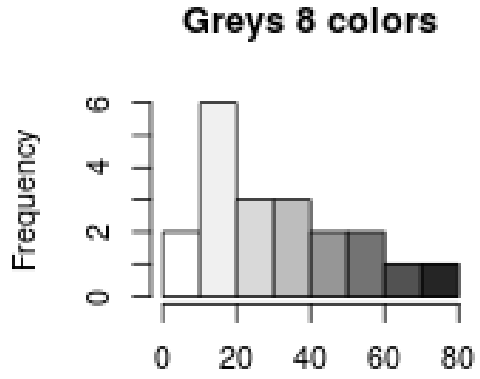


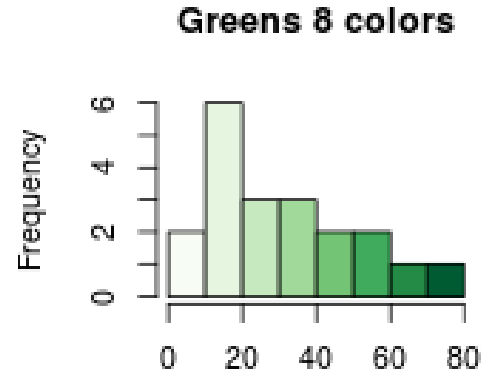














Color Palettes

sequential

YlOrRd
YlOrBr
YlGnBu
YlGn
Reds
RdPu
Purples
PuRd
PuBuGn
PuBu
OrRd
Oranges
Greys
Greens
GnBu
BuPu
BuGn
Blues

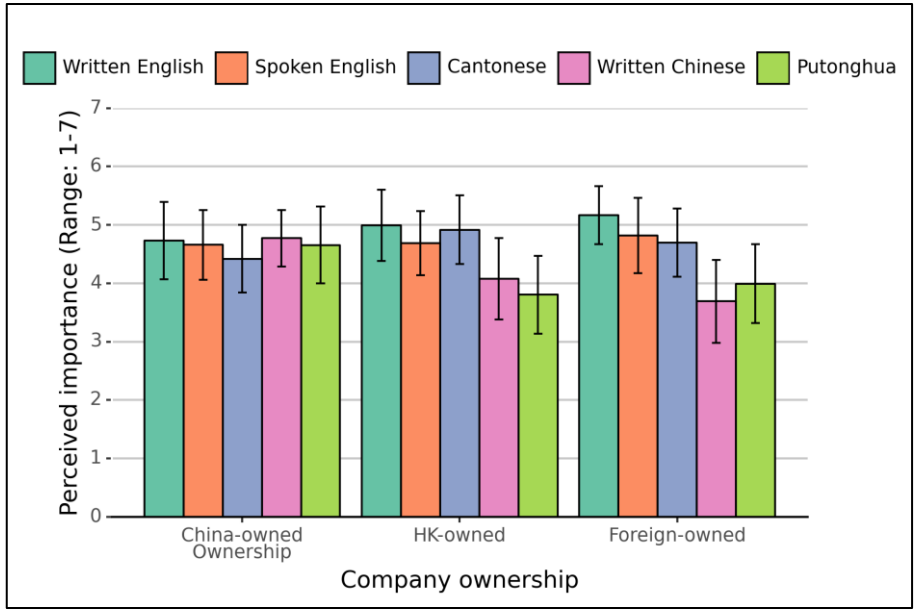
qualitative

Set3
Set2
Set1
Pastel2
Pastel1
Paired
Dark2
Accent

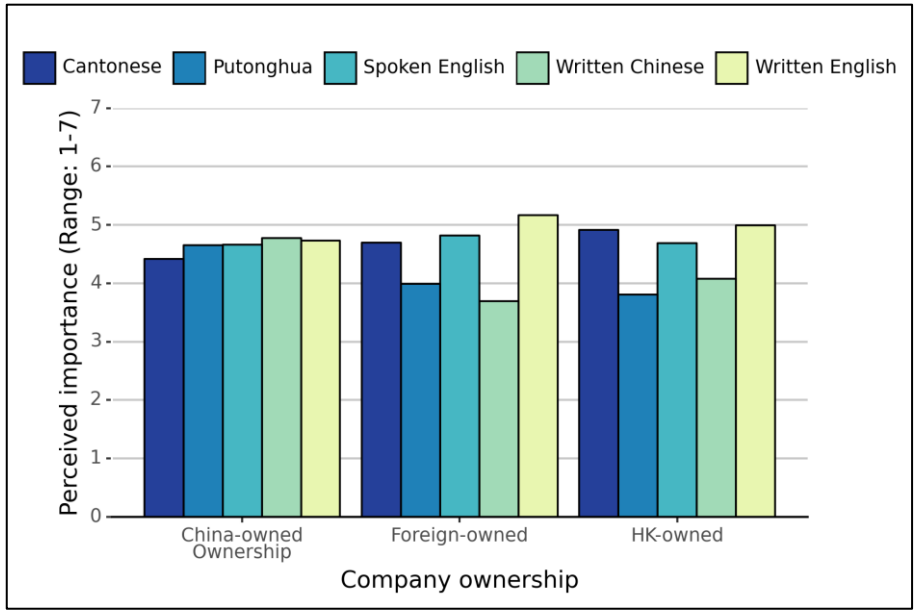
diverging

Spectral
RdYlGn
RdYlBu
RdGy
RdBu
PuOr
PRGn
PiYG
BrBG

Set2



YlGnBu



Color Palettes

```
#Extract HEX color codes from colormap in Matplotlib
import matplotlib.pyplot as plt
from matplotlib.colors import to_hex
# Extract HEX color codes
hex_colors = [to_hex(color) for color in plt.get_cmap('Set1').colors]
# Display the result
print(hex_colors)
```

Hex Color Code: A 6-digit code starting with # used to represent colors in digital design and coding. It follows the format #RRGGBB (Red, Green, Blue). For example, #FF0000 is pure red, and #000000 is black.



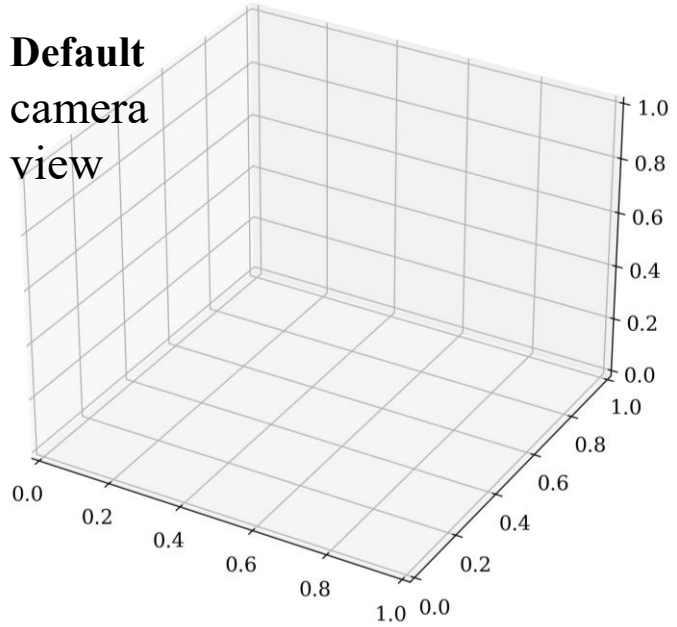
Matplotlib: 3D Plot

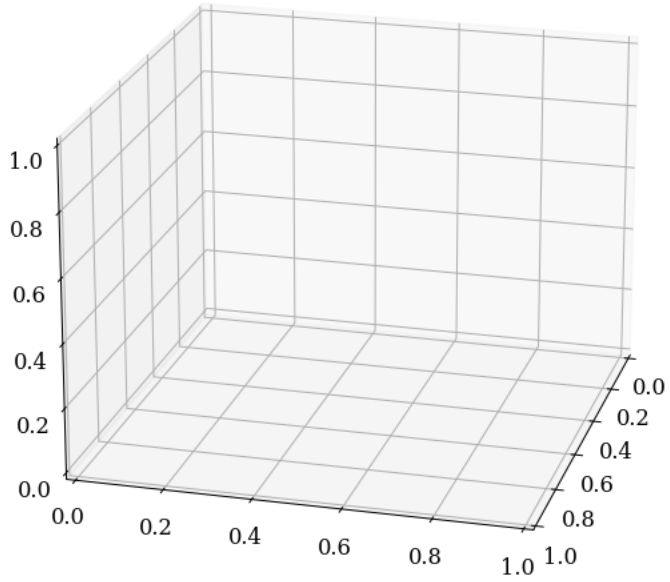
```
import matplotlib.pyplot as plt
import numpy as np

#============================================================
fig, ax = plt.subplots(figsize=(10, 8), dpi=90, subplot_kw={'projection': '3d'})

# Apply data objects on the axes for 3D plotting
ax.view_init(azim=15, elev=20)
# Rotate the camera view 15°  along the Z-axis and 20°  along the Y-axis
plt.show()
```

Default camera view





Matplotlib: 3D Plot

```
#Dataset creation
X = np.arange(-4, 4, 0.25)
Y = np.arange(-4, 4, 0.25)
X, Y = np.meshgrid(X, Y)

R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)
```

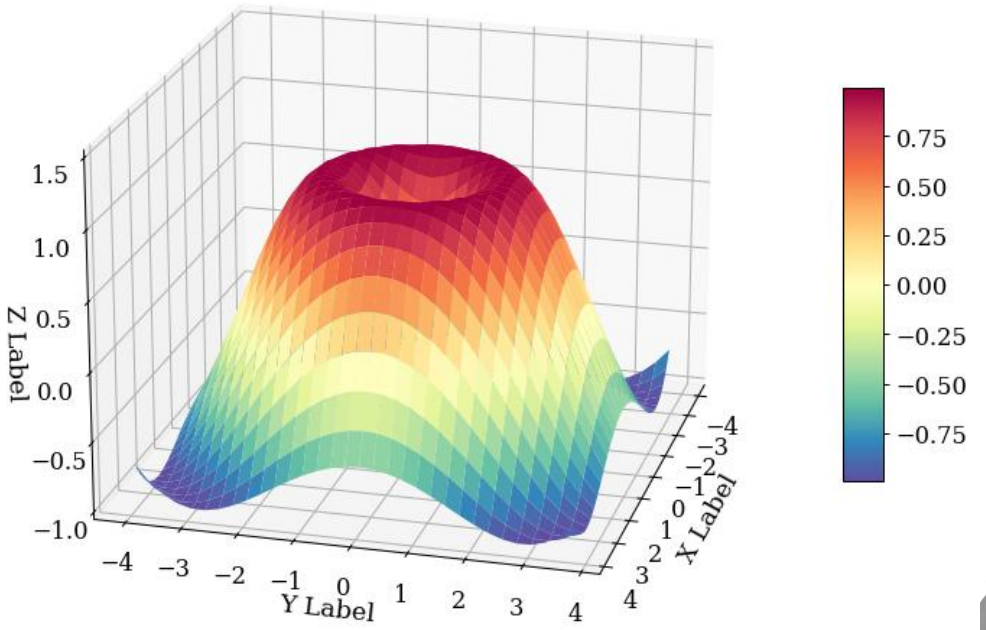
```
fig, ax = plt.subplots(figsize=(10, 8), dpi=90, subplot_kw={'projection': '3d'})

# Apply data objects on the axes for 3D plotting
ax.view_init(azim=15, elev=20)  # Rotate the camera view 15° along the Z-axis and 20° along the Y-axis

p = ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap='Spectral_r', alpha=1)
# rstride: stride between rows is 1; cstride=1: stride between columns is 1;
# cmap='Spectral_r': color mapping as spectral rainbow colors

ax.set_xlabel('X Label')  # Set X-axis label
ax.set_ylabel('Y Label')  # Set Y-axis label
ax.set_zlabel('Z Label')  # Set Z-axis label
ax.set_zlim(-1, 1.5)  # Set the range of values on the Z-axis

# Colorbar with shrinkage by half and aspect ratio of 10:1
fig.colorbar(p, shrink=0.5, aspect=10)
# fig.savefig('3D_Surface_Plot.pdf')
plt.show()
```



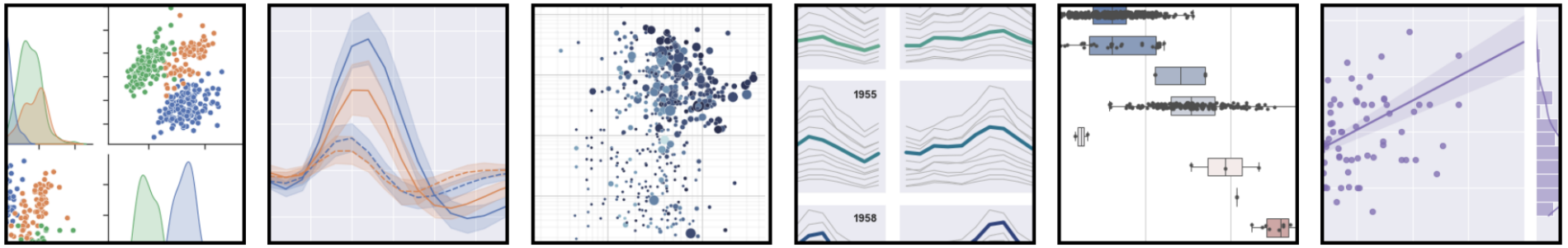
Seaborn

https://seaborn.pydata.org/

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

seaborn Installing Gallery Tutorial API Releases Citing FAQ

seaborn: statistical data visualization



Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

For a brief introduction to the ideas behind the library, you can read the introductory notes or the paper. Visit the installation page to see how you can download the package and get started with it. You can browse the example gallery to see some of the things that you can do with seaborn, and then check out the tutorials or API reference to find out how.

To see the code or report a bug, please visit the GitHub repository. General support questions are most at home on stackoverflow, which has a dedicated channel for seaborn.

Contents

Installing
Gallery
Tutorial
API
Releases
Citing
FAQ

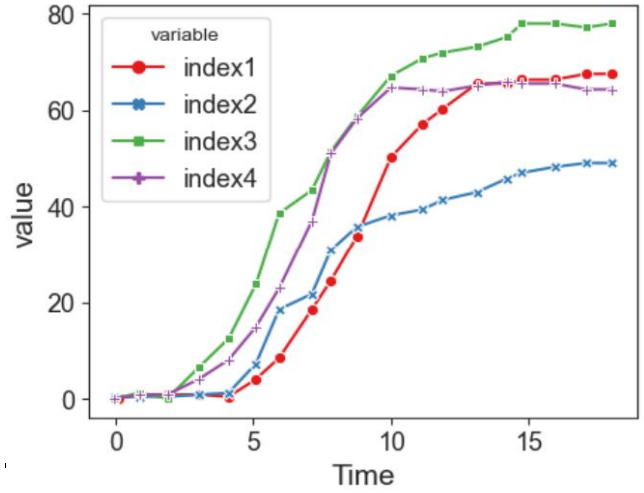
Features

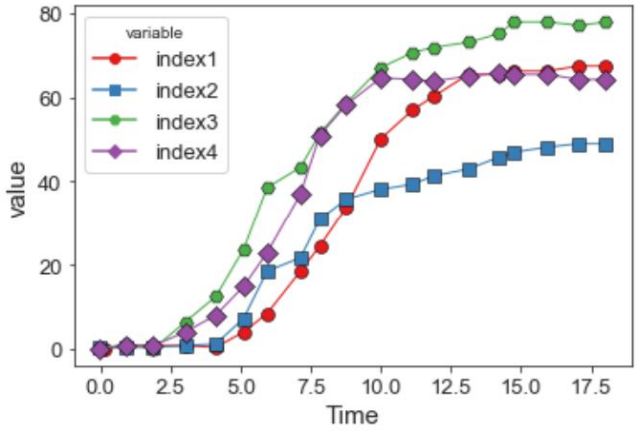
- New Objects: API | Tutorial
- Relational plots: API | Tutorial
- Distribution plots: API | Tutorial
- Categorical plots: API | Tutorial
- Regression plots: API | Tutorial
- Multi-plot grids: API | Tutorial
- Figure theming: API | Tutorial
- Color palettes: API | Tutorial



Seaborn: Scatter plot with a line connecting markers.

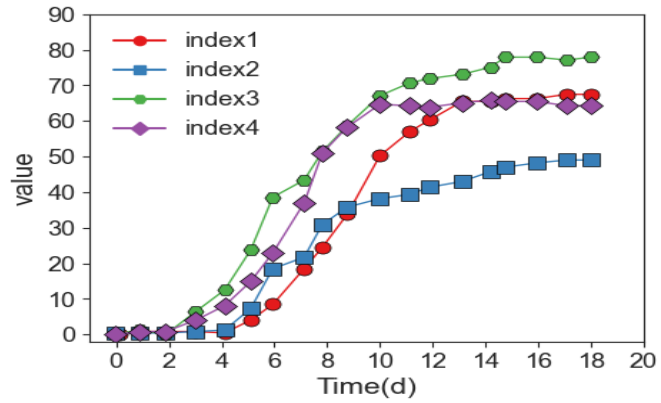
```
df=pd.read_csv("MappingAnalysis_Data.csv")
sns_line=sns.lineplot(x='Time',y='value',hue='variable',data=df,
                      markers=True,dashes=False)
plt.show()
#-----------------------------------------------------------------------
markers=['o','s','H','D']
sns_line=sns.lineplot(x='Time',y='value',hue='variable',style=
                      'variable',data=df,markers=markers,dashes=
                      False,markersize=8,markeredgewidth=0.5,
                      markeredgecolor='k',linewidth=1)
plt.show()
```

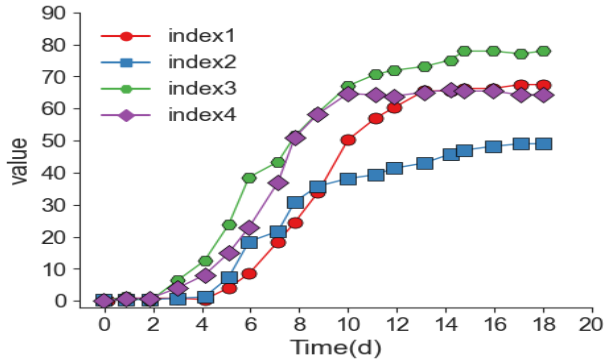




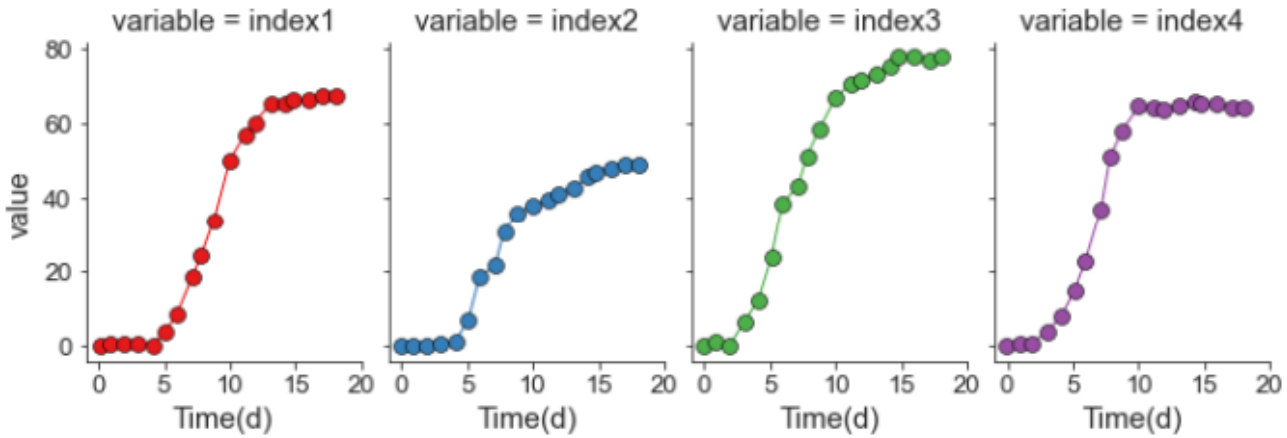
Seaborn: Scatter plot with a line connecting markers.

```
plt.xlabel("Time(d)")
plt.ylabel('value')
plt.xlim(-1,20)
plt.ylim(-2,90)
plt.xticks(np.linspace(0,20,11,endpoint=True))
plt.yticks(np.linspace(0,90,10,endpoint=True))
plt.legend(loc='upper left',edgecolor='none',facecolor='none')
#----------------------------------------------------------------------
ax=plt.gca() ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
handles,labels=ax.get_legend_handles_labels()
ax.legend(handles=handles[0:],labels=labels[0:],
          loc='upper left',edgecolor='none',facecolor='none')
plt.show()
```





Seaborn: Faceted Chart



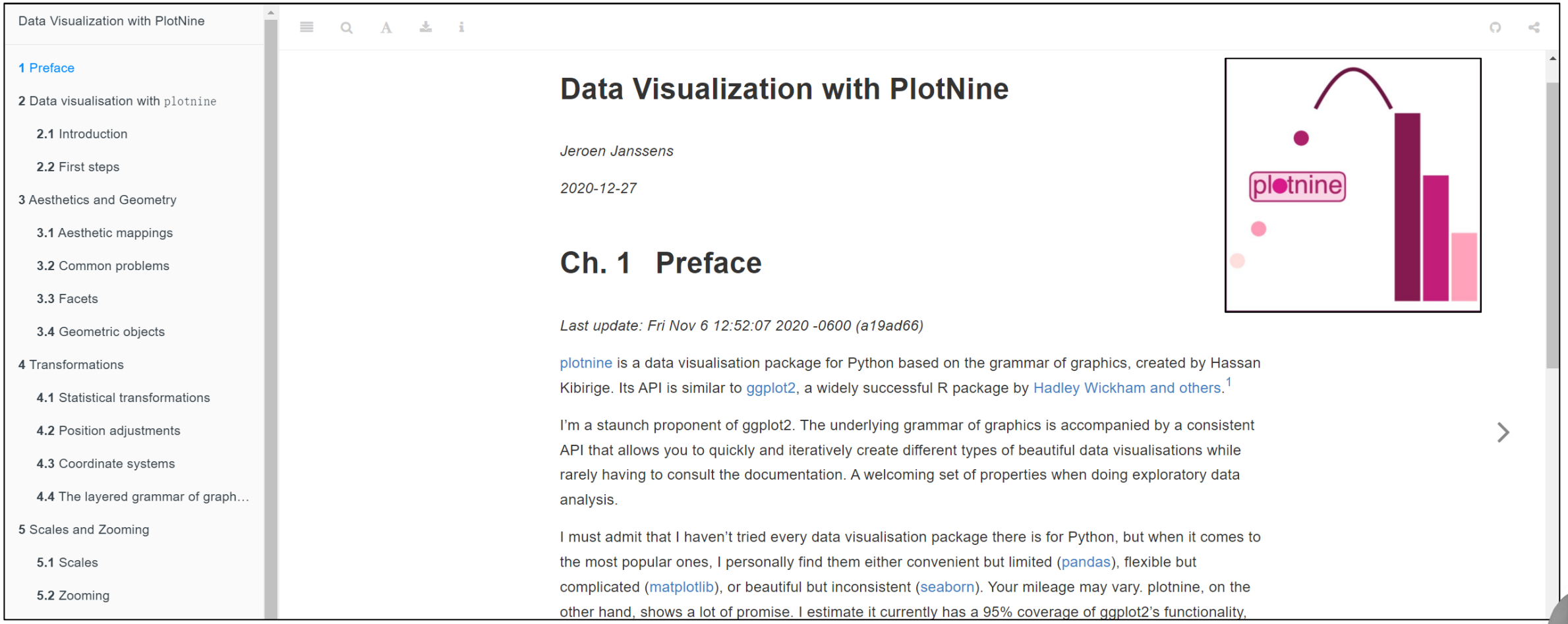

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

df=pd.read_csv("MappingAnalysis_Data.csv")
#-------------------------------------------------------------------------------------------
g=sns.FacetGrid(df,col='variable',hue='variable',size=3,aspect=0.9,
                    gridspec_kws={"wspace":0.1})  # Grid layout
g.map(sns.lineplot,"Time","value",marker='o',dashes=False,linewidth=1,
          markersize=8,markeredgewidth=0.5,markeredgecolor='k')
g.set_xlabels("Time(d)")
g.set_ylabels('value')
plt.xticks(np.linspace(0,20,5,endpoint=True))
plt.yticks(np.linspace(0,80,5,endpoint=True))
#-------------------------------------------------------------------------------------------
g.set_titles(row_template='{row_name}',col_template='{col_name}')
```

Plotnine

https://f0nzie.github.io/rmarkdown-python-plotnine/
https://github.com/has2k1/plotnine

Plotnine is a data visualisation package for Python based on the grammar of graphics, created by Hassan Kibirige. Its API is similar to ggplot2, a widely successful R package by Hadley Wickham and others.



Why Plotnine

Describes all the non-data ink

Plotting space for the data

Statistical models & summaries

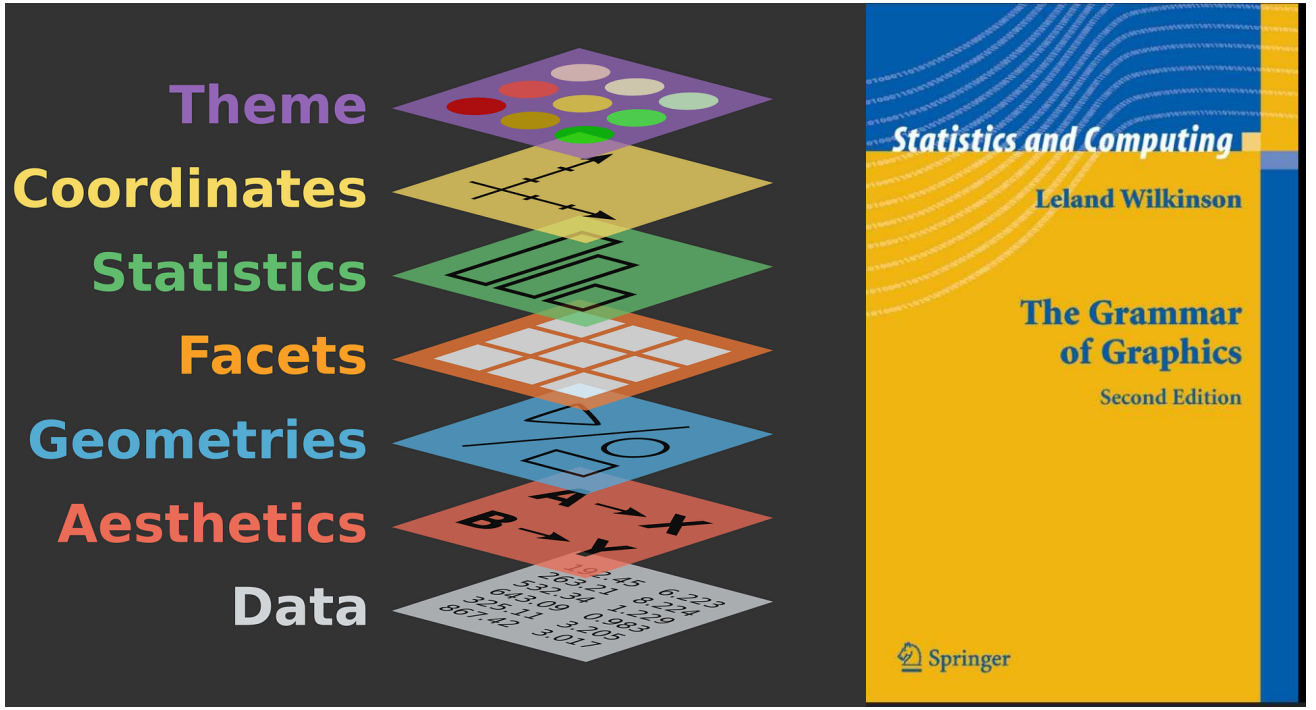
Rows and columns of sub-plots

Shapes used to represent the data

Scales onto which data is mapped

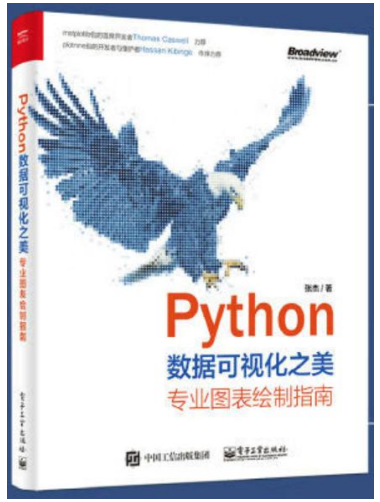
The actual variables to be plotted

Main Components of the Grammar of Graphics



Python: Plotnine https://github.com/has2k1/plotnine
R: ggplot2 https://ggplot2-book.org/mastery https://ggplot2.tidyverse.org/
Javascript: G2 https://github.com/antvis/G2
Software: Tableau https://www.tableau.com/

Plotnine: Beautiful Data Visualization with Python



When trying to understand or communicate information one usually asks, what is the best visualisation(s) I can make? If this question often comes to you and that you tend not to have an answer, then "Beautiful Visualisation with Python" is meant for you. Python is an excellent language for data analysis and visualization. Secondly, the book helps you build a practical toolbox for most visualisations that you may want to create. It strikes a delicate balance between a book that introduces and teaches and a gallery that you can always comeback to for ideas. It is not one to throw away after reading. As it is titled, you will learn how to turn data in into beautiful visualisations by making the best choices at every step. Most important for me is that it covers the biggest idea in data visualisation in the last 20 years, that is, "The Grammar of Graphics".

Hassan Kibirige

Author/Maintainer of plotnine（plotnine包的开发者与维护者）





Plotnine

Plotnine's entire syntax framework includes both the data visualization and aesthetic details parts, and it features the following characteristics:

- It adopts a layered design approach, facilitating structured thinking in data visualization. It has a clear beginning (initiated with ggplot()) and end. Layers are overlaid using the "+" operator, with the layers further back appearing beneath those in front. Typically, a geom_xxx() function or stat_xxx() function can create a layer.
- By separating data representation from graphical details, it allows for quick visualization rendering, making creative plotting easier to achieve. Common statistical transformations can be integrated into the plot using stat_xxx() functions.

For installing the Plotnine package, you can use the command "pip install plotnine". In Python, the default import statement is:

```
from plotnine import *
```

To import datasets that come with the Plotnine package, you can use the following statement:

```
from plotnine.data import *
```



- pip install plotnine # 1. should be sufficient for most

- # Or using conda
- conda install -c conda-forge plotnine



Plotnine: grammar

Complete the template below to build a graph.

```
ggplot (data = <DATA>) +
<GEOM_FUNCTION>(mapping = aes(<MAPPINGS>),
 stat =<STAT>, position =<POSITION>) +
<COORDINATE_FUNCTION>+
<FACET_FUNCTION>+
<SCALE_FUNCTION>+
<THEME_FUNCTION>
```

required

Not required, sensible defaults supplied

ggplot(data = mpg, **aes(**x = cty, y = hwy**))** Begins a plot that you finish by adding layers to. Add one geom function per layer.

Plotnine is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data** set, a **coordinate system**, and **geoms**—visual marks that represent data points.

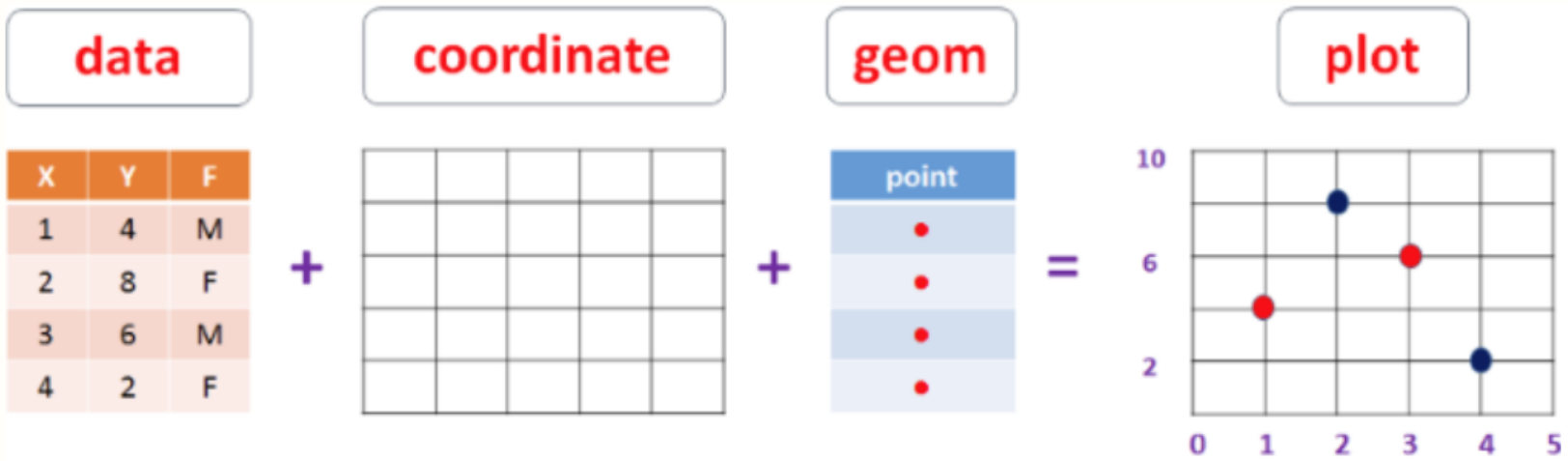


To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.



Plotnine

Source: https://shiny.abdn.ac.uk/Stats/R_Python_tutorial/python-plotnine.html



Arguments	Explanation
data =	The DATA that you want to plot
aes()	AESTHETICS of the geometric and statistical objects, such as color, size, shape and position.
geom_	The GEOMETRIC shapes that will represent the data.

Essential part

Aes

Common aesthetic values.

color and **fill** - string ("red", "#RRGGBB")

linetype - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dotdash", 5 = "longdash", 6 = "twodash")

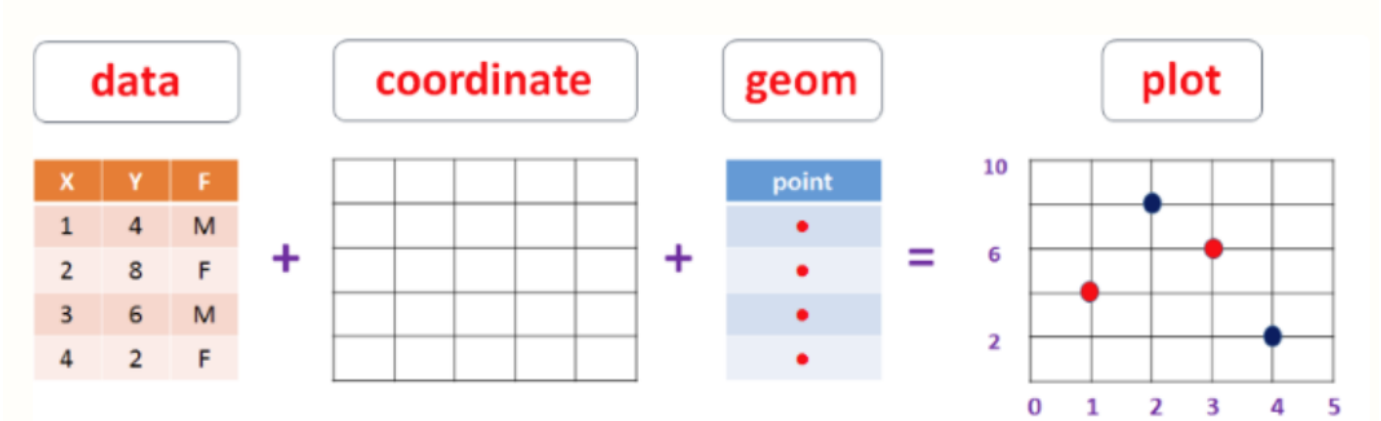
size - integer (in mm for size of points and text)

linewidth - integer (in mm for widths of lines)

shape - integer/shape name or a single character ("a")



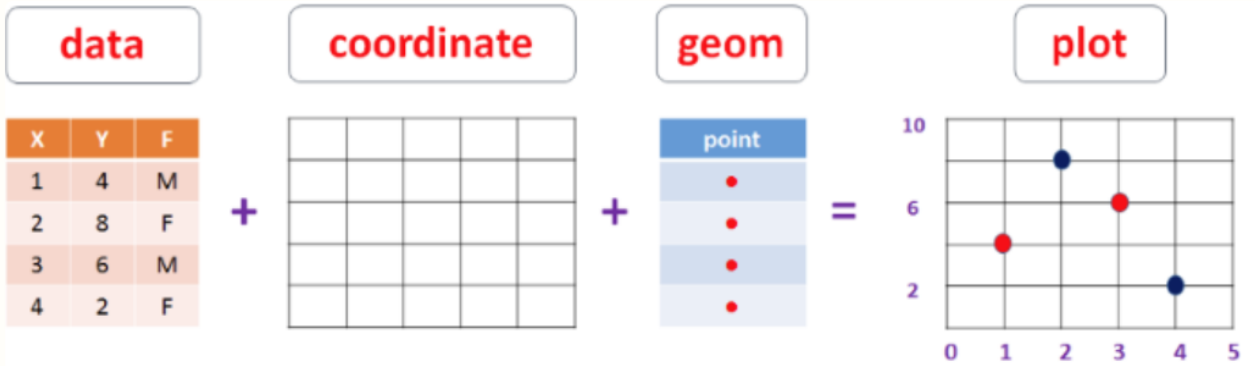
Plotnine



Advanced part

Arguments	Explanation
stat_	STATISTICAL summaries of the data that can be plotted, such as quantiles, fitted curves (loess, linear models, etc.), sums and so o.
coord_	The transformation used for mapping data COORDINATES into the plane of the data rectangle.
facet_	The arrangement of the data into a grid of plots
theme_	The overall visual THEMES of a plot: background, grids, axe, default typeface, sizes, colors, etc.
scale_	MAP between the data and the aesthetic dimensions, such as data range to plot width or factor values to colors.

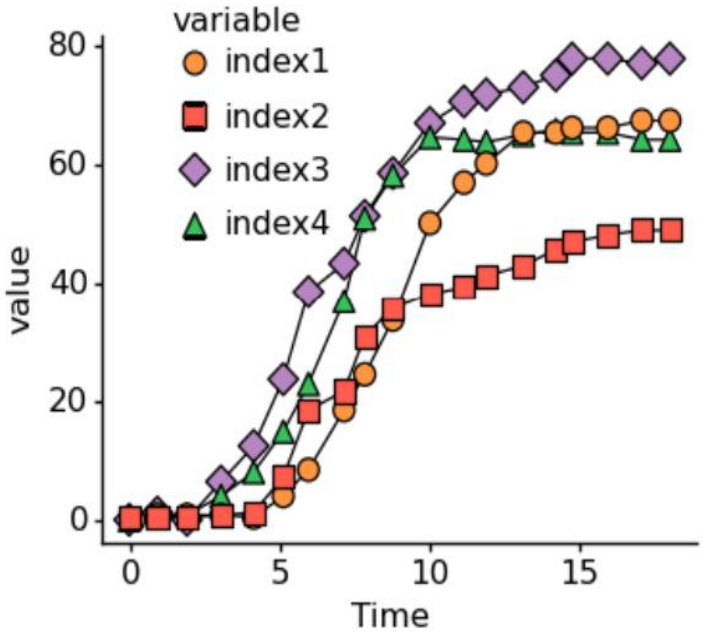
Function	Plot	Graphical_parameters
`geom_histogram`	Histogram	`colour`, `fill`, `alpha`
`geom_freqpoly`	Frequency polygon	`colour`, `fill`, `alpha`
`geom_density`	Density plot	`colour`, `fill`, `alpha`, `linetype`
`geom_rug`	Rug plot	`colour`, `side`
`geom_qq`	Quantile-Quantile plot	`colour`, `alpha`, `linetype`, `size`
`geom_boxplot`	Box plot	`colour`, `fill`, `alpha`, `notch`, `width`
`geom_violin`	Violin plot	`colour`, `fill`, `alpha`, `linetype`, `size`
`geom_point`	Scatter plot	`colour`, `alpha`, `shape`, `size`
`geom_jitter`	Jittered points	`colour`, `alpha`, `shape`, `size`
`geom_text`	Text	`colour`, `alpha`, `size`, `label`, `family`, `fontface`
`geom_bar`	Bar chart	`colour`, `fill`, `alpha`
`geom_line`	Line graph	`colour`, `alpha`, `linetype`, `size`
`geom_hline`	Horizontal line	`colour`, `alpha`, `linetype`, `size`
`geom_vline`	Vertical line	`colour`, `alpha`, `linetype`, `size`
`geom_smooth`	Fitted line	`method`, `formula`, `colour`, `fill`, `linetype`, `size`



Geometric functions

Plotnine: Scatter plot with a line connecting markers

```
import pandas as pd
import numpy as np
from plotnine import *
df=pd.read_csv("MappingAnalysis_Data.csv")
p6=(ggplot(df,aes(x='Time',y='value',shape='variable',fill='variable'))+
      geom_line()+
      geom_point(size=4,color='k')+
      scale_fill_manual(values=('#FF9641','#FF5B4E','#B887C3','#38C25D'))+
      scale_shape_manual(values=('o','s','D','^'))+
      theme_classic()+
      theme(text=element_text(size=12,color='k'),aspect_ratio=0.8,
            dpi=100,  figure_size=(4,4),
            legend_position=(0.32,0.75),
            legend_background=element_rect(fill='none')))
#print(p6)
p6.show()
#p6.save(plotnine(f).pdf)
```



Long-format vs Wide-format data

Name	Subject	Grade
Peter	English	99
Peter	Math	84
Peter	Chinese	95
Jack	English	83
Jack	Math	93
Jack	Chinese	92
Jon	English	82
Jon	Math	90
Jon	Chinese	84

Name	English	Math	Chinese
Peter	99	84	95
Jack	83	93	92
Jon	82	90	84



Plotnine: Aesthetic parameter mapping

Plotnine can be used to map aesthetics to variables, with the main parameters including color/col/colour, fill, angle, linetype, shape, vjust, and hjust. Among these, linetype and shape parameters are only suitable for categorical variables.

1. **Color/Col/Colour, Fill, and Alpha**: These properties are related to color aesthetics.
    1. **Color/Col/Colour**: Specifies the color of points, lines, and outlines.
    2. **Fill**: Determines the color of filled regions.
    3. **Alpha**: Sets the transparency of colors, ranging from 0 (completely transparent) to 1 (opaque).
2. **Size**: Represents the size of points or the width of lines, typically in millimeters. It can be used to create bubble charts by mapping size on top of a scatter plot created with geom_point().
3. **Angle**: Refers to the angle, applicable to certain geometric objects such as the orientation of text in geom_text() or short bars in geom_spoke().



Plotnine: Aesthetic parameter mapping

4. **vjust and hjust**: Both are aesthetic parameters related to position adjustment.

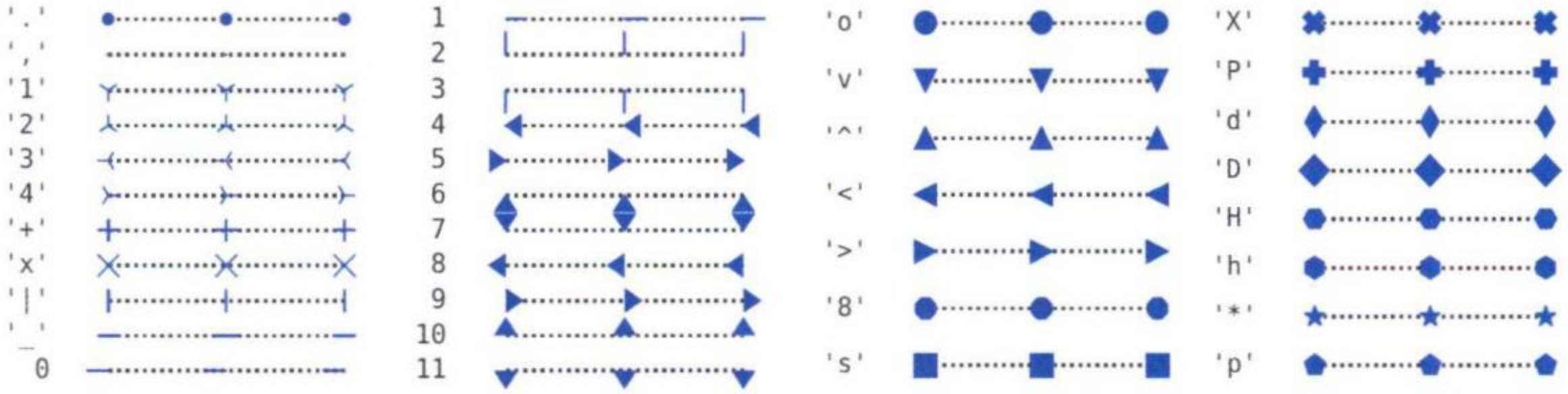
- **vjust**: Specifies vertical alignment adjustments, using numbers or position strings within the (0,1) range where 0="bottom", 0.5="middle", and 1="top". Numbers outside this range control the adjustment proportion unevenly.
- **hjust**: Refers to horizontal alignment adjustments, using numbers or position strings within the (0,1) range where 0="left", 0.5="center", and 1="right". Numbers outside this range control the adjustment proportion unevenly.

5. **linetype**: Represents the type of line, including solid lines (1="solid"), dashed lines (2="dashed"), dotted lines (3="dotted"), dash-dot lines (4="dotdash"), long dashed lines (5="longdash"), and two-dash lines (6="twodash").

6. **shape**: Indicates the shape of points, with 26 integers in the range [0,25], corresponding to various shapes like squares, circles, triangles, diamonds, etc. Only point types 21 to 26 have fill color (fill) attributes; others have only outline color (color) attributes.

Plotnine: Aesthetic parameter mapping

6. **shape**: Indicates the shape of points, with 26 integers in the range [0,25], corresponding to various shapes like squares, circles, triangles, diamonds, etc. Only point types 21 to 26 have fill color (fill) attributes; others have only outline color (color) attributes.



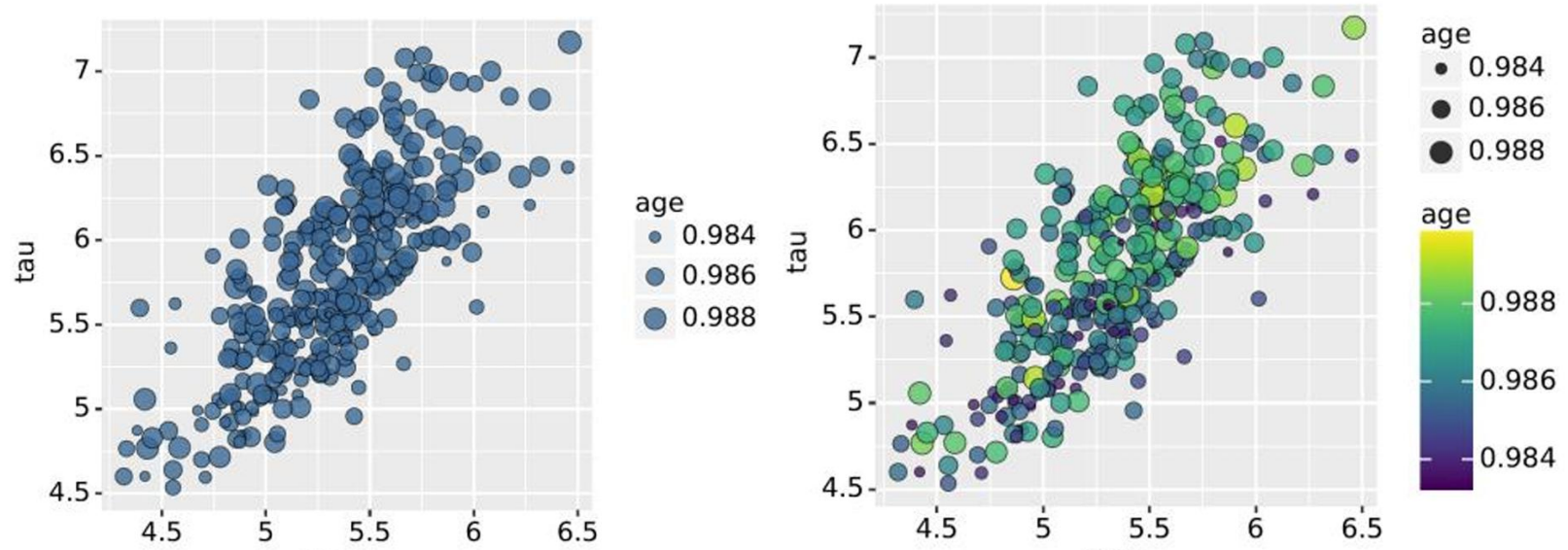
Plotnine: Aesthetic parameter mapping

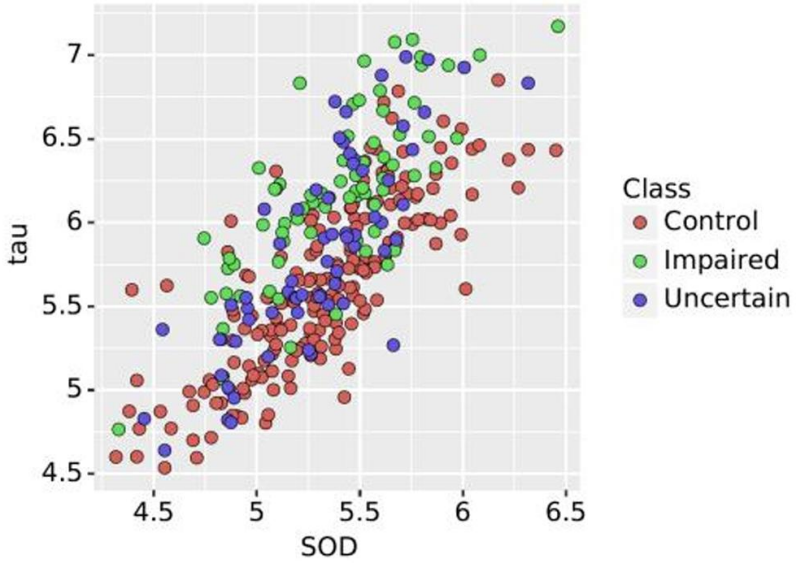
In Plotnine, the basic display elements of the `geom_xxx()` series functions can be categorized into four types: **points, lines, polygons, and text**. Commonly used aesthetic parameter mappings in Plotnine are as shown in the table below:

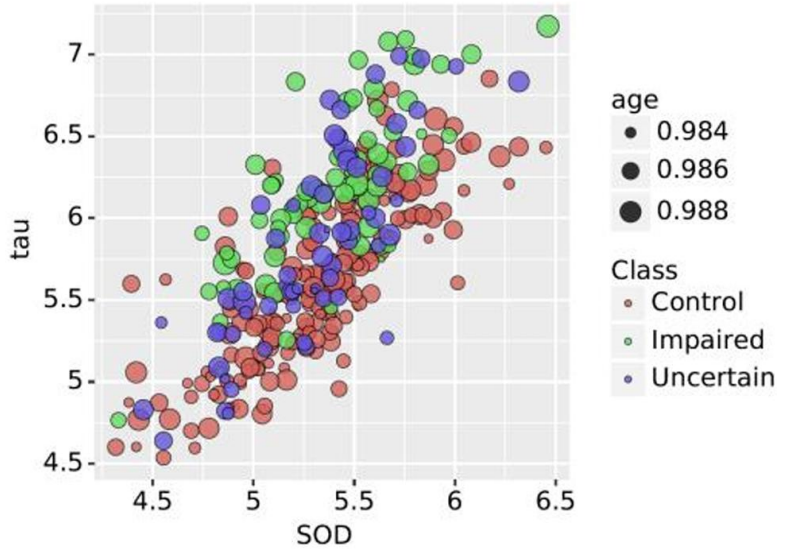
Element	geom_XXX() function	Categorical Parameters	Numeric Parameters
点 (point)	geom_point()、geom_jitter()、geom_dotplot()等	color、fill、shape	color、fill、alpha、size
线 (line)	geom_line()、geom_path()、geom_curve()、geom_density()、geom_linerange()、geom_step()、geom_abline()、geom_hline()等	color、linetype	color、size
多边形 (polygon)	geom_polygon()、geom_rect()、geom_bar()、geom_ribbon()、geom_area()、geom_histogram()、geom_violin()等	color、fill	color、fill、alpha
文本 (text)	geom_label()、geom_text()	color	color、angle、size、alpha



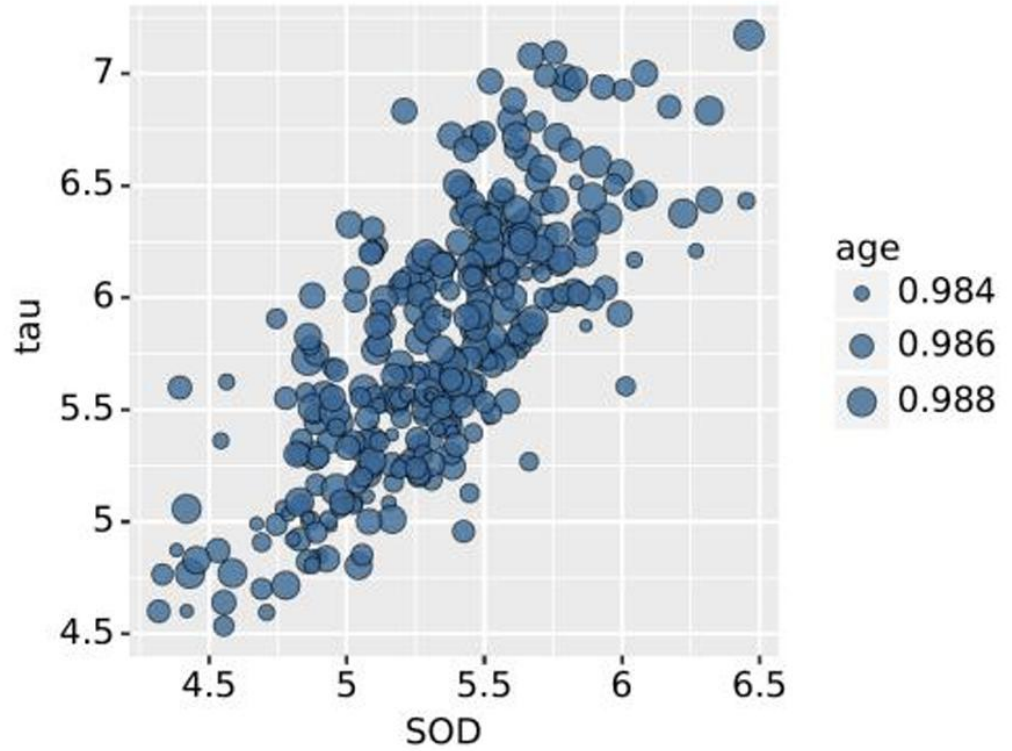
Plotnine: Aesthetic parameter mapping



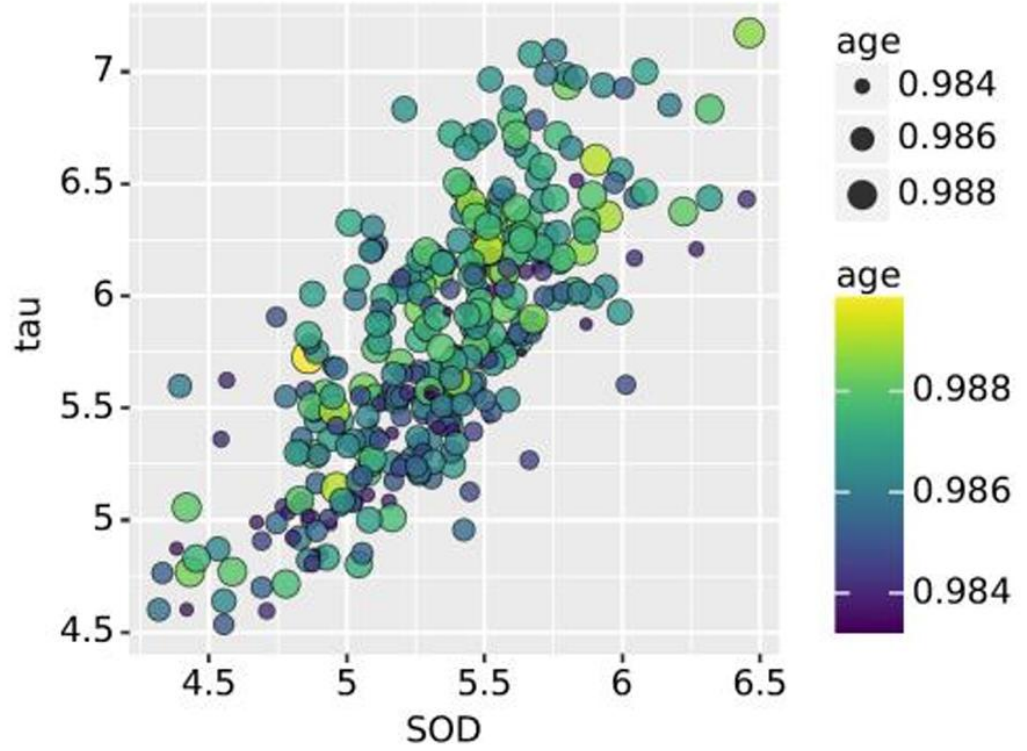




Plotnine: Aesthetic parameter mapping

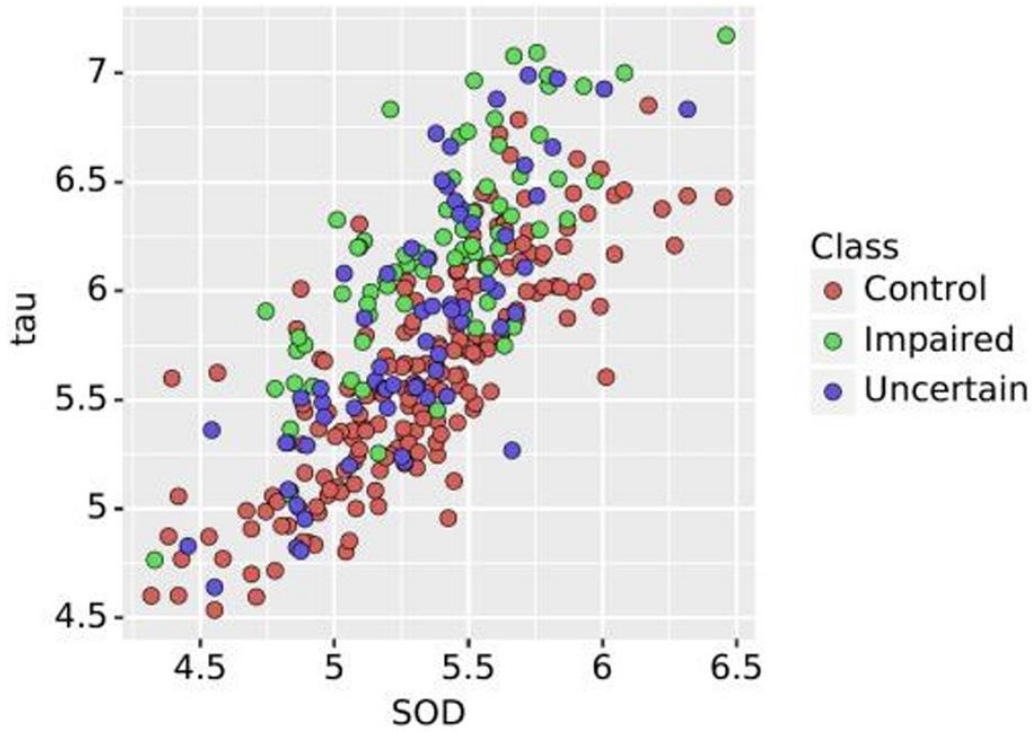



```
(ggplot(df, aes(x='SOD',y='tau',size='age')) +
    geom_point(shape='o',color="black",
               fill="#336A97",stroke=0.25,alpha=0.8))
```

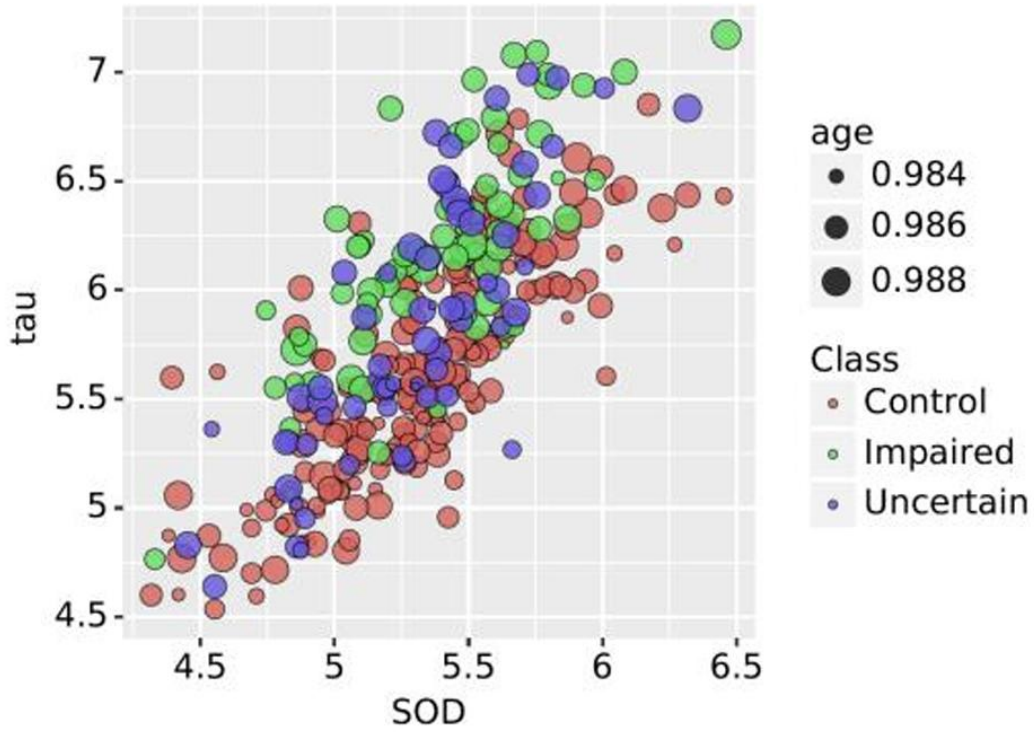



```
(ggplot(df, aes(x='SOD',y='tau',size='age',fill='age')) +
    geom_point(shape='o',color="black",stroke=0.25,
alpha=0.8))
```

Plotnine: Aesthetic parameter mapping

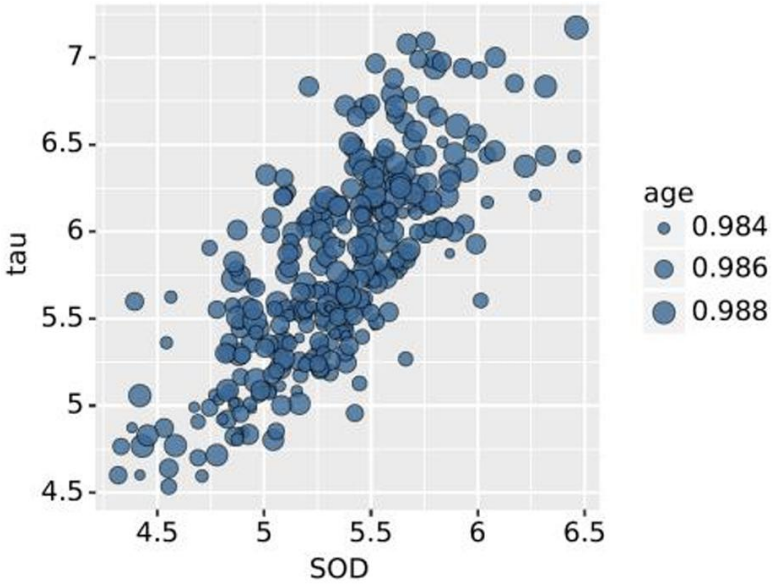



```
(ggplot(df, aes(x='SOD',y='tau',fill='Class')) + geom_
point(shape='o',size=3,colour="black",
stroke=0.25))
```




```
(ggplot(df, aes(x='SOD',y='tau',size='age',fill='Class')) +
   geom_point(shape='o',colour="black",stroke=0.25,
alpha=0.8))
```

Plotnine: Aesthetic parameter mapping




```
import pandas as pd
from plotnine import *
df=pd.read_csv('Facet_Data.csv')
#-------------------------------------------------------------------------------
p1=(ggplot(df,aes(x='SOD',y='tau',size='age'))+
      geom_point(shape='o', color='black',fill='#336A97',stroke=0.25,  alpha=0.8) +
      #geom_point(shape='s',color='black',fill='red',stroke=0.1,  alpha=0.8) +

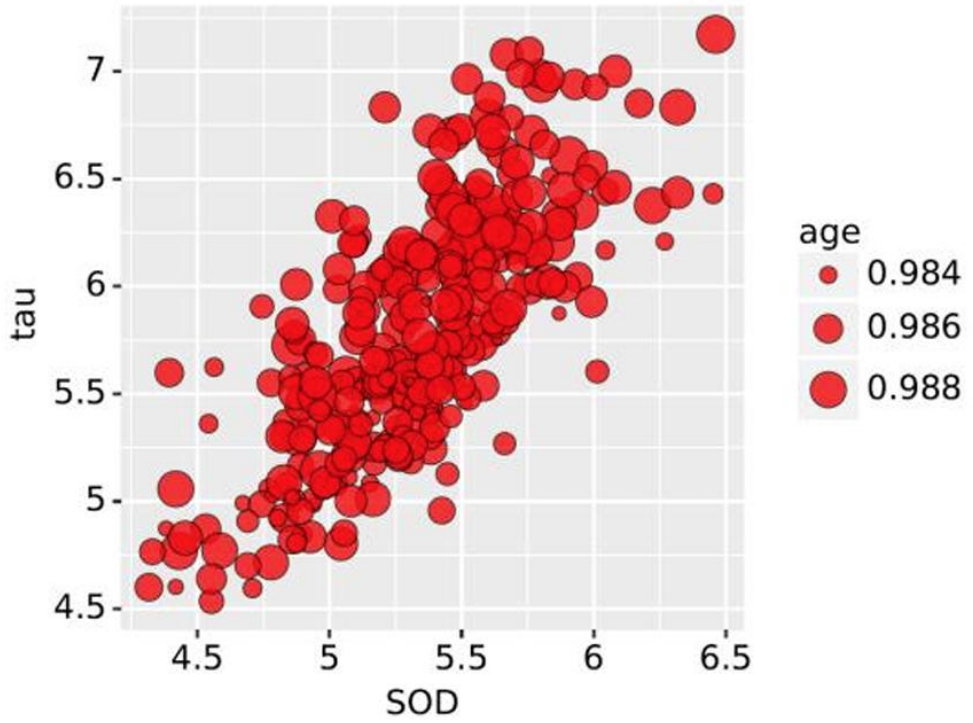

      #scale_fill_distiller(type='seq', palette='Reds') +
      theme(text=element_text(size=12,colour ='black'),
            aspect_ratio =1, dpi=100, figure_size=(4,4)))
p1.show()
#p1.save('plotnine1.pdf')
```

Plotnine: Scale parameter change

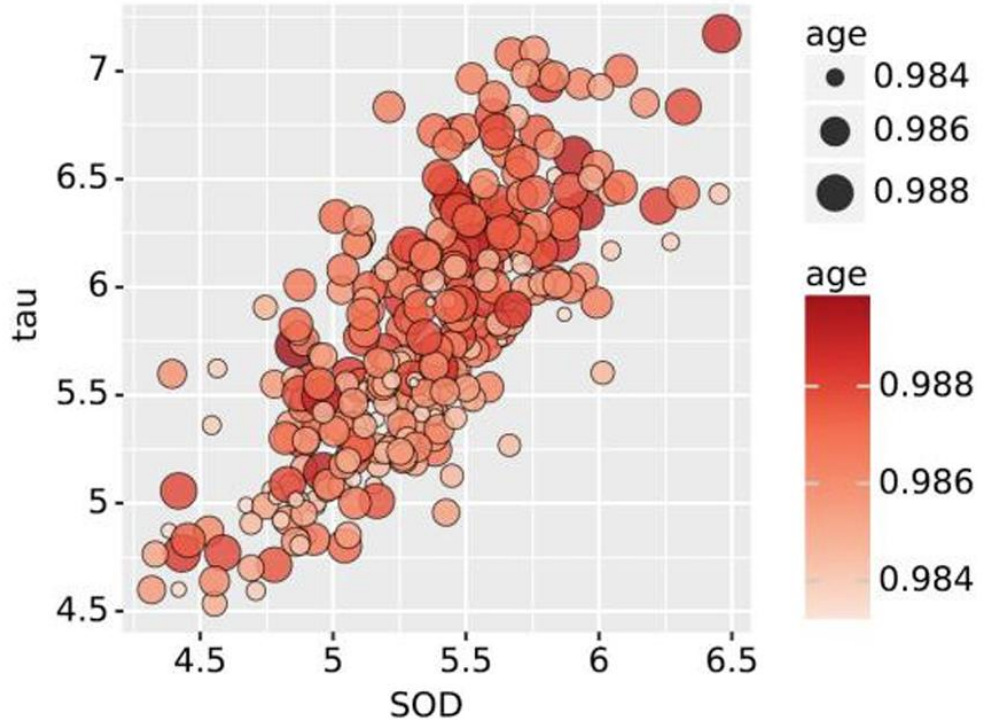
度量（scale）	Categorical Parameters	Numeric Parameters
x：*X* 轴度量 y：*Y* 轴度量	scale_x/y_continuous() scale_x/y_log10() scale_x/y_sqrt() scale_x/y_reverse() scale_x/y_date() scale_x/y_datetime() scale_x/y_time()	scale_x/y_discrete()
colour：轮廓颜色度量 fill：填充颜色度量	scale_fill_cmap() scale_color/fill_continuous() scale_fill_distiller() scale_color/fill _gradient() scale_color/ fill _gradient2() scale_color/fill _gradientn()	scale_color/fill_hue() scale_color/fill_discrete() scale_color/fill_brewer() scale_color/ fill_manual()
alpha：透明度度量	scale_alpha_continuous()	scale_alpha_discrete() scale_alpha_manual()
linetype：线形状		scale_linetype_discrete() scale_linetype_manual()
shape：形状度量		scale_shape() scale_shape_manual()



Plotnine: Scale parameter change



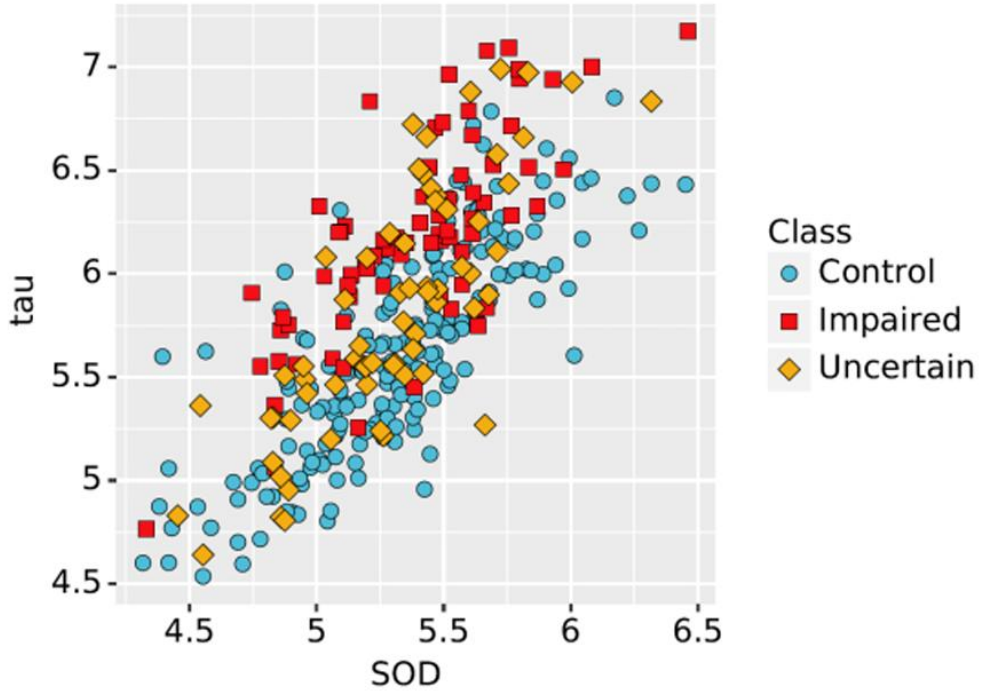

```
(ggplot(df, aes(x='SOD',y='tau',size='age')) +
    geom_point(shape='o',color="black",
                    fill="#FF0000",stroke=0.25,alpha=0.8)+
scale_size(range = (1, 8)))
```



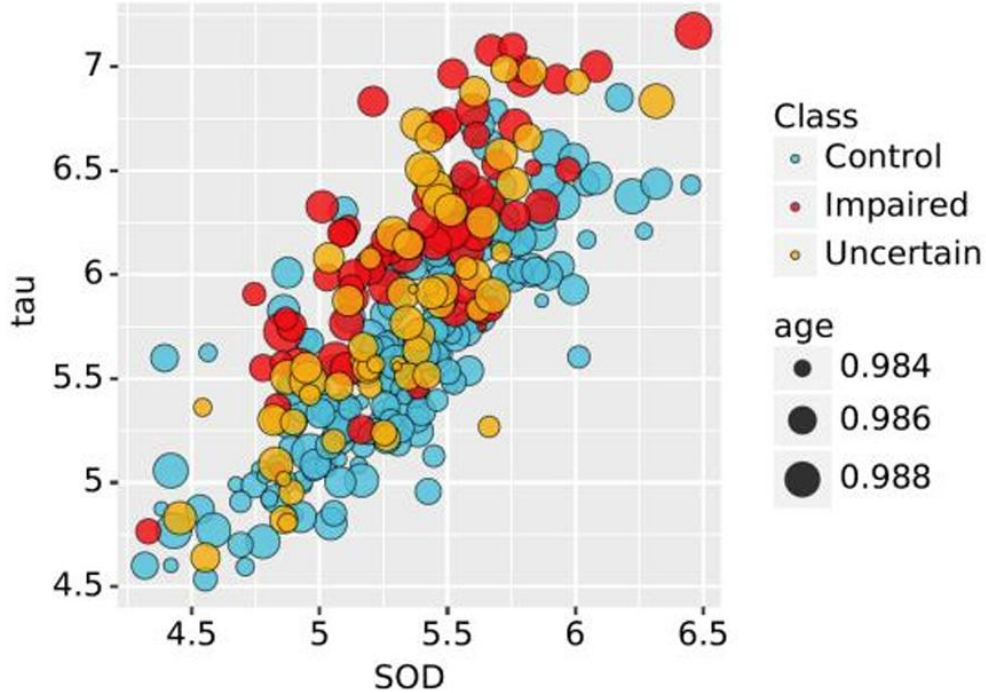

```
(ggplot(df, aes(x='SOD',y='tau',fill='age',size='age')) +
    geom_point(shape='o',color="black",stroke=0.25,
alpha=0.8)+
    scale_size(range = (1, 8))+                    Reds
  scale_fill_distiller(type='seq', palette='reds'))
```




Plotnine: Scale parameter change

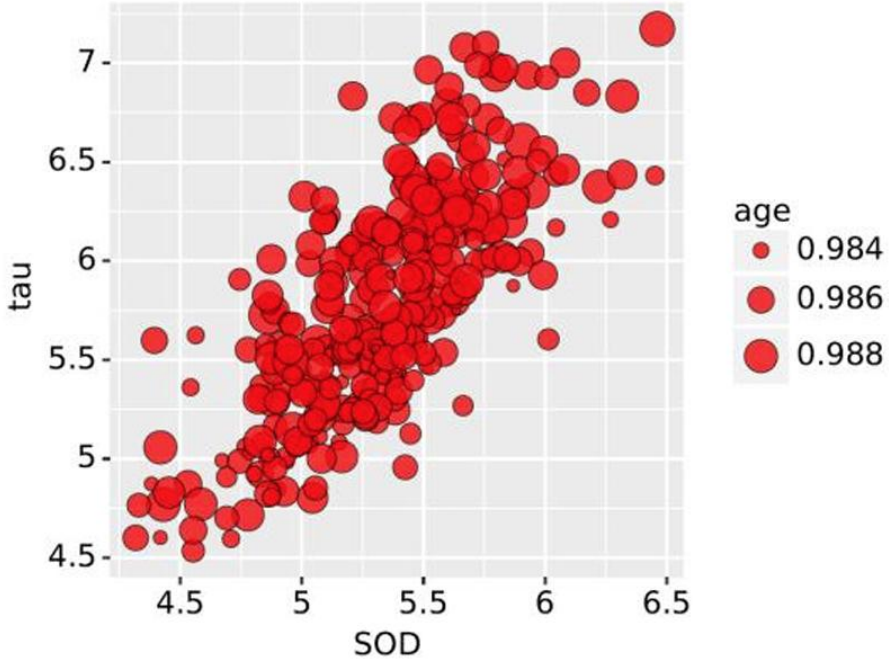



```
(ggplot(df, aes(x='SOD',y='tau',fill='Class',shape='Class')) +
    geom_point(size=3,colour="black",stroke=0.25)+
    scale_fill_manual(values=("#36BED9","#FF0000",
"#FBAD01"))+
scale_shape_manual(values=('o','s','D')))
```




```
(ggplot(df, aes(x='SOD',y='tau',size='age',fill='Class')) +
    geom_point(shape='o',colour="black",stroke=0.25,
alpha=0.8)+
    scale_fill_manual(values=("#36BED9","#FF0000",
"#FBAD01"))+
scale_size(range = (1, 8)))
```

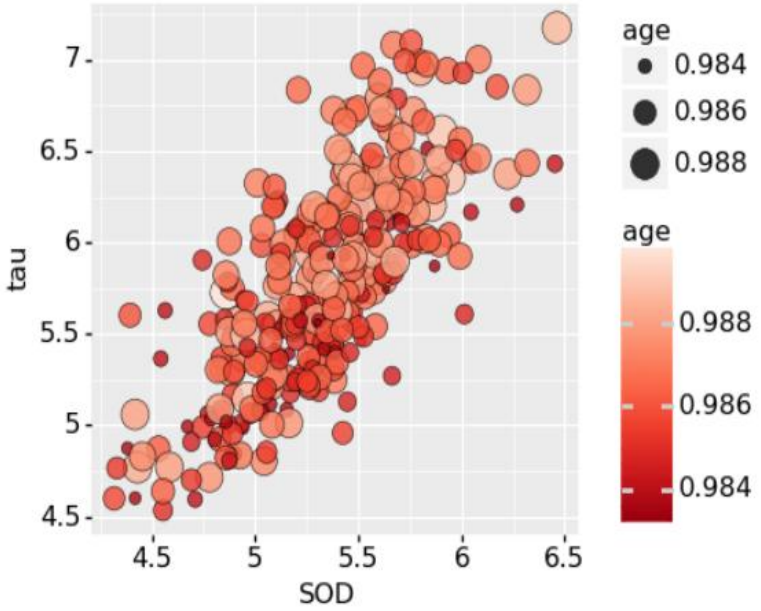
Plotnine: Scale parameter change




```
import pandas as pd
from plotnine import *
df=pd.read_csv('Facet_Data.csv')
#-----------------------------------------------------------------------------
p5=(ggplot(df,aes(x='SOD',y='tau',size='age'))+
      geom_point(shape='o',color='black',fill='#FF0000', stroke=0.25,alpha=0.8)+
      scale_size(range=(1,8))+
      theme(text=element_text(size=12,colour = "black"), aspect_ratio =1,
                dpi=100,   figure_size=(4,4)))
p5.show()
```



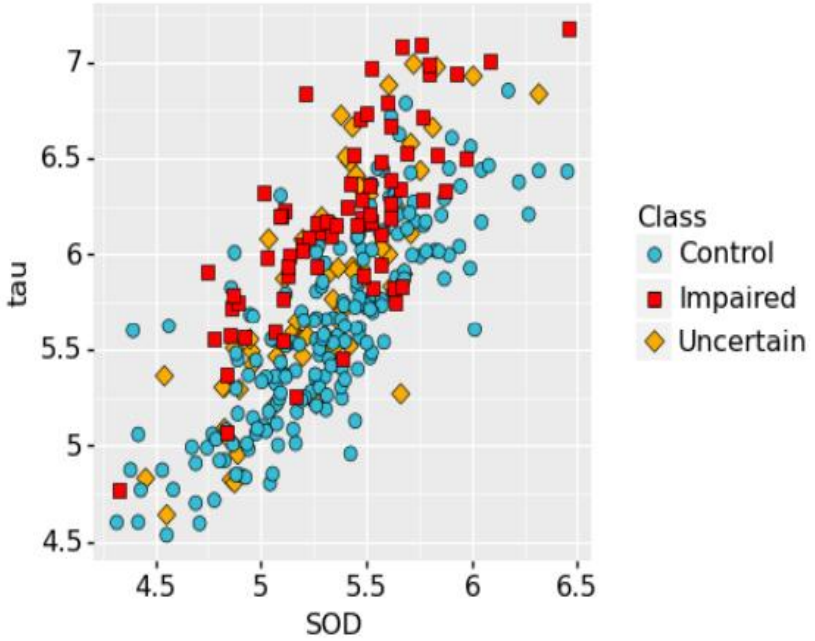

Plotnine: Scale parameter change




```
import pandas as pd
from plotnine import *
df=pd.read_csv('Facet_Data.csv')
#-------------------------------------------------------------------------------
p6=(ggplot(df,aes(x='SOD',y='tau',fill='age',size='age'))+
      geom_point(shape='o',color='black',stroke=0.25,alpha=0.8)+
      scale_size(range=(1,8))+
      scale_fill_distiller(type='seq',palette='Reds')+
      theme(text=element_text(size=12,colour = "black"), aspect_ratio =1,
                dpi=100,  figure_size=(4,4)))
p6.show()
#p6.save("plotnine6.pdf")
```

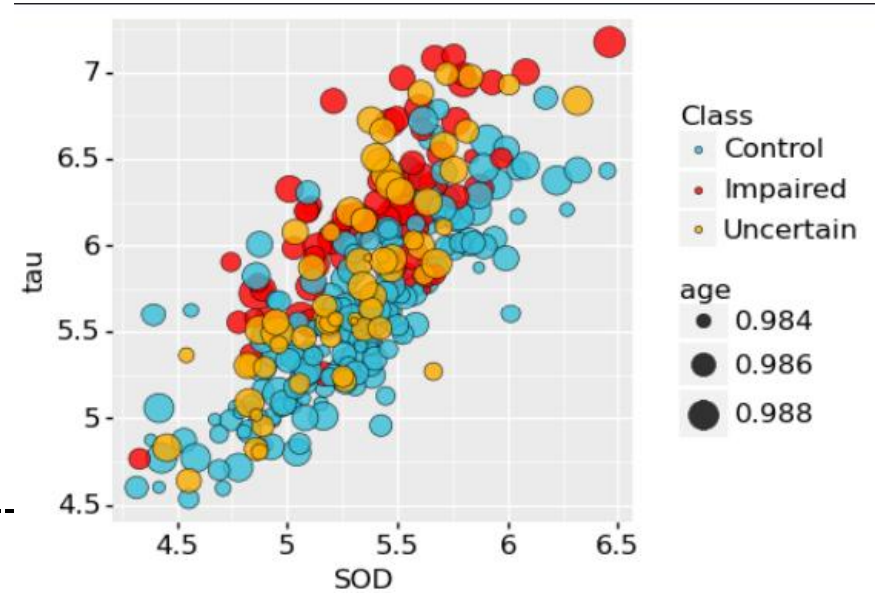
Plotnine: Scale parameter change

```
import pandas as pd
from plotnine import *
df=pd.read_csv('Facet_Data.csv')
#-------------------------------------------------------------------------------
p7=(ggplot(df,aes(x='SOD',y='tau',fill='Class',shape='Class'))+
      geom_point(size=3,color='black',stroke=0.25)+
      scale_fill_manual(values=('#36BED9','#FF0000','#FBAD01'))+
      scale_shape_manual(values=('o','s','D'))+
      theme(text=element_text(size=12,colour = "black"),
            aspect_ratio=1,   dpi=100,
            figure_size=(4,4)))
p7.show()
#p7.save("plotnine7.pdf")
```



Plotnine: Scale parameter change

```
import pandas as pd
from plotnine import *
df=pd.read_csv('Facet_Data.csv')
#-------------------------------------------------------------------------------
p8=(ggplot(df,aes(x='SOD',y='tau',size='age',fill='Class'))+
      geom_point(shape='o',color='k',stroke=0.25,alpha=0.8)+
      scale_fill_manual(values=('#36BED9','#FF0000','#FBAD01'))+
                        scale_size(range=(1,8))+
      theme(text=element_text(size=12,color='black'),  aspect_ratio=1,
            dpi=100,  figure_size=(4,4)))
p8.show()
#p7.save("plotnine7.pdf")
```



Plotnine: Layer usage rules

To proficiently use Plotnine for creating visualizations, a deep understanding of the relationship between ggplot and geom objects is essential. There are three scenarios where various relationships can occur:

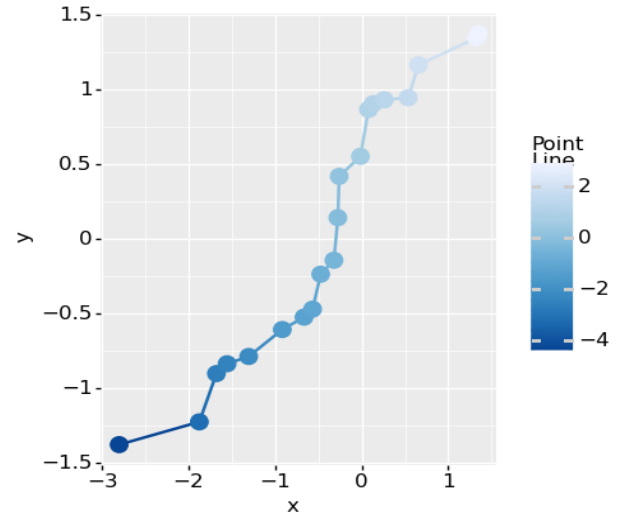
- All layers share the same data source and aesthetic mapping parameters.
- All layers share only the data source.
- Each layer uses independent data sources and aesthetic mapping parameters.

The relationship between these two primarily manifests in two aspects:

- ggplot(data=NULL, mapping=aes()): Within ggplot, the data and mapping parameters have global priority and can be inherited by all subsequent geom objects (provided the relevant parameters are not specified within the geom).
- geom_xxx(data=NULL, mapping=aes()): Similarly, within geom objects, there are data and mapping parameters. However, the data and mapping parameters within geom are considered local parameters and only affect the internal workings of the geom object.



Plotnine: Layer usage rules

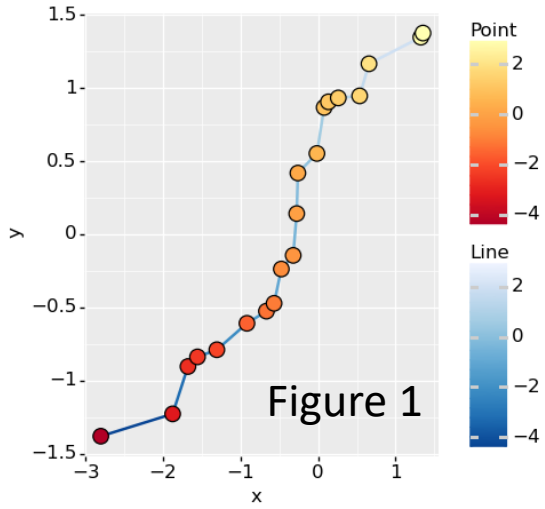


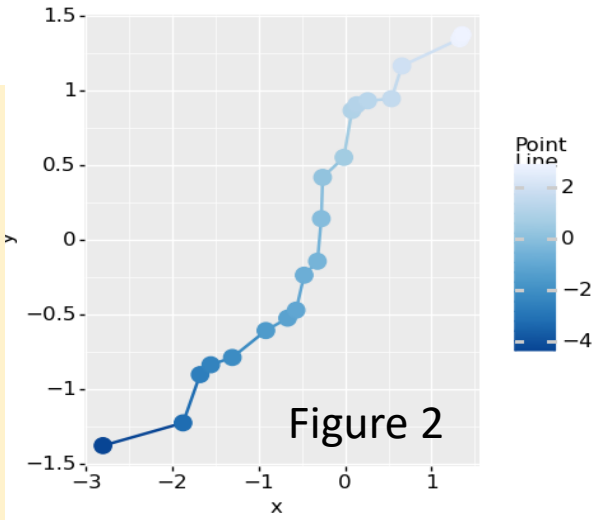

```
import pandas as pd
import numpy as np
from plotnine import *
import matplotlib.pyplot as plt
plt.rc('font',family='Times New Roman')
N=20
df1 =pd.DataFrame(dict(x=np.sort(np.random.randn(N)),y=np.sort(np.random.randn(N))))
df2 =pd.DataFrame(dict(x=df1.x+0.3*np.sort(np.random.randn(N)),y=df1.y+0.1*np.sort(np.random.randn(N))))

#----------1. All layers share the same data source and aesthetic mapping parameters.----------
p1=(ggplot(df1,aes('x','y',colour='x+y'))+
      geom_line(size=1)+
      geom_point(shape='o',size=5)+
      scale_color_distiller(name='Line',palette='Blues')+
      guides(color=guide_colorbar(title='Point\nLine'))+
      theme(text=element_text(size=12,color='k'), aspect_ratio=1.25,dpi=100,figure_size=(4,4)))
p1.show()
#p1.save('plotnine_Advance1.pdf')
```

Plotnine: Layer usage rules

```
#----------------------2. All layers share only the data source.----------------------
p2=(ggplot(df1,aes('x','y'))+
      geom_line(aes(color='x+y'),size=1)+
      geom_point(aes(fill='x+y'),color='k',shape='o',size=5)+
      scale_fill_distiller(name='Point',palette="YlOrRd")+
      scale_color_distiller(name='Line',palette='Blues')+
      theme(text=element_text(size=12,color='k'),
            aspect_ratio=1.25,dpi=100,figure_size=(4,4)))
p2.show()
```



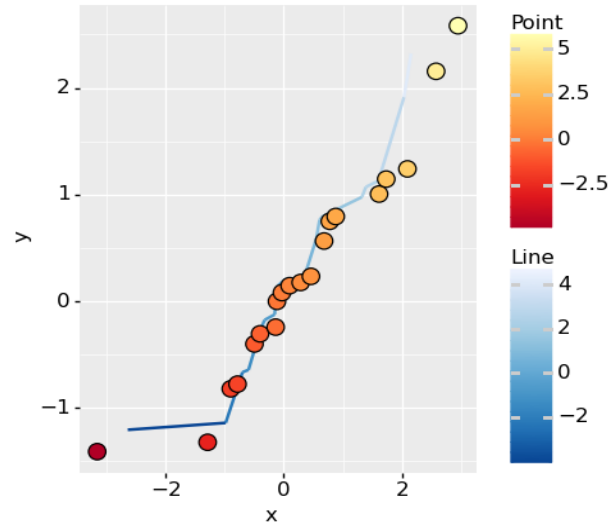


Comparison:
In Figure 1, the data and mapping parameters are specified in ggplot. According to the parameter inheritance rules, regardless of how many additional layers need to specify data and mapping later on, it is only necessary to specify them once within ggplot. Subsequent geoms will automatically inherit these parameters.

In Figure 2, only the data is shared, and aesthetic mapping parameters will be individually specified within the respective geom layers.

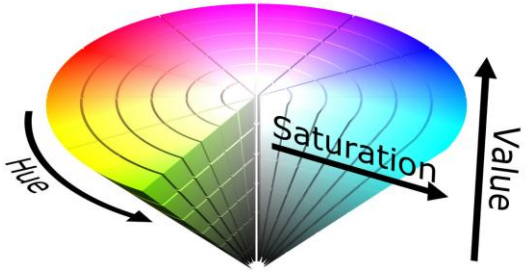
Plotnine: Layer usage rules

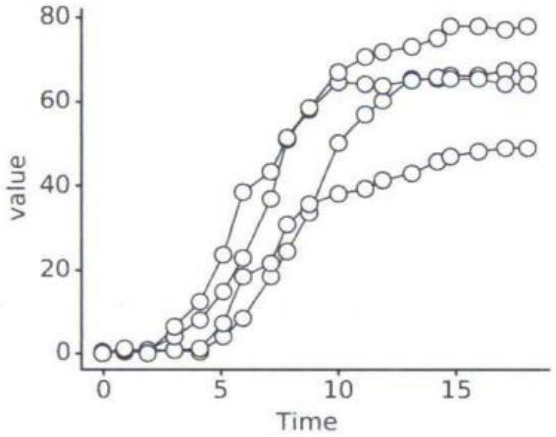
```
#----------------3. Each layer object uses independent data sources and aesthetic mapping parameters.-----------
p3=(ggplot()+
      geom_line(aes('x','y',color='x+y'),df1,size=1)+
      geom_point(aes('x','y',fill='x+y'),df2,color='k', shape='o',size=5)+
      scale_fill_distiller(name='Point',palette='YlOrRd')+
      scale_color_distiller(name='Line',palette='Blues')+
      theme(text=element_text(size=12,color='k'),
            aspect_ratio=1.25,dpi=100,figure_size=(4,4)))
p3.show()
#p3.save('plotnine_Advance3.pdf')
```



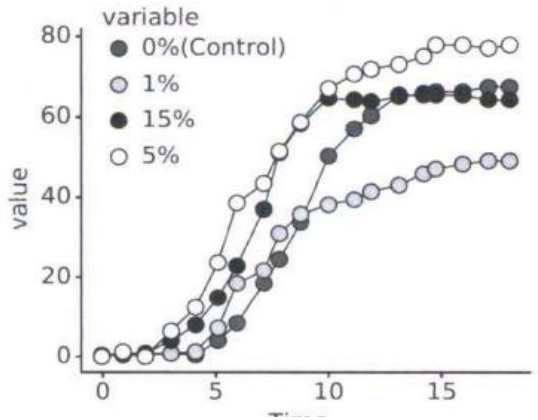
This is a special case that is only encountered when designing advanced plots or complex multi-layered geospatial visualizations. Since each layer does not share any data or mapping, if there are N layers that need to be mapped, all data and mapping parameters must be explicitly specified within each respective geom layer.

Plotnine: Scale parameter change

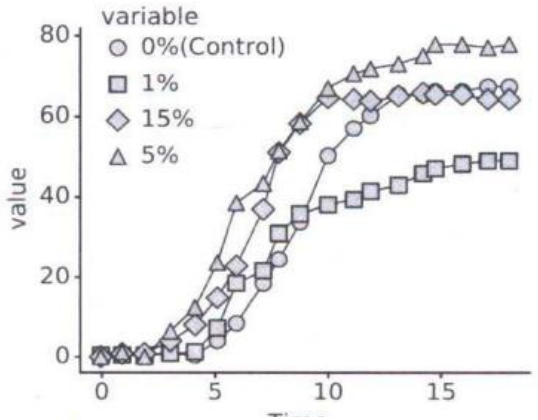




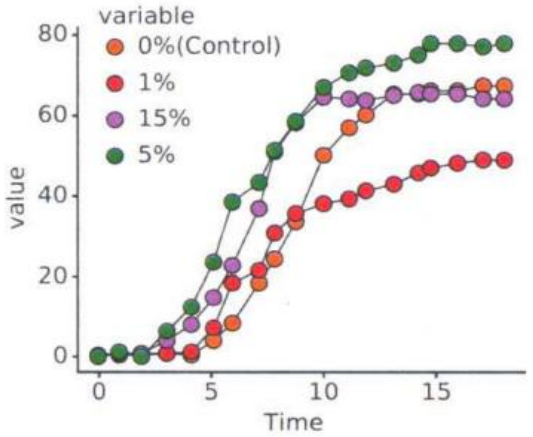
(a) Position + Direction



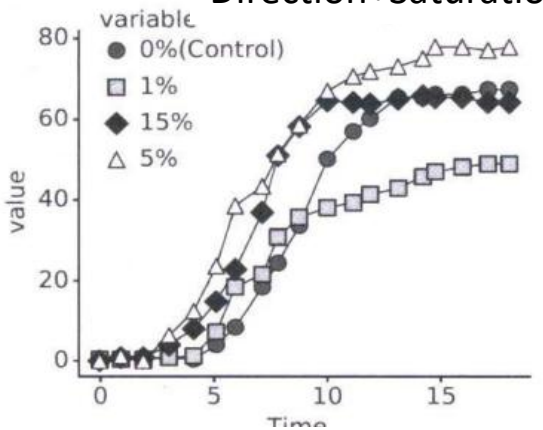
(b) Position + Direction+Saturation



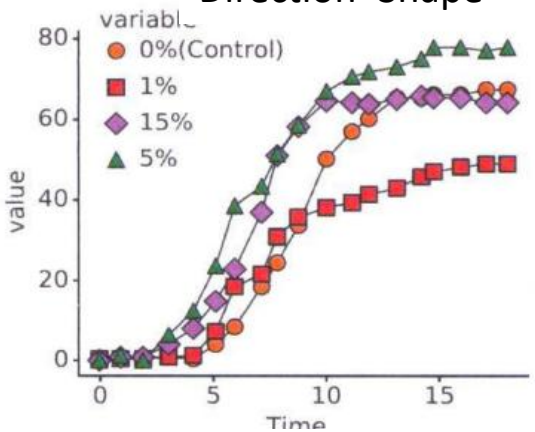
(c) Position + Direction+Shape



(d) Position + Direction+Hue



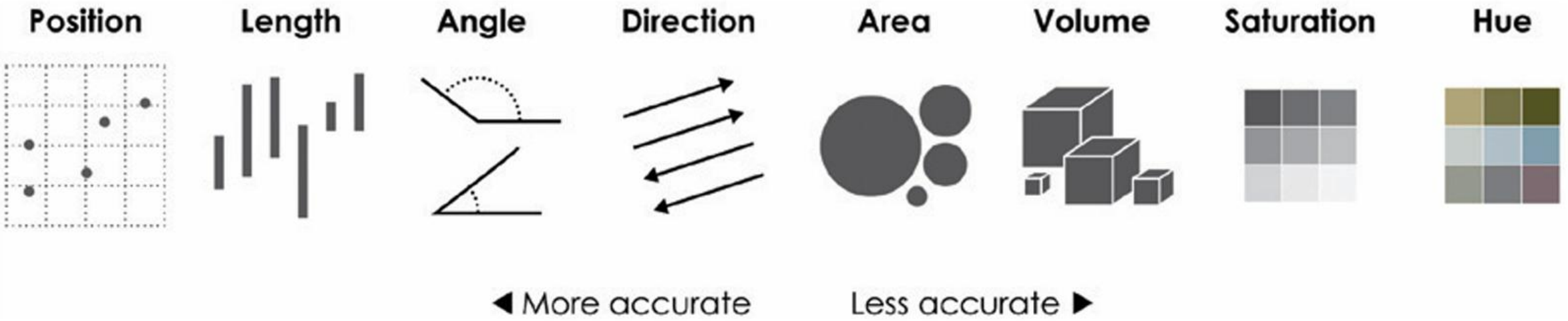
(e) Position + Direction+ Saturation+Shape



(f) Position + Direction+ Hue +Shape

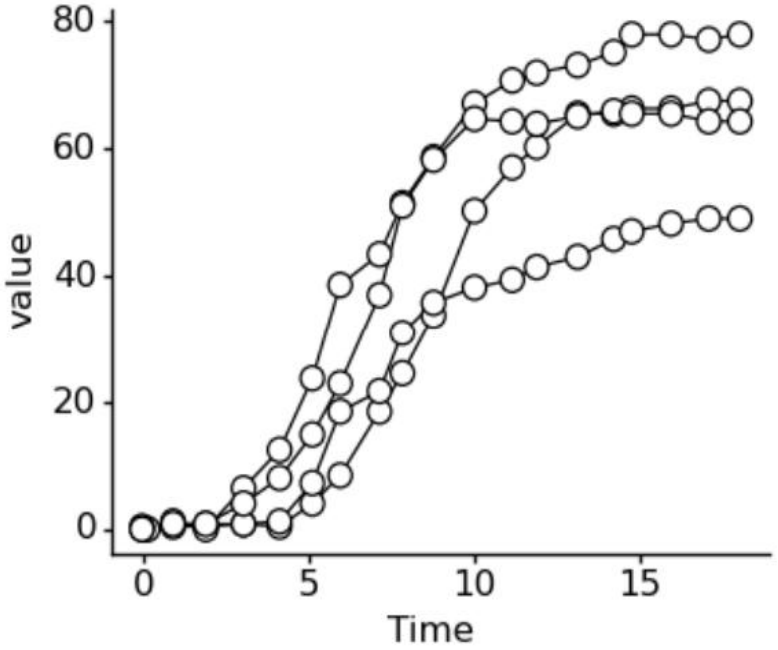
Plotnine: Scale parameter change

Visual cues typically include length, area, volume, angle, direction, shape, and color. The ability to choose appropriate visual cues depends on your understanding of shape, color, and size, as well as the data itself and the objective. Different types of information in charts should use different visual cues, and a reasonable combination of visual cues can better facilitate readers in understanding the data in the chart. There are 6 ways to combine different visual cues for the same dataset, as shown in the table below:



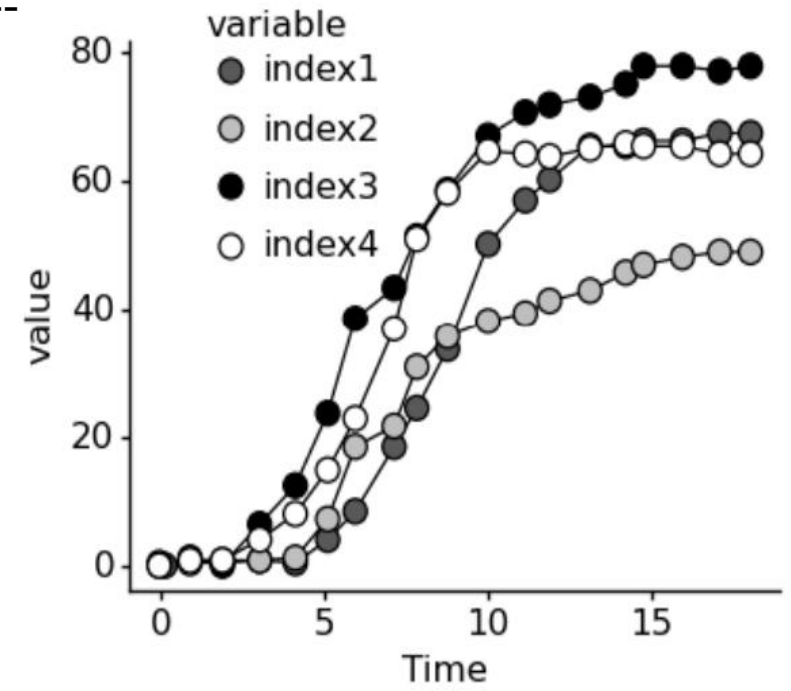
Plotnine: Scale parameter change

```
import pandas as pd
import numpy as np
from plotnine import *
df=pd.read_csv("MappingAnalysis_Data.csv")
#-----------------------(a)Position +Direction----------------------------
p1=(ggplot(df,aes(x='Time',y='value',group='variable'))+
      geom_line()+
      geom_point(shape='o',size=4,color='k',fill='white')+
      theme_classic()+
      theme(text=element_text(size=12,color='k'),aspect_ratio=0.8,
            dpi=100,  figure_size=(4,4)))
p1.show()
#p1.save(plotnine(a).pdf)
```



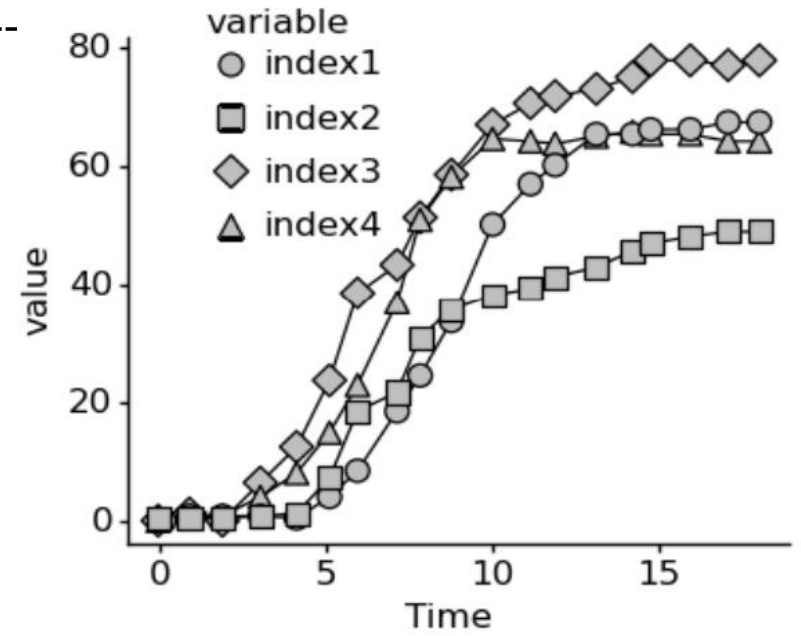
Plotnine: Scale parameter change

```
#------------------(b) Position + Direction+Saturation----------------------------
p2=(ggplot(df,aes(x='Time',y='value',fill='variable'))+
      geom_line()+
      geom_point(shape='o',size=4,color='k')+
      scale_fill_manual(values=('#595959','#BFBFBF','black','white'))+
      theme_classic()+
      theme(text=element_text(size=12,color='k'),aspect_ratio=0.8,
            dpi=100, figure_size=(4,4),
            legend_position=(0.32,0.75),
            legend_background=element_rect(fill='none')))
p2.show()
#p2.save(plotnine(b).pdf)
```



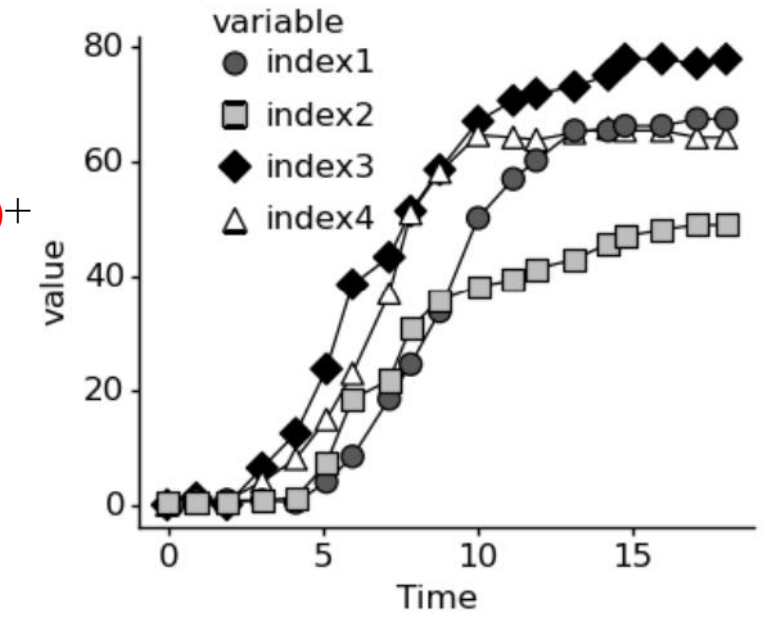
Plotnine: Scale parameter change

```
#------------------------------(c) Position + Direction+Shape----------------------
p3=(ggplot(df,aes(x='Time',y='value',shape='variable'))+
      geom_line()+
      geom_point(size=4,color='k',fill='#BFBFBF')+
      scale_shape_manual(values=('o','s','D','^'))+
      theme_classic()+
      theme(text=element_text(size=12,color='k'),aspect_ratio=0.8,
            dpi=100, figure_size=(4,4),
            legend_position=(0.32,0.75),
            legend_background=element_rect(fill='none')))
p3.show()
#p3.save(plotnine(c).pdf)
```



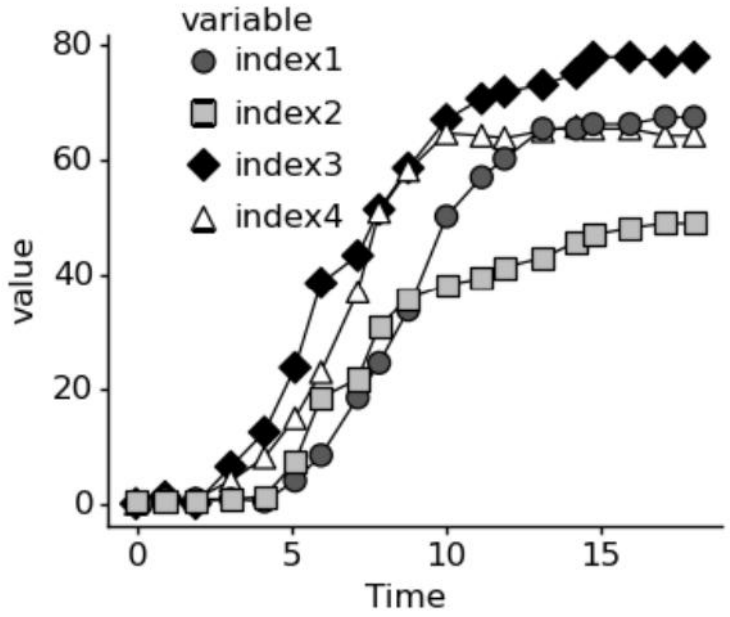
Plotnine: Scale parameter change

```
#-------------------------(d) Position + Direction+Hue-----------------------------
p4=(ggplot(df,aes(x='Time',y='value',fill='variable'))+
      geom_line()+
      geom_point(shape='o',size=4,color='k')+
      scale_fill_manual(values=('#FF9641','#FF5B4E','#B887C3','#38C25D'))+
      theme_classic()+
      theme(text=element_text(size=12,color='k'),aspect_ratio=0.8,
            dpi=100,  figure_size=(4,4),
            legend_position=(0.32,0.75),
            legend_background=element_rect(fill='none')))
p4.show()
#p4.save(plotnine(d).pdf)
```



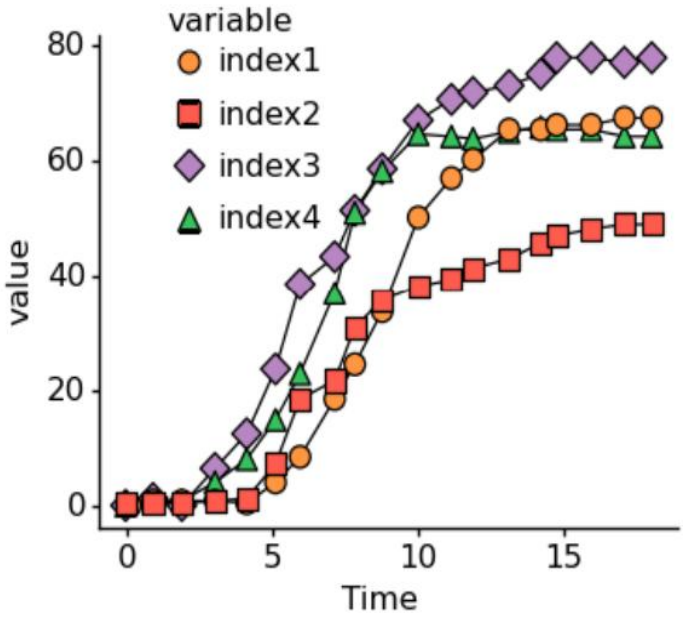
Plotnine: Scale parameter change

```
#-----------(e) Position + Direction+Saturation+Shape---------------------------------
p5=(ggplot(df,aes(x='Time',y='value',shape='variable',fill='variable'))+
      geom_line()+
      geom_point(size=4,color='k')+
      scale_fill_manual(values=("#595959",'#BFBFBF','black','white')) +
      scale_shape_manual(values=('o','s','D','^'))+
      theme_classic()+
      theme(text=element_text(size=12,color='k'),aspect_ratio=0.8,
            dpi=100,figure_size=(4,4),
            legend_position=(0.32,0.75),
            legend_background=element_rect(fill='none')))
p5.show()
#p5.save(plotnine(e).pdf)
```



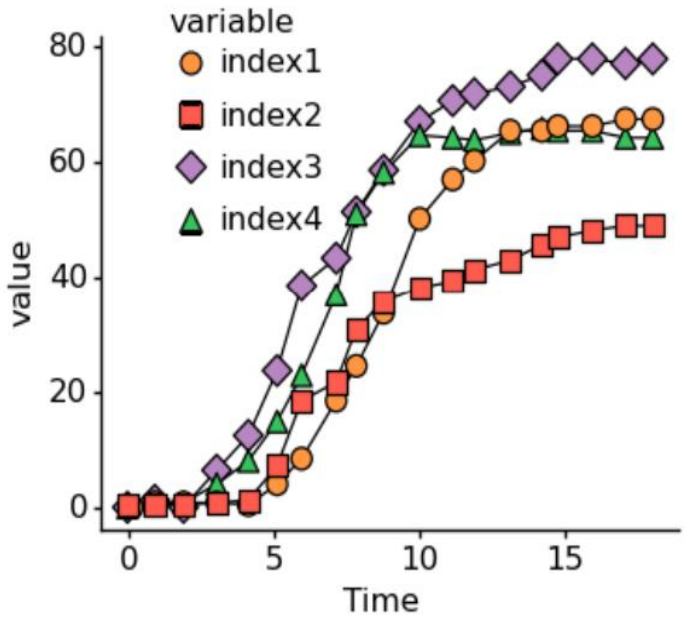
Plotnine: Scale parameter change

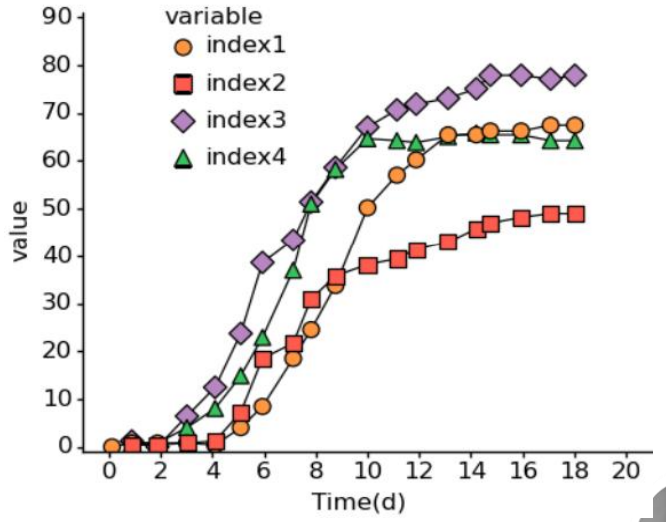
```
#--------------(f) Position + Direction+Hue +Shape------------------------------
p6=(ggplot(df,aes(x='Time',y='value',shape='variable',fill='variable'))+
      geom_line()+
      geom_point(size=4,color='k')+
      scale_fill_manual(values=('#FF9641','#FF5B4E','#B887C3','#38C25D'))+
      scale_shape_manual(values=('o','s','D','^'))+
      theme_classic()+
      theme(text=element_text(size=12,color='k'),aspect_ratio=0.8,
            dpi=100,  figure_size=(4,4),
            legend_position=(0.32,0.75),
            legend_background=element_rect(fill='none')))
p6.show()
#p6.save(plotnine(f).pdf)
```



Plotnine: Scale parameter change

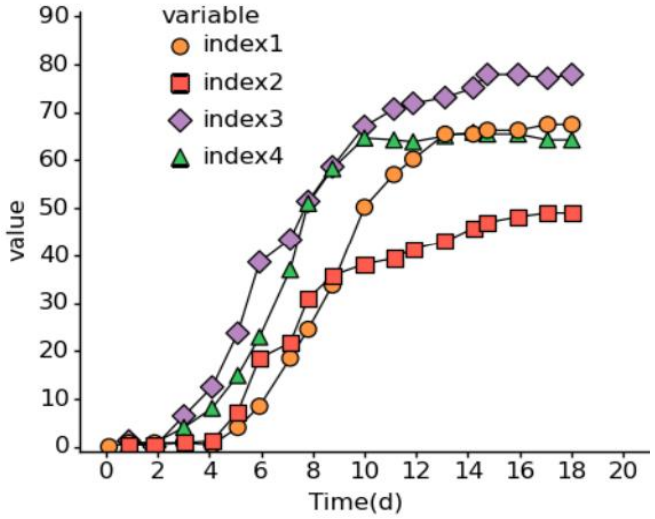
```
#-----(g) Position + Direction+Hue +Shape+ Coordinate axes scale refinement----------
p7=(ggplot(df,aes(x='Time',y='value',shape='variable',fill='variable')) +
      geom_line()+
      geom_point(size=4,colour="black") +
      scale_fill_manual(values=("#FF9641","#FF5B4E","#B887C3","#38C25D"))+
      scale_shape_manual(values=('o','s','D','^'))+
      scale_x_continuous(name="Time(d)",breaks=np.arange(0,21,2),limits=(0,20))+
      scale_y_continuous(breaks=np.arange(0,91,10),limits=(0,90),expand =(0, 1))+
      theme_classic()+
      theme(text=element_text(size=12,colour = "black"),
            aspect_ratio =0.8,  dpi=100,  figure_size=(5,5),
            legend_position=(0.32,0.75),
            legend_background=element_rect(fill="none")))
p7.show()
#p7.save("plotnine(g).pdf")
```

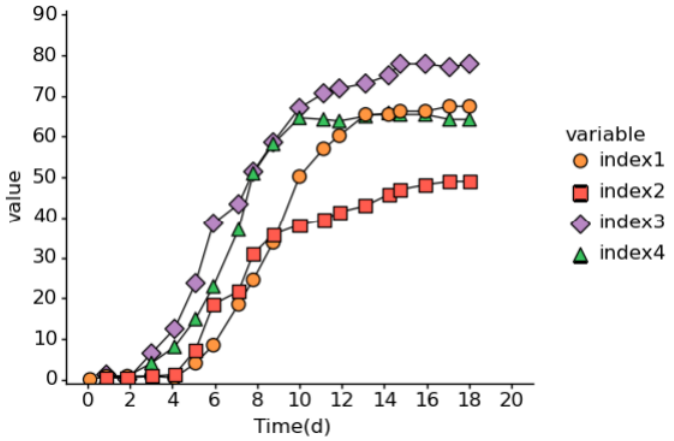




Plotnine: Scale parameter change

```
#h) Position + Direction+Hue +Shape+ Coordinate axes scale refinement +Legend position adjustment----------
p8=(ggplot(df,aes(x='Time',y='value',shape='variable',fill='variable')) +
      geom_line()+
      geom_point(size=4,colour="black") +
      scale_fill_manual(values=("#FF9641","#FF5B4E","#B887C3","#38C25D"))+
      scale_shape_manual(values=('o','s','D','^'))+
      scale_x_continuous(name="Time(d)",breaks=np.arange(0,21,2),limits=(0,20))+
      scale_y_continuous(breaks=np.arange(0,91,10),limits=(0,90),expand =(0, 1))+
      theme_classic()+
      theme(text=element_text(size=12,colour = "black"),
            aspect_ratio =0.8, dpi=100,
            figure_size=(5,5)))
p8.show()
#p8.save("plotnine(h).pdf")
```





Plotnine: Faceted Chart

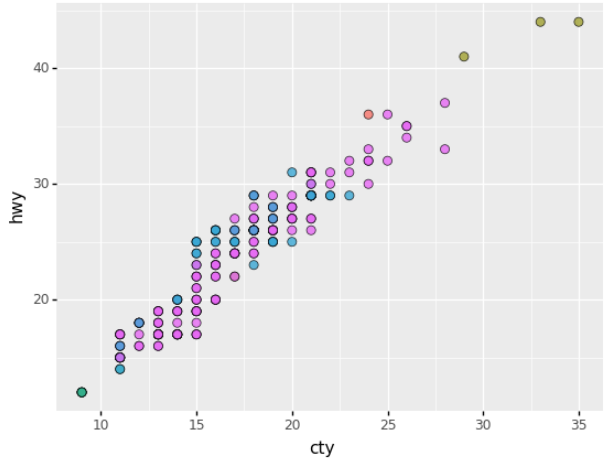
Faceted system

When representing three-dimensional or four-dimensional data in three-dimensional charts, it may not be easy to clearly observe data patterns and present data information. Using faceted plots can be a solution.

In plotnine, there are two functions, **facet_wrap()** and **facet_grid()**, that can be used to draw a series of subplots based on categorical attributes, such as matrix faceting, row faceting, and column faceting.

Faceted plots involve creating subplots based on data categories in rows or columns, using basic chart types like scatter plots, bubble charts, bar charts, or line charts to reveal relationships between the data.

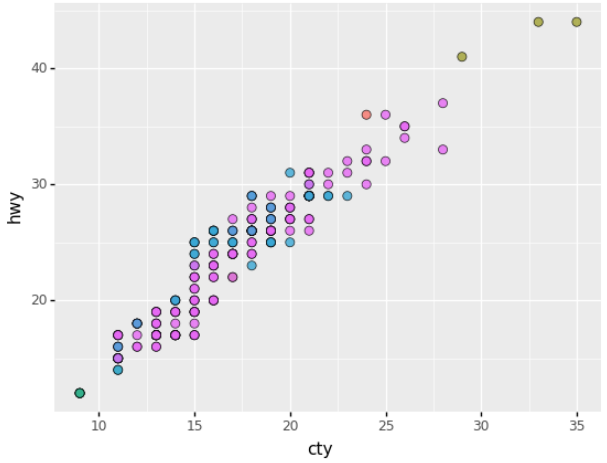
```
from plotnine import *
from plotnine.data import mpg
t=(ggplot(mpg,aes('cty','hwy',fill='fl'))+
    geom_point(size=3, stroke=0.3, alpha=0.8, show_legend=False)+
    scale_fill_hue(s=0.9, l=0.65, h=0.0417,color_space='husl'))
```



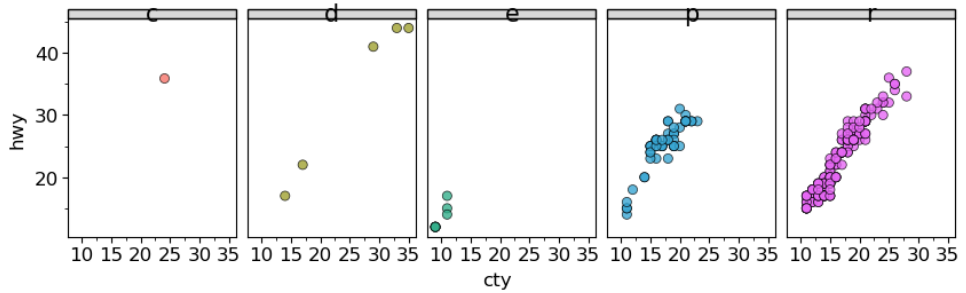
Plotnine: Faceted Chart

Faceted system

```
from plotnine import *
from plotnine.data import mpg
t=(ggplot(mpg,aes('cty','hwy',fill='fl'))+
    geom_point(size=3, stroke=0.3, alpha=0.8, show_legend=False)+
    scale_fill_hue(s=0.9, l=0.65, h=0.0417,color_space='husl'))
```



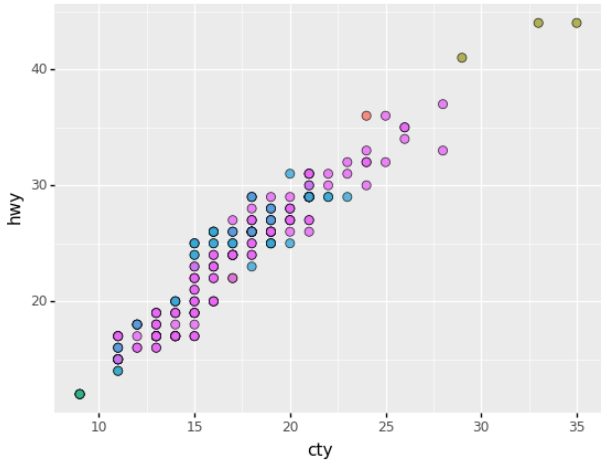

```
 #-----------------------Arrange based on variables in columns-----------------------------------
t1=(t + facet_grid('.~fl')
     + theme_matplotlib()
     + theme(text=element_text(size=12, colour = 'black'),
                strip_text=element_text(size=15, colour='black'),
                strip_background=element_rect(color='k'),
                aspect_ratio = 1.3,  dpi=100, figure_size=(10,10)))
t1.show()
```



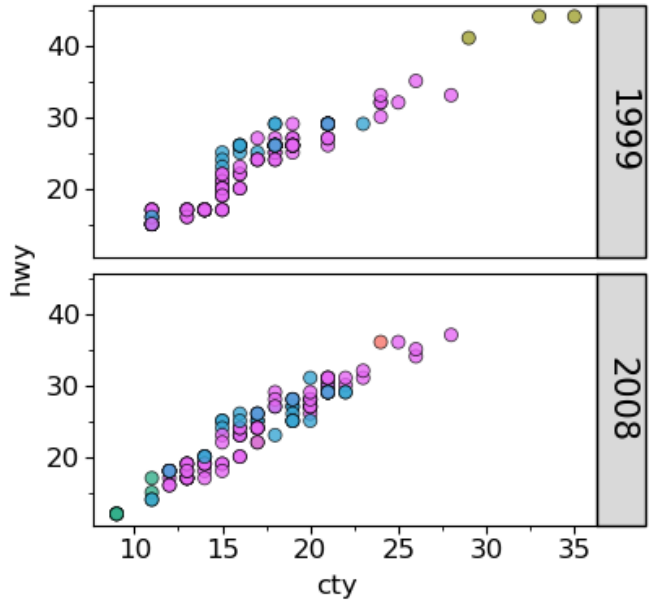
Plotnine: Faceted Chart

Faceted system

```
from plotnine import *
from plotnine.data import mpg
t=(ggplot(mpg,aes('cty','hwy',fill='fl'))+
    geom_point(size=3, stroke=0.3, alpha=0.8, show_legend=False)+
    scale_fill_hue(s=0.9, l=0.65, h=0.0417,color_space='husl'))
```



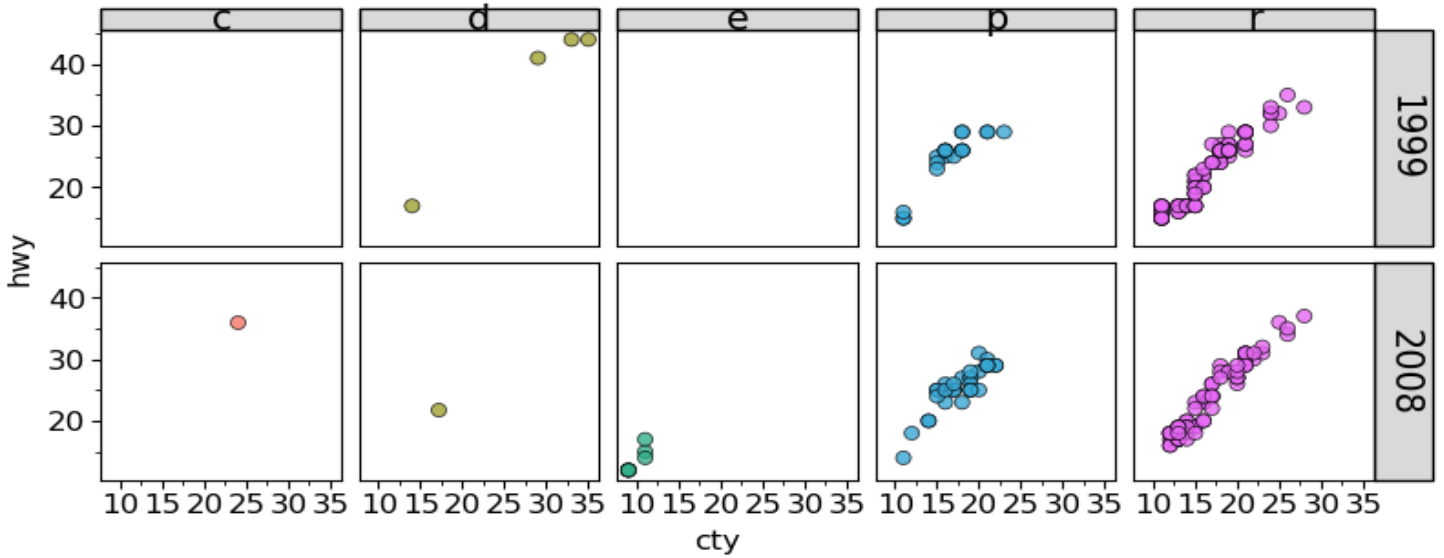

```
 #-----------------------Arrange based on variables in rows-----------------------------------
t2 = (t + facet_grid('year ~ .')
      + theme_matplotlib()
      + theme(text=element_text(size=12, colour = 'black'),
                strip_text=element_text(size=15, colour='black'),
                strip_background=element_rect(color='k'),
                aspect_ratio = 0.5,
                dpi=100,
                figure_size=(4,4)))
t2.show()
```



Plotnine: Faceted Chart

Faceted system

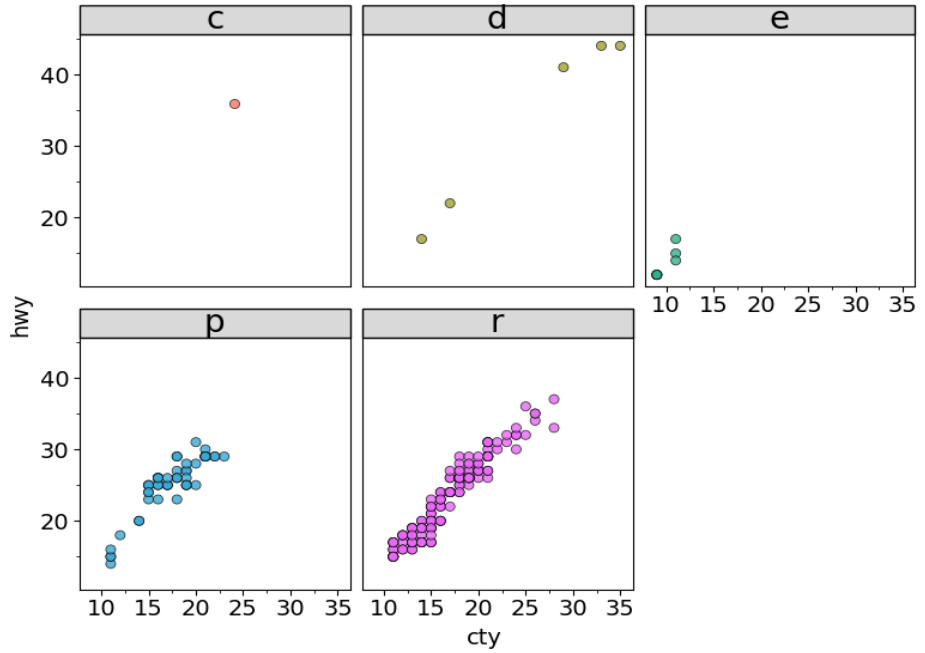
```
#------------------------Arrange based on two variables in a matrix layout.----------------------------
t3 = (t + facet_grid('year ~fl')
      + theme_matplotlib()
      + theme(text=element_text(size=12,color='black'),
              strip_text=element_text(size=15,color='black'),
              strip_background=element_rect(color='k'),
              aspect_ratio =1,
              dpi=100,
              figure_size=(9,9)))
t3.show()
```



Plotnine: Faceted Chart

Faceted system

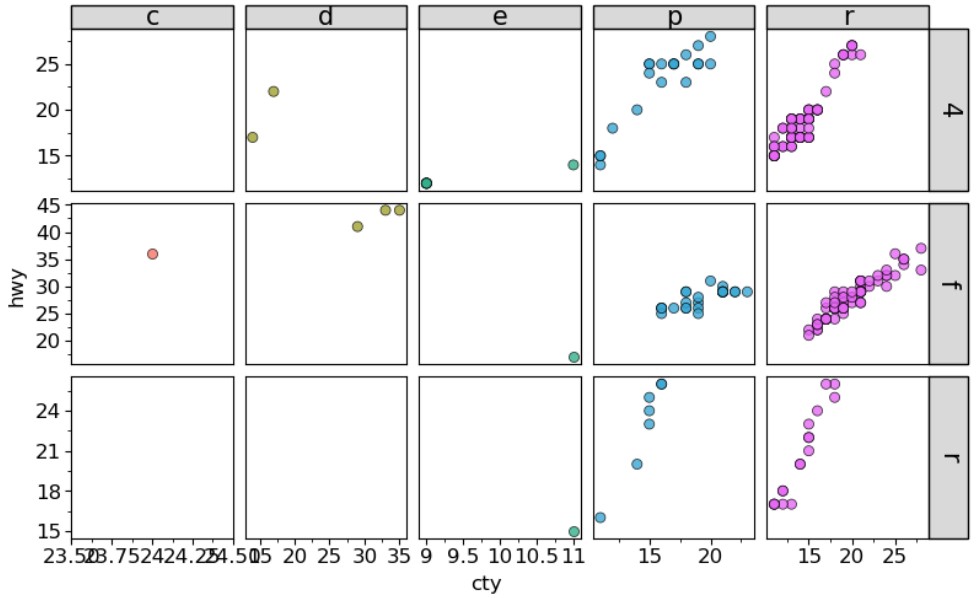
```
#-------------------------Arrange based on variables in a rectangular layout.----------------------
t4 = (t + facet_wrap('~ fl')
      + theme_matplotlib()
      + theme(text=element_text(size=14, color='black'),
                strip_text=element_text(size=20,color='black'),
                strip_background=element_rect(color='k'),
                aspect_ratio=1,
                dpi=100,
                figure_size=(9,9)))
t4.show()
```



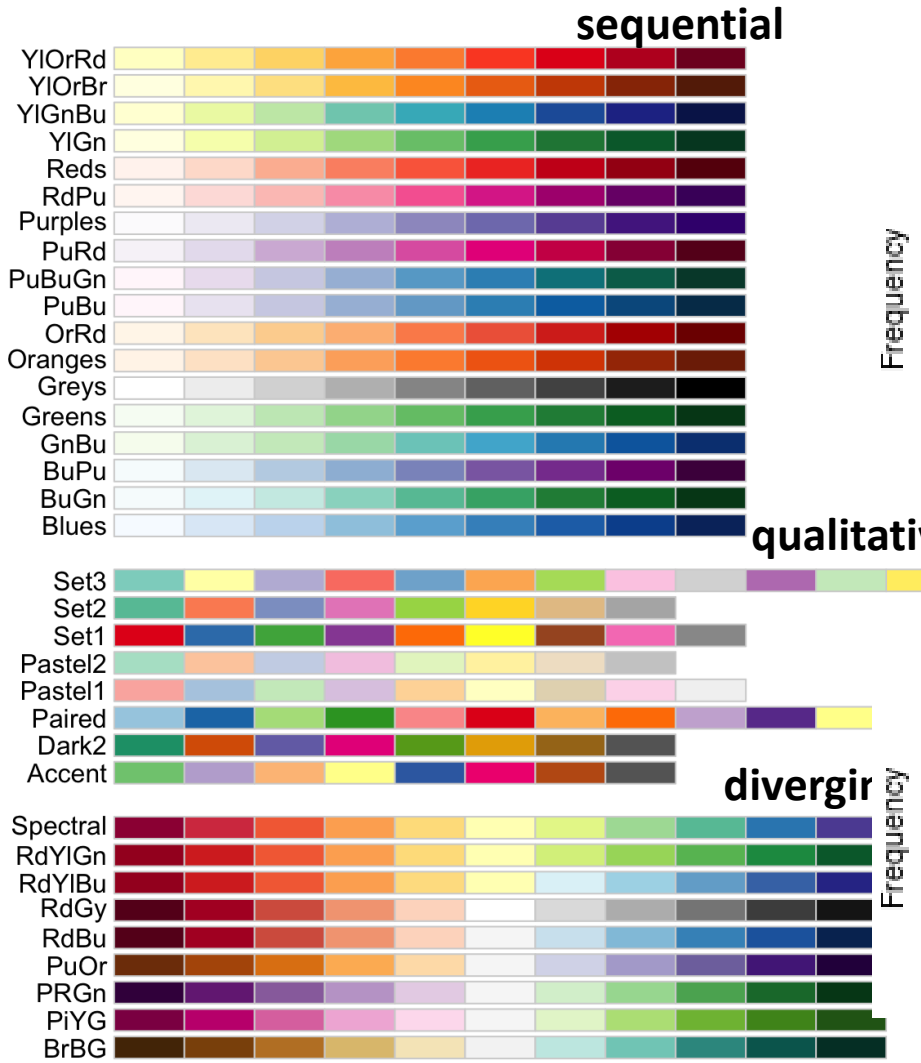
Plotnine: Faceted Chart

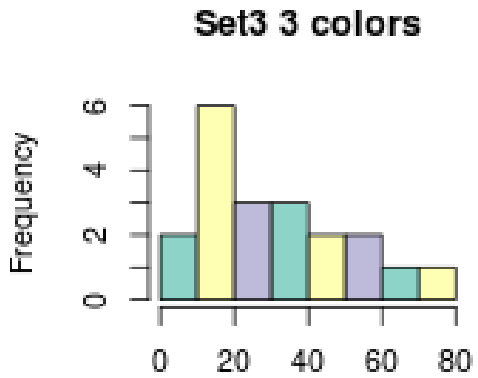
Faceted system

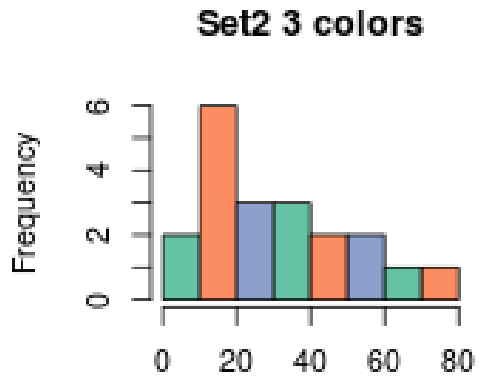
```
#---------------------------- Adjust the range of values on the X-axis and Y-axis. --------------------
t5 = (t + facet_grid('drv ~ fl',scales = 'free')
      + theme_matplotlib()
      + theme(text=element_text(size=12,color='black'),
              strip_text=element_text(size=15,color='black'),
              strip_background=element_rect(color='k'),
              aspect_ratio=1,
              dpi=100,
              figure_size=(9,9)))
t5.show()
#t5.save('分面5.pdf')
```

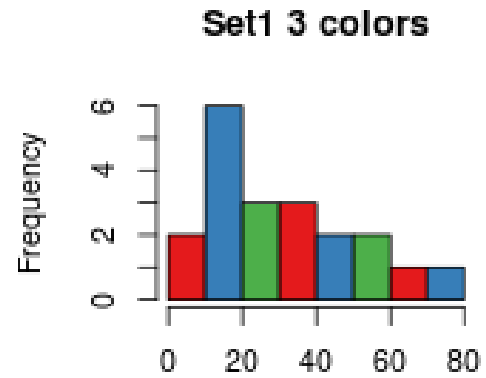


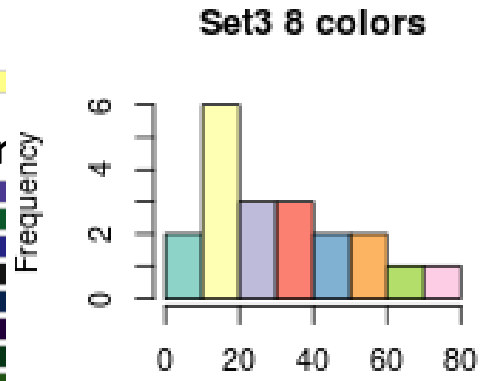
Plotnine: Color Theme

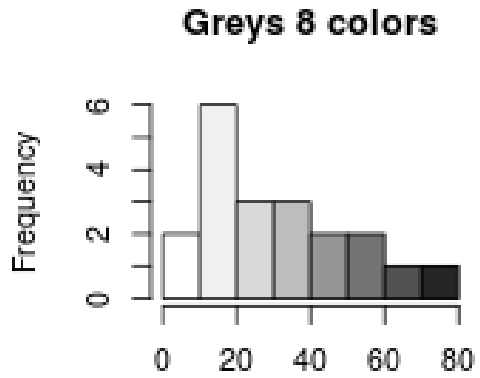


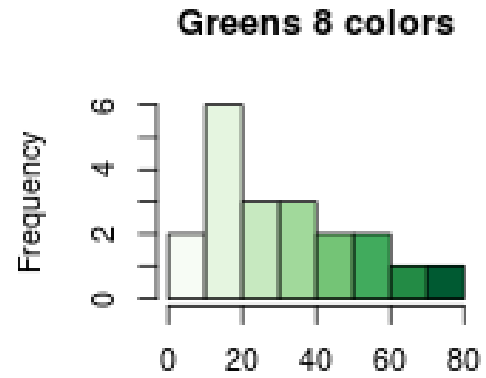














Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Discrete color themes

Codes	Description
(p+scale_fill_discrete())	**Default color themes**
(p+scale_fill_brewer(type='qualitative', palette='Set1'))	**Qualitative color themes: Set1**
(p+scale_fill_hue(s = 1, l = 0.65, h=0.0417,color_space='husl'))	**Discrete color themes: HSLuv**
(p+scale_fill_manual(values=("#E7298A","#66A61E","#E6AB02")))	**Manually designed colors: Hex**

Continues color themes

Codes	Description
(p+scale_fill_distiller(type='div',palette="RdYlBu"))	Diverging **color theme: RdYlBu**
(p+scale_fill_cmap(name='viridis'))	**Continues color theme: viridis**
(p+scale_fill_gradient2(low="#00A08A",mid="white", high="#FF0000",midpoint = np.mean(df.age)))	**Manually designed** Diverging color theme**: Hex**
(p++scale_fill_gradientn(colors=("#82C143","white","#CB1B81")))	**Manually designed colors: Hex**

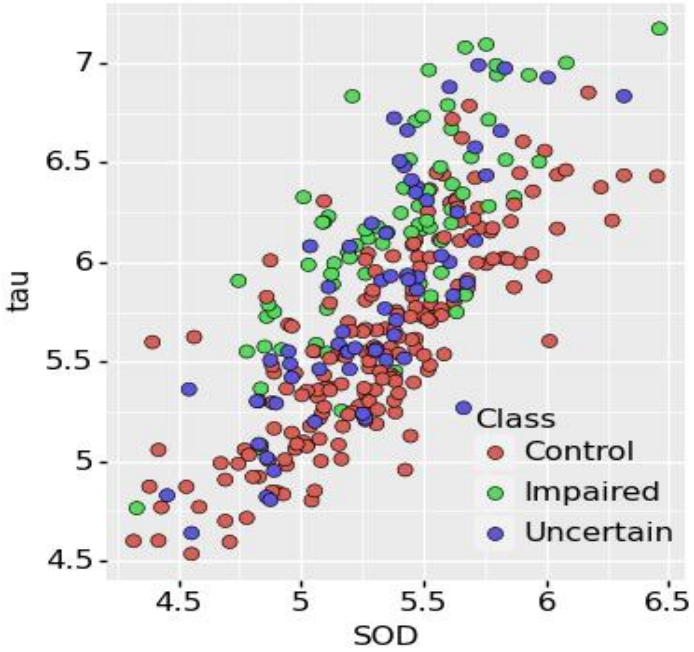


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Discrete color themes

```
import pandas as pd
import numpy as np
from plotnine import *
df=pd.read_csv("Facet_Data.csv")
#-------------------------(a) Default color themes----------------------------
p1=(ggplot(df,aes(x='SOD',y='tau',fill='Class'))
      +geom_point(shape='o',color='black',size=3,stroke=0.25,alpha=1)
      +scale_fill_discrete()    #plotnine默认配色方案
      +theme(text=element_text(size=12,color='k'),
             legend_background=element_blank(),    #图例背景设置为无
             legend_position=(0.75,0.25), aspect_ratio=1.15,
             dpi=100,figure_size=(4,4)))
print(p1)
```

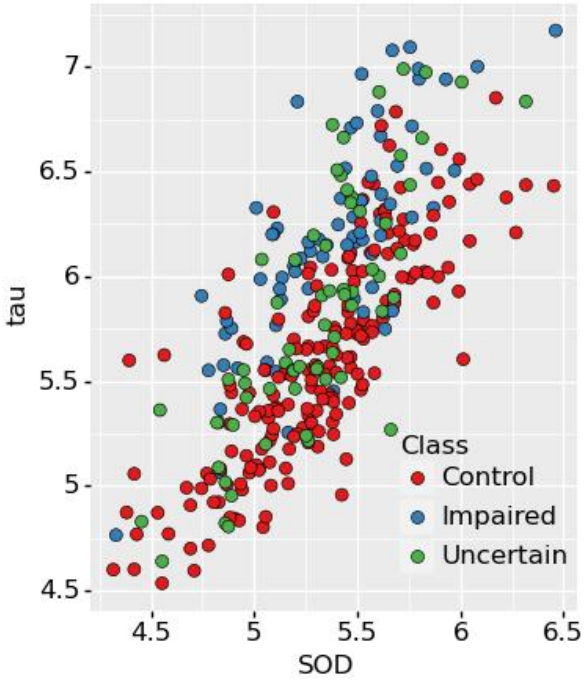


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Discrete color themes

```
#----------------(b) Qualitative color themes: Set1----------------------------
p2=(ggplot(df,aes(x='SOD',y='tau',fill='Class'))
    +geom_point(shape='o',color='k',size=3,stroke=0.25,alpha=1)
    +scale_fill_brewer(type='qualitative',palette='Set1')
    +theme(text=element_text(size=12,color='k'),
           legend_background=element_blank(),
           legend_position=(0.75,0.25),
           aspect_ratio=1.25,dpi=100,
           figure_size=(4,4)))
print(p2)
```

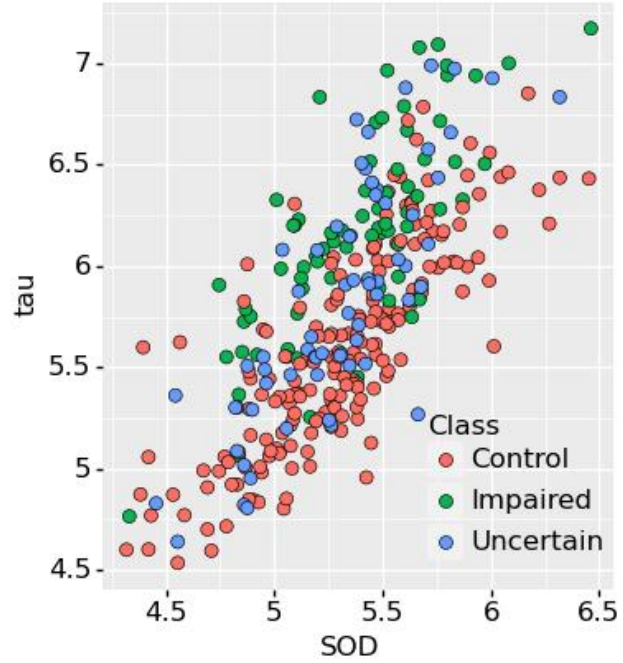


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Discrete color themes

```
#----------------(c) Discrete color themes: HSLuv---------------------------
p3=(ggplot(df,aes(x='SOD',y='tau',fill='Class'))
      +geom_point(shape='o',color='k',size=3,stroke=0.25,alpha=1)
      +scale_fill_hue(s=1,l=0.65,h=0.0417,color_space='husl')
      +theme(text=element_text(size=12,color='k'),
             legend_background=element_blank(),
             legend_position=(0.75,0.25),
             aspect_ratio=1.15,dpi=100,
             figure_size=(4,4)))
print(p3)
```

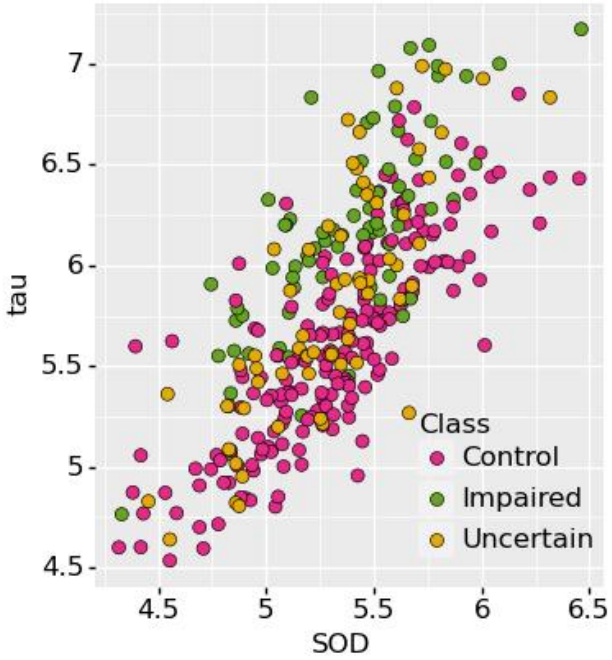


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Discrete color themes

```
#----------------(d) Manually designed colors: Hex-----------------------------
p4=(ggplot(df,aes(x='SOD',y='tau',fill='Class'))
      +geom_point(shape='o',color='k',size=3,stroke=0.25,alpha=1)
      +scale_fill_manual(values=('#E7298A','#66A61E','#E6AB02'))
      +theme(text=element_text(size=12,color='k'),
                legend_background=element_blank(),
                legend_position=(0.75,0.25),
                aspect_ratio=1.15,dpi=100,
                figure_size=(4,4)))
print(p4)
```

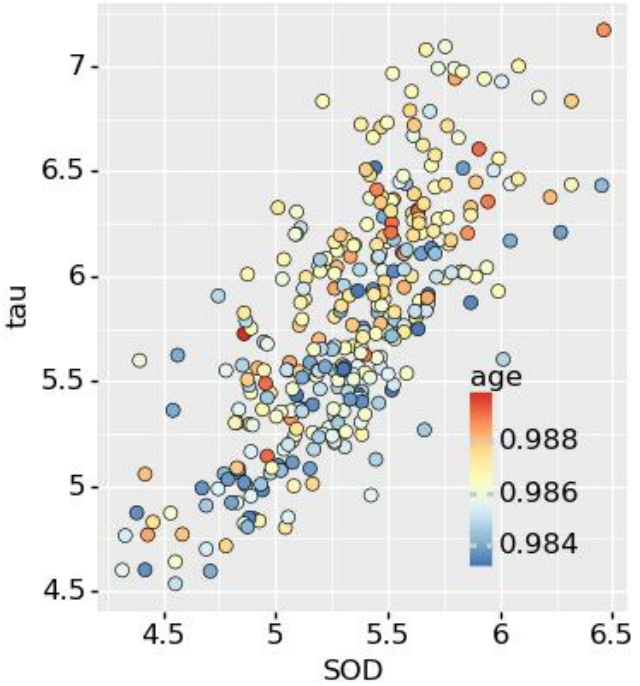


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Continues color themes

```
import pandas as pd
import numpy as np
from plotnine import *
df=pd.read_csv('Facet_Data.csv')
#--------------------------(a) Diverging color theme: RdYlBu ------------------
p5=(ggplot(df,aes(x='SOD',y='tau',fill='age'))
      +geom_point(shape='o',color='k',size=3,stroke=0.25,alpha=1)
      +scale_fill_distiller(type='div',palette='RdYlBu')
      +guides(fill=guide_colorbar(barheight=10,barwidth=6))
      +theme(text=element_text(size=12,color='k'),
             legend_background=element_blank(),
             legend_position=(0.75,0.3), aspect_ratio=1.15,dpi=100,
             figure_size=(4,4)))
print(p5)
```

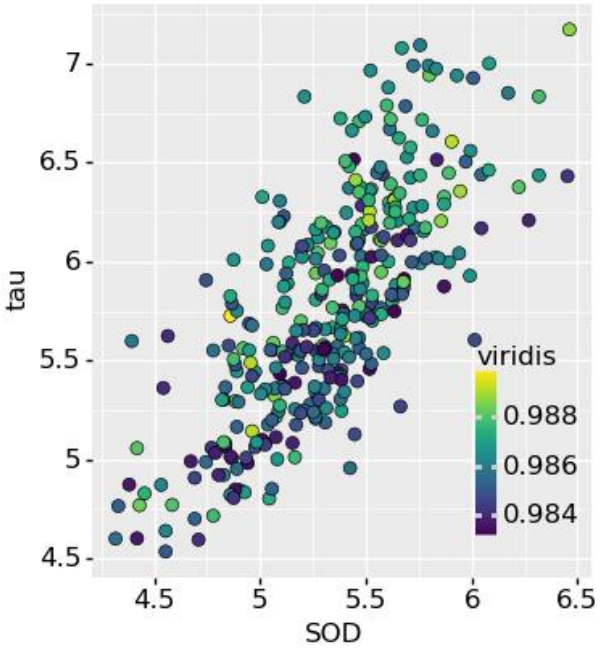


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Continues color themes

```
#-----------------------(b) Continues color theme: viridis-----------------------------
p6=(ggplot(df,aes(x='SOD',y='tau',fill='age'))
      +geom_point(shape='o',color='k',size=3,stroke=0.25,alpha=1)
      +scale_fill_cmap(name='viridis')
      +guides(fill=guide_colorbar(barheight=10,barwidth=6))
      +theme(text=element_text(size=12,color='k'),
                legend_background=element_blank(),
                legend_position=(0.8,0.3),
                aspect_ratio=1.15,dpi=100,
                figure_size=(4,4)))
print(p6)
```

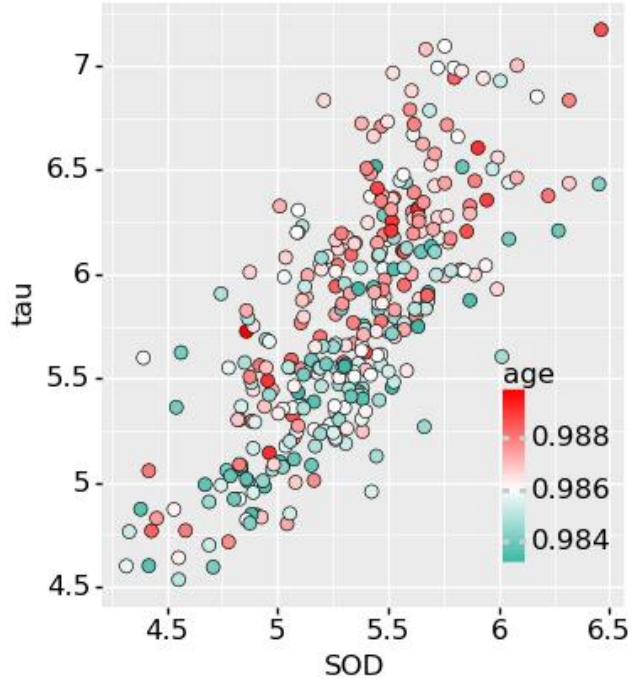


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Continues color themes

```
#----------------(c) Manually designed Diverging color theme: Hex----------
p7=(ggplot(df,aes(x='SOD',y='tau',fill='age'))
      +geom_point(shape='o',color='k',size=3,stroke=0.25,alpha=1)
      +scale_fill_gradient2(low='#00A08A',mid='white',high='#FF0000',
                            midpoint=np.mean(df.age))
      +guides(fill=guide_colorbar(barheight=10,barwidth=6))
      +theme(text=element_text(size=12,color='k'),
             legend_background=element_blank(),
             legend_position=(0.8,0.3),
             aspect_ratio=1.15,dpi=100,
             figure_size=(4,4)))
print(p7)
```

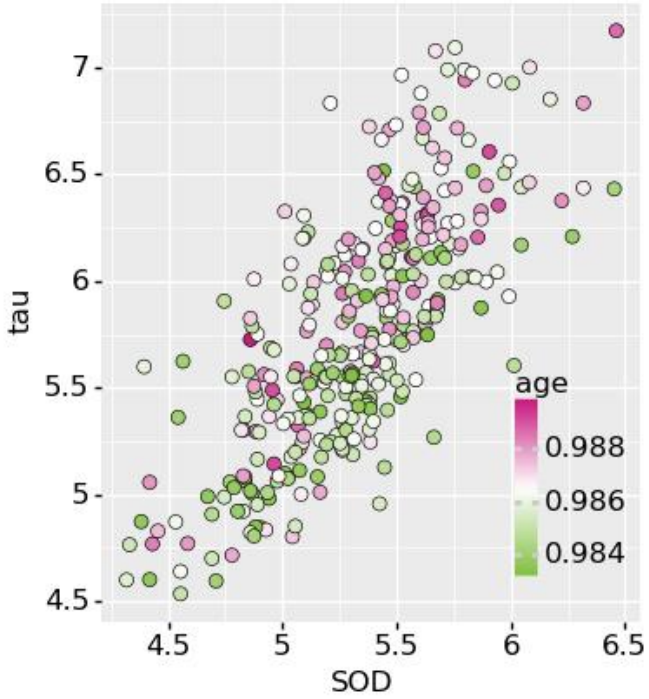


Plotnine: Color Theme

Based on the type of data mapped to the variables, the application of the scale_color/fill_*() functions for adjusting color metrics is mainly categorized into two types: discrete and continuous.

Continues color themes

```
#----------------------(d) Manually designed colors: Hex-------------------------
p8=(ggplot(df,aes(x='SOD',y='tau',fill='age'))
      +geom_point(shape='o',color='k',size=3,stroke=0.25,alpha=1)
      +scale_fill_gradientn(colors=('#82C143','White','#CB1B81'))
      +guides(fill=guide_colorbar(barheight=10,barwidth=6))
      +theme(text=element_text(size=12,color='k'),
                legend_background=element_blank(),
                legend_position=(0.8,0.3),
                aspect_ratio=1.15,dpi=100,
                figure_size=(4,4)))
print(p8)
```



Plotnine: Position Adjustment

```
import pandas as pd
import numpy as np
from plotnine import *
N=100
df=pd.DataFrame(dict(group=np.repeat([1,2],N*2),

y=np.append(np.append(np.random.normal(5,1,N),np.random.normal(2,1,N)),
                      np.append(np.random.normal(1,1,N),np.random.normal(3,1,N))),
                    x=np.tile(['A','B','A','B'],N)))
print(df)
```

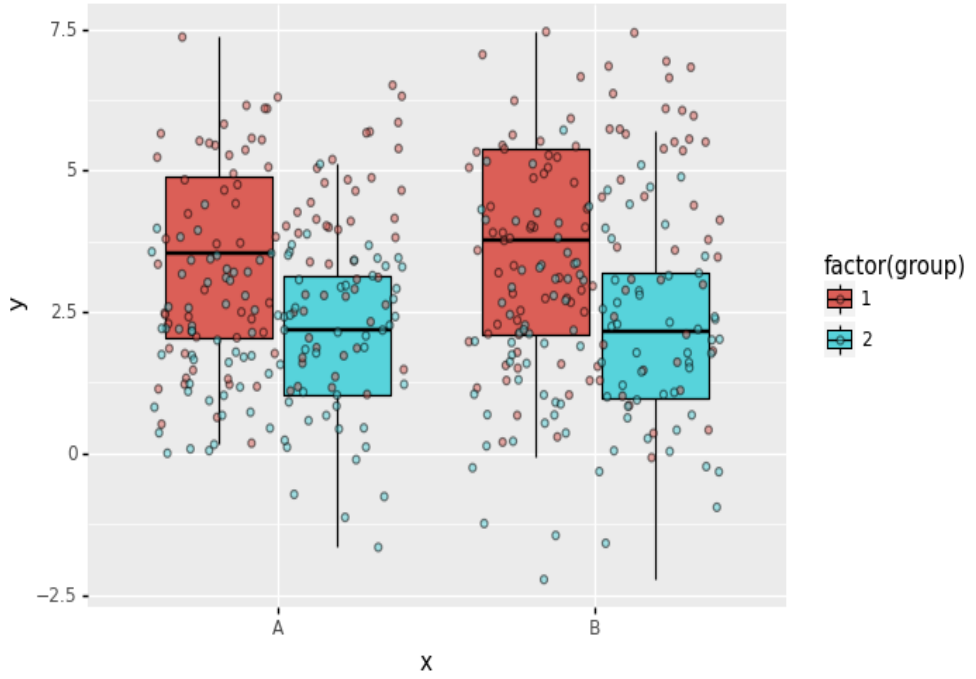
```
     group         y  x
0        1  4.153523  A
1        1  5.268806  B
2        1  5.848146  A
3        1  4.866941  B
4        1  4.942788  A
..     ...       ... ..
395      2  2.184278  B
396      2  0.450279  A
397      2  3.238696  B
398      2  3.412356  A
399      2  3.064870  B

[400 rows x 3 columns]
```



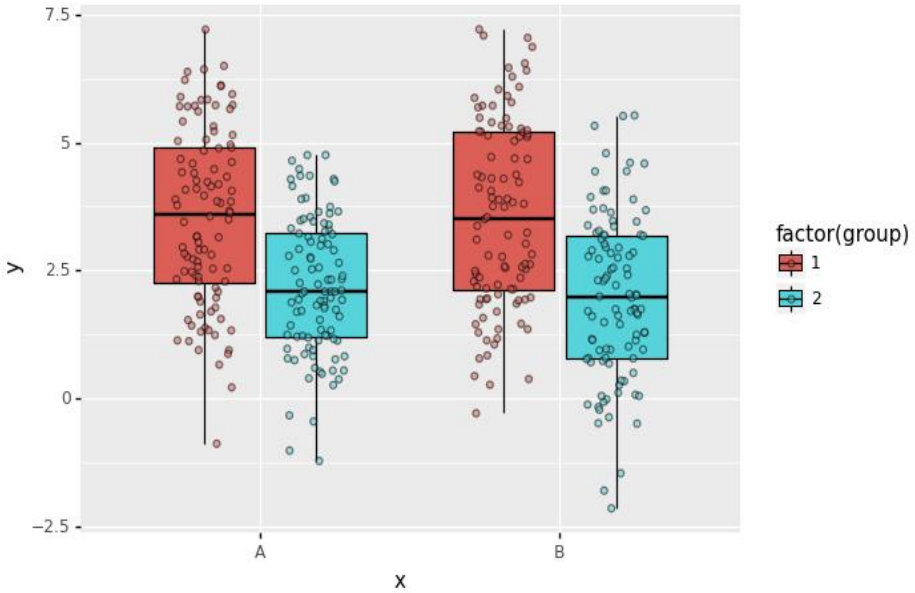

Plotnine: Position Adjustment

```
#----------------Adjust the spacing between the boxplot and the jittered scatter plot----------------------
p1=(ggplot(df,aes(x='x',y='y',fill='factor(group)'))
      +geom_boxplot(outlier_size=0,color='k')
      +geom_jitter(aes(group='factor(group)'),
                        shape='o',alpha=0.5))
print(p1)
```



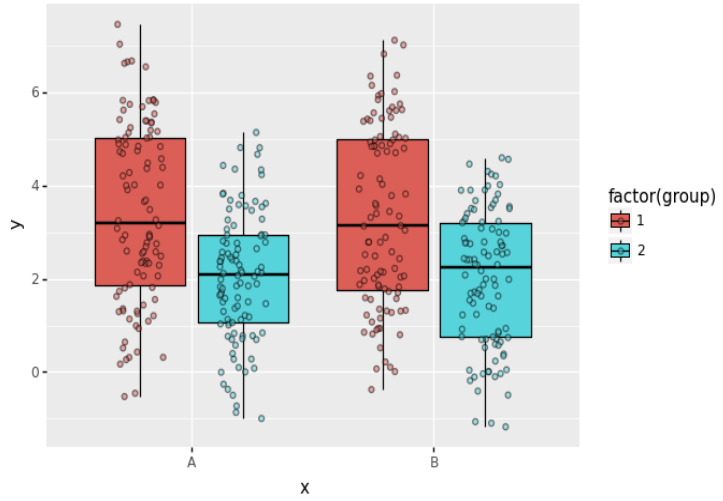
Plotnine: Position Adjustment

```
#-----------------Adjust the spacing of the jittered scatter plot.-----------------------------------
p2=(ggplot(df,aes(x='x',y='y',fill='factor(group)'))
      +geom_boxplot(outlier_size=0,color='k')
      +geom_jitter(aes(group='factor(group)'),
                        shape='o',alpha=0.5,
                        position=position_jitterdodge()))
print(p2)
```



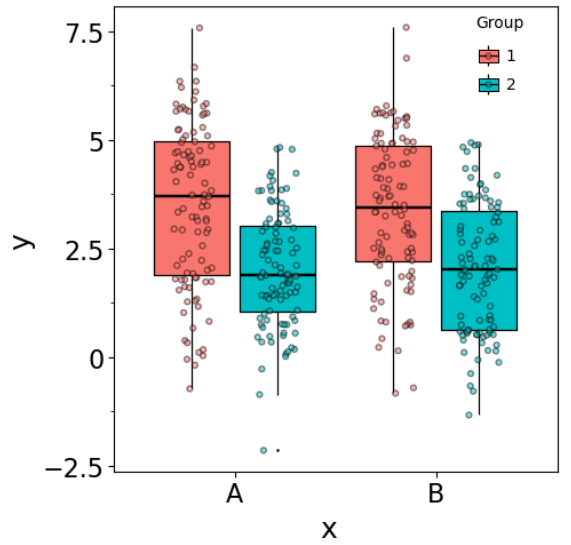
Plotnine: Position Adjustment

```
#----------Simultaneously adjust the spacing between the boxplot and the jittered scatter plot.----------
p3=(ggplot(df,aes(x='x',y='y',fill='factor(group)'))
      +geom_boxplot(position=position_dodge(0.85),
                              outlier_size=0,color='k')
      +geom_jitter(aes(group='factor(group)'),
                          shape='o',alpha=0.5,
                          position=position_jitterdodge(dodge_width=0.85)))
print(p3)
```



Plotnine: Position Adjustment

```
import matplotlib.pyplot as plt
#Simultaneously adjust the spacing between the box plot and the jittered scatter plot (adding themes).
  +scale_fill_manual(values = ("#F8766D","#00BFC4"),guide = guide_legend(title='Group'))
  +theme_matplotlib()
  +theme(
              #text=element_text(size=15,face="plain",color="black"),
               axis_title=element_text(size=18,face="plain",color="black"),
               axis_text = element_text(size=16,face="plain",color="black"),
               legend_position=(0.8,0.8),aspect_ratio =1.05,
               figure_size = (5,5),dpi = 100))
print(base_plot)
```



Sorting for Plotnine

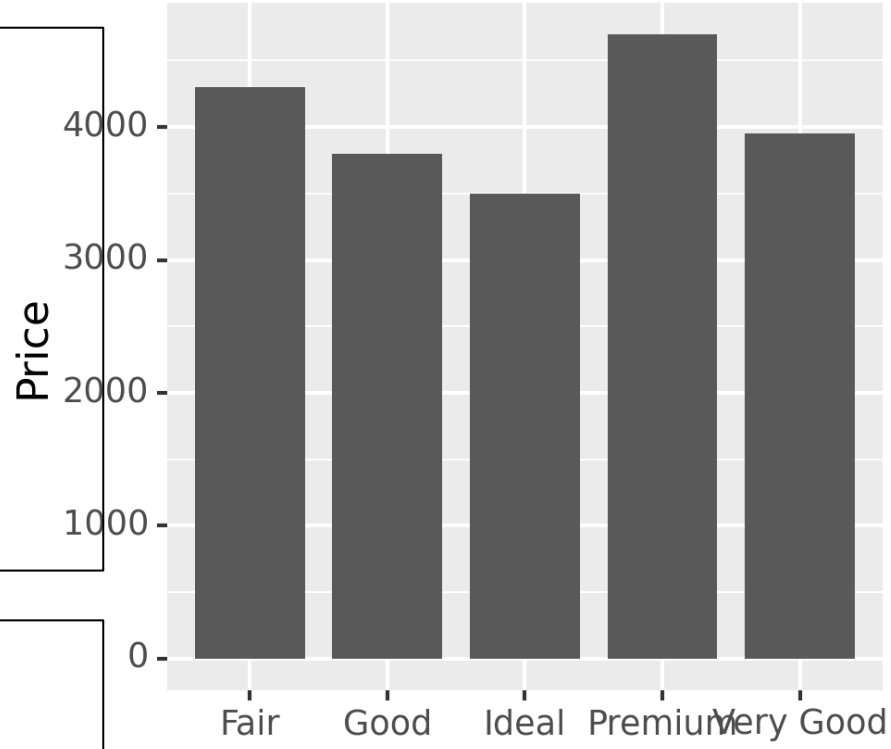


Categorical Data in DataFrame: Application Scenarios

```
import pandas as pd
from plotnine import *

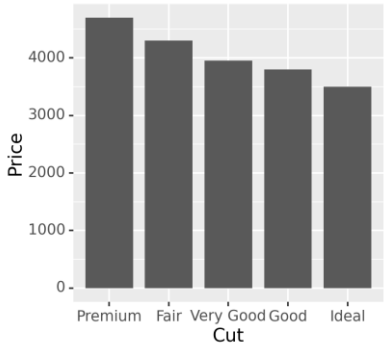
mydata = pd.DataFrame({'Cut': ["Fair", "Good", "Very Good", "Premium",
"Ideal"], 'Price': [4300, 3800, 3950, 4700, 3500]})

(ggplot(mydata , aes(x='Cut', y='Price'))+
        geom_bar(stat="identity", width=0.8)+
         theme(figure_size=(3.2,3),dpi=300))
```




```
Sort_data = mydata.sort_values(by='Price', ascending=False)

(ggplot(Sort_data , aes(x='Cut', y='Price'))+
        geom_bar(stat="identity", width=0.8)+
         theme(figure_size=(3.2,3),dpi=300))
```





Categorical Data in DataFrame: Application Scenarios

```
import pandas as pd
from plotnine import *
from pandas.api.types import CategoricalDtype

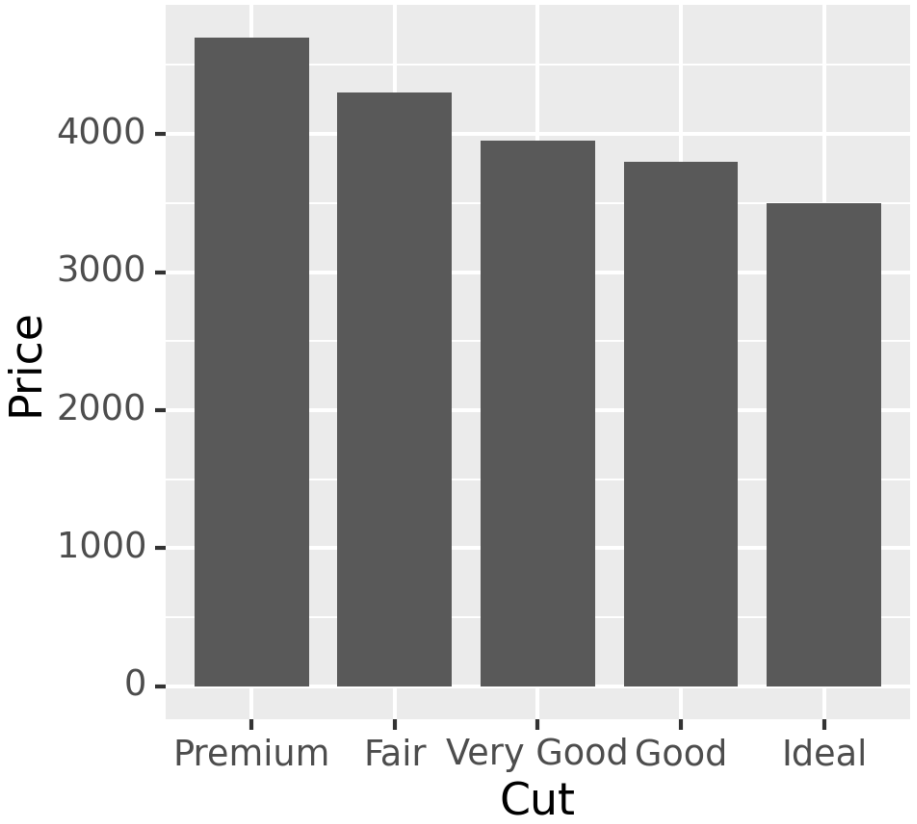
mydata = pd.DataFrame({'Cut': ["Fair", "Good", "Very Good", "Premium",
"Ideal"], 'Price': [4300, 3800, 3950, 4700, 3500]})

Sort_data = mydata.sort_values(by='Price', ascending=False)

#C= ["Premium", "Fair", "Good", "Very Good", "Ideal" ],

Sort_data['Cut'] = Sort_data['Cut'].astype(CategoricalDtype(categories
=Sort_data['Cut'].values,ordered=True))

(ggplot(Sort_data, aes(x='Cut', y='Price'))+
        geom_bar(stat="identity", width=0.8)+
         theme(figure_size=(3.2,3),dpi=300))
```



Categorical Data in DataFrame: Categorical Data Creation

- Pandas has a special data structure type called Categorical, which can be used to hold data that represents categories based on integers. **Categorical data can be divided into categorical and ordinal types**, similar to factor vectors in the R language.
- Categorical data types can be seen as lists that contain additional information, which is the different categories, often referred to as "categories.
- In Python, the categorical data type is crucial in the plotnine package as it determines how data is analyzed and presented visually.

Categorical data, which includes the categorical variables and potential categories, can be managed using the pd.Categorical() function. **By default, when working with character types, categories are arranged alphabetically**: [Fair, Good, Ideal, Premium, Very Good].

```
Cut = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
Cut_Factor1 = pd.Categorical(Cut)
```

If the default alphabetical order is unsatisfactory, **custom categories can be defined to replace the default sorting**. For instance, changing the categories to [Good, Fair, Very Good, Ideal, Premium] can be achieved directly within the pd.Categorical() function.

```
Cut_Factor2 = pd.Categorical(["Fair", "Good", "Very Good", "Premium", "Ideal"],
                             categories=["Good", "Fair", "Very Good", "Ideal", "Premium"], ordered=True)
```



Categorical Data in DataFrame: Categorical Data Modification

For existing categorical data or data frames, you can use the *.astype() function to specify category options and override the default sorting, thereby changing the categories of the categorical data to [Good, Fair, Very Good, Ideal, Premium].

With the update to Pandas version 1.0.3, the original method for changing categories requires using pandas.api.types.CategoricalDtype to define categories. When ordered=True, the categories are ordered as [Good < Fair < Very Good < Ideal < Premium].

```
from pandas.api.types import CategoricalDtype
Cut = pd.Series(["Fair", "Good", "Very Good", "Premium", "Ideal"])
Cut_Factor2 = Cut.astype(CategoricalDtype(categories=["Good", "Fair", "Very Good", "Ideal", "Premium"], ordered=True))
```




Categorical Data in DataFrame: Categorical Data Transformation

We need to obtain the categories and codes of categorical data. This is akin to converting categorical data into numerical data.

```
Cut = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
Cut_Factor1 = pd.Categorical(Cut)
```

```
Cut_Factor2 = pd.Categorical(["Fair", "Good", "Very Good", "Premium", "Ideal"],
                             categories=["Good", "Fair", "Very Good", "Ideal", "Premium"], ordered=True)
```

Statement	Output
Cut_Facor1.codes	array([0, 1, 4, 3, 2], dtype=int8)
Cut_Facor1.categories	Index(['Fair', 'Good', 'Ideal', 'Premium', 'Very Good'], dtype='object')
Cut_Facor2.array.codes	array([1, 0, 2, 4, 3], dtype=int8)
Cut_Facor2.array.categories	Index(['Fair', 'Good', 'Ideal', 'Premium', 'Very Good'], dtype='object')



The beauty of data visualization

https://www.ted.com/talks/david_mccandless_the_beauty_of_data_visualization?subtitle=en



Thanks

